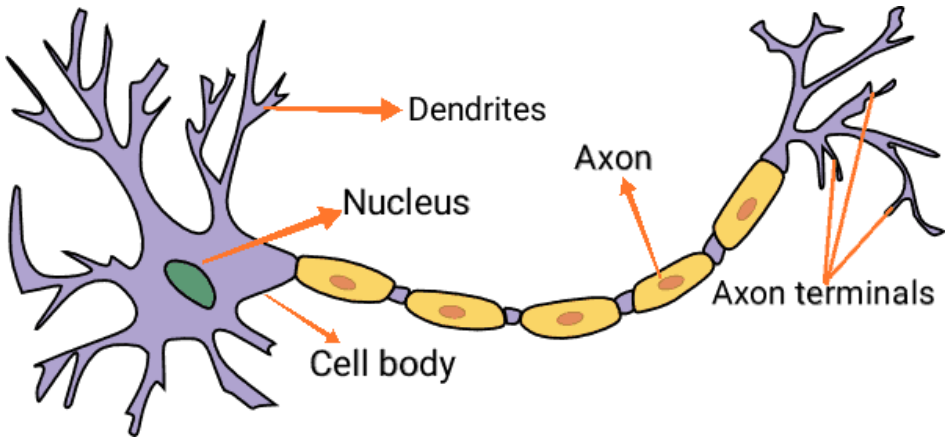
Artificial Neural Networks

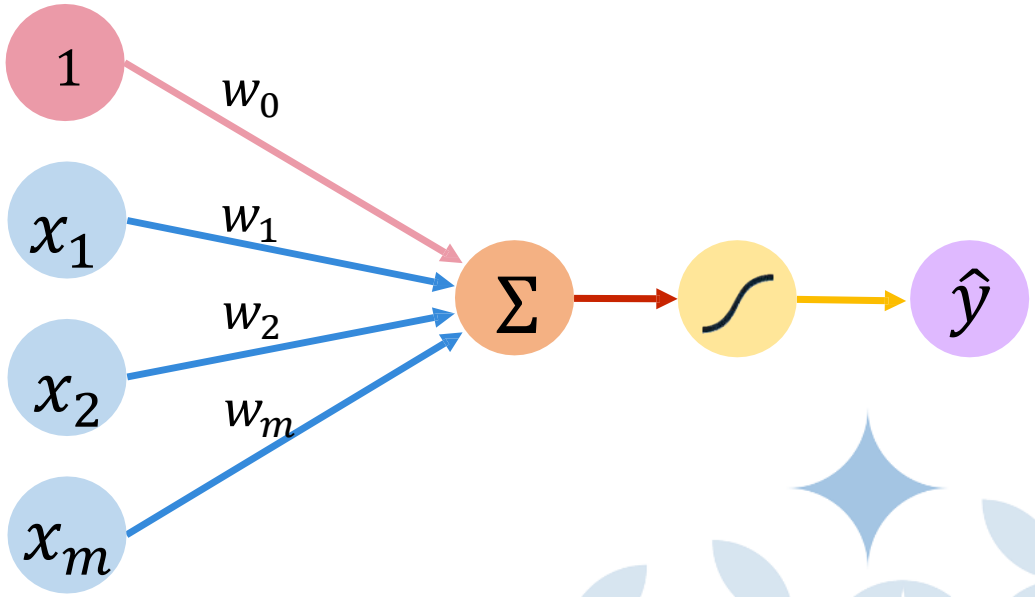
Qiang Sun
University of Toronto

Artificial Neural Network

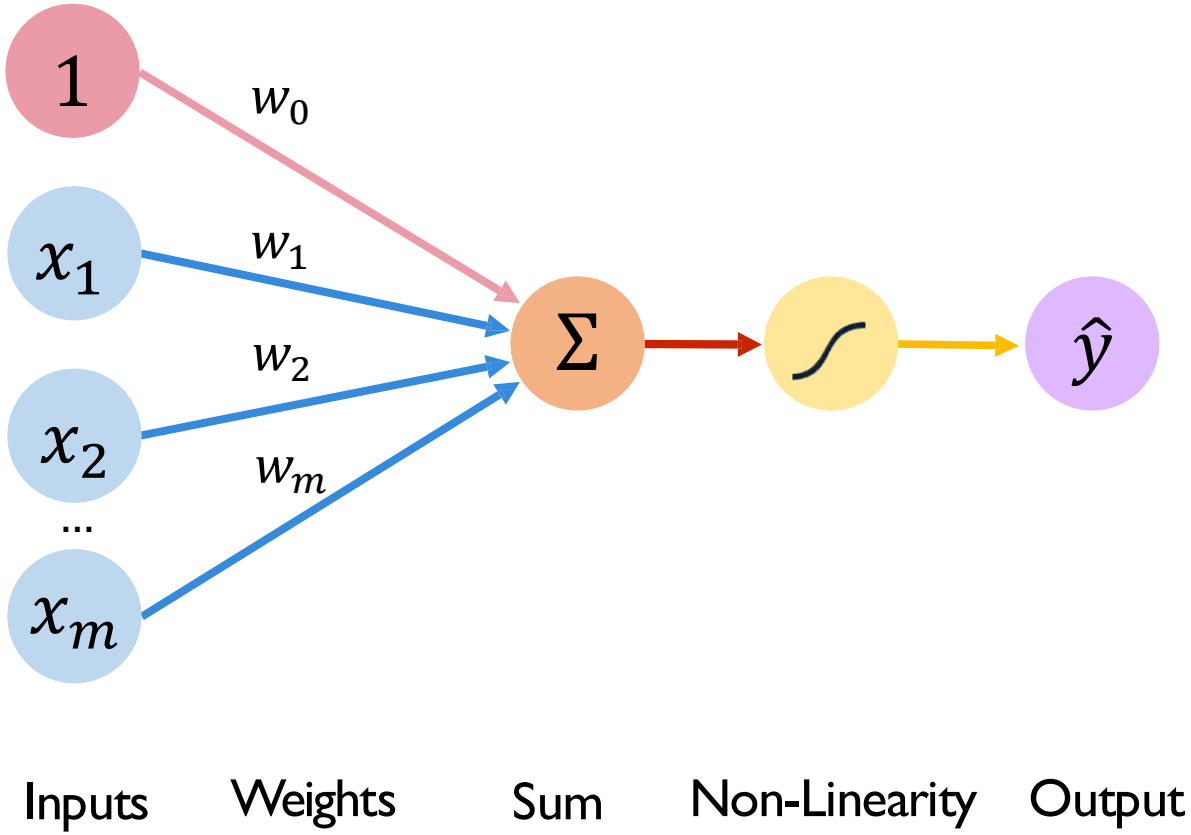
- A network of artificial neurons that mimics real biological neural networks
  - Each has a body, axon, and many dendrites
  - A neuron can fire or rest
  - A threshold to activate (fire)



- Perceptron (Frank Rosenblatt, 1956)
  - Fundamental building block of deep learning
  - What is Perceptron?
  - How is it defined?
  - Build deep NN from a Perceptron



The Perceptron – A single Neuron



Output

Linear combination of inputs

$$\hat{y} = g\left(w_0 + \sum_{i=1}^{m} x_i w_i\right)$$

Non-linear activation function

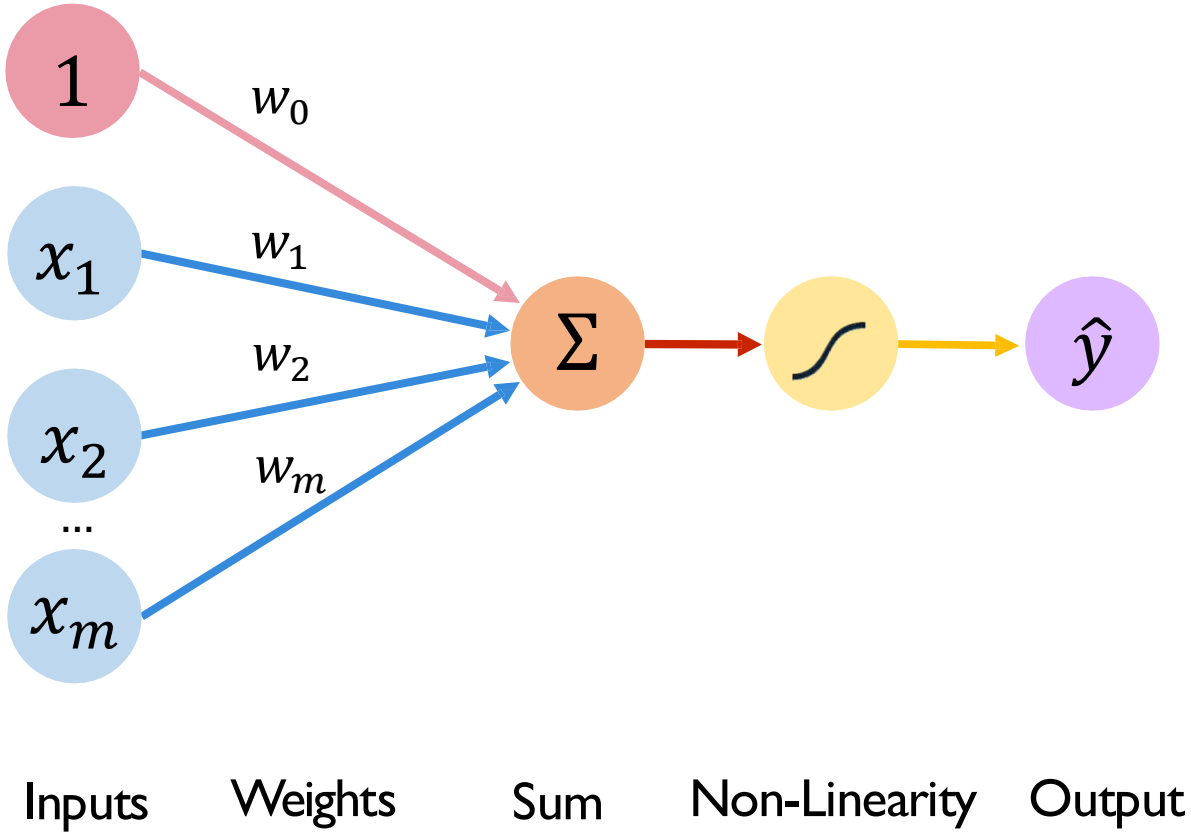
Bias

Binary Classification:

$(x_1, \dots, x_m)$ feature of a data

$\hat{y} \in [0,1]$: probability belongs to the positive class

The Perceptron

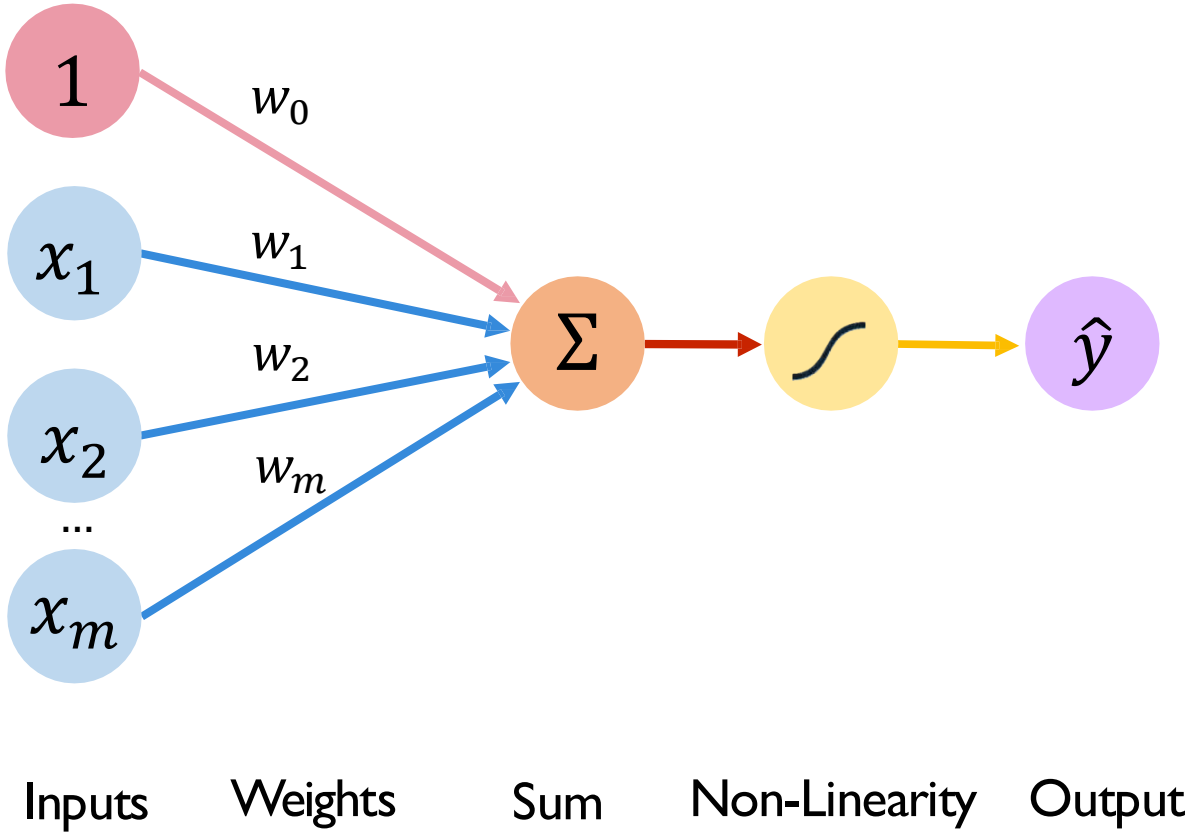


$$\hat{y} = g(w_0 + \sum_{i=1}^{m} x_i w_i)$$

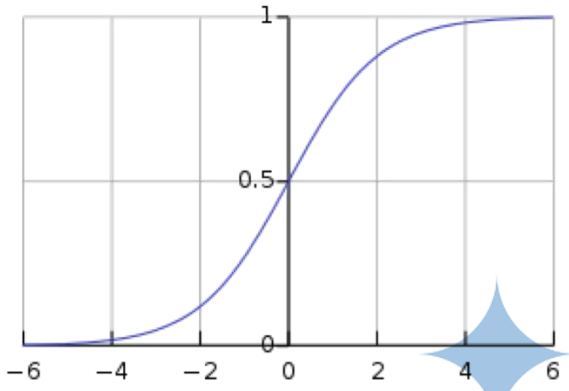
$$\hat{y} = g(w_0 + X^\top W)$$

$$\text{where: } \boldsymbol{X} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix} \text{ and } \boldsymbol{W} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}$$

The Perceptron



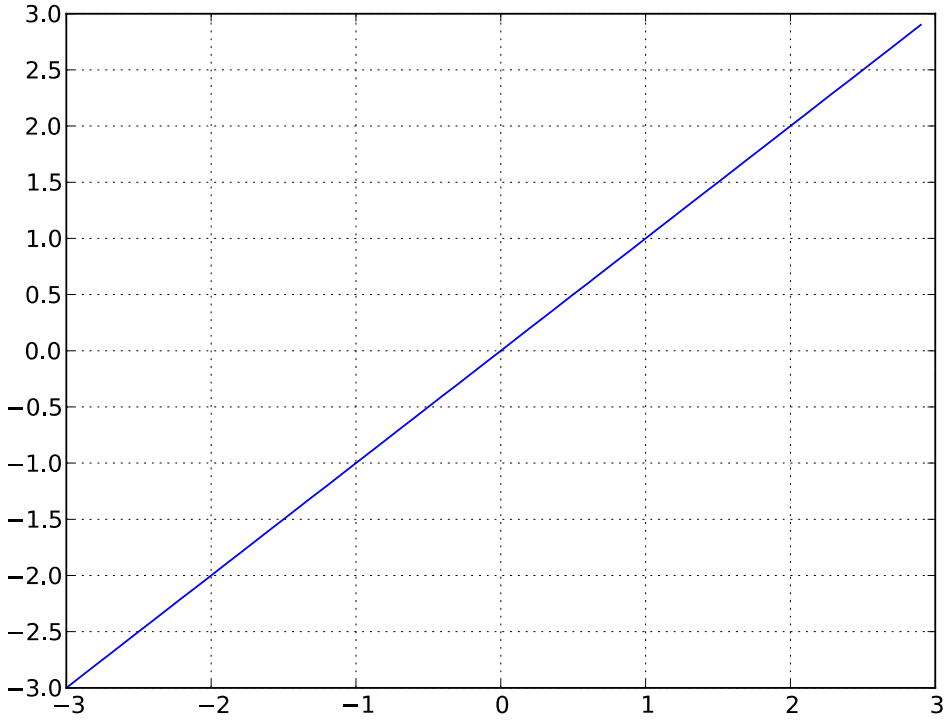
$$\hat{y} = g\left(w_0 + \sum_{i=1}^{m} x_i w_i\right)$$



Activation Functions

Linear activation function:

- $g(a) = a$
- Correspond to linear regression



Non-linear Activation Functions

Sigmoid activation function:

- Squashes the neuron's output between 0 and 1

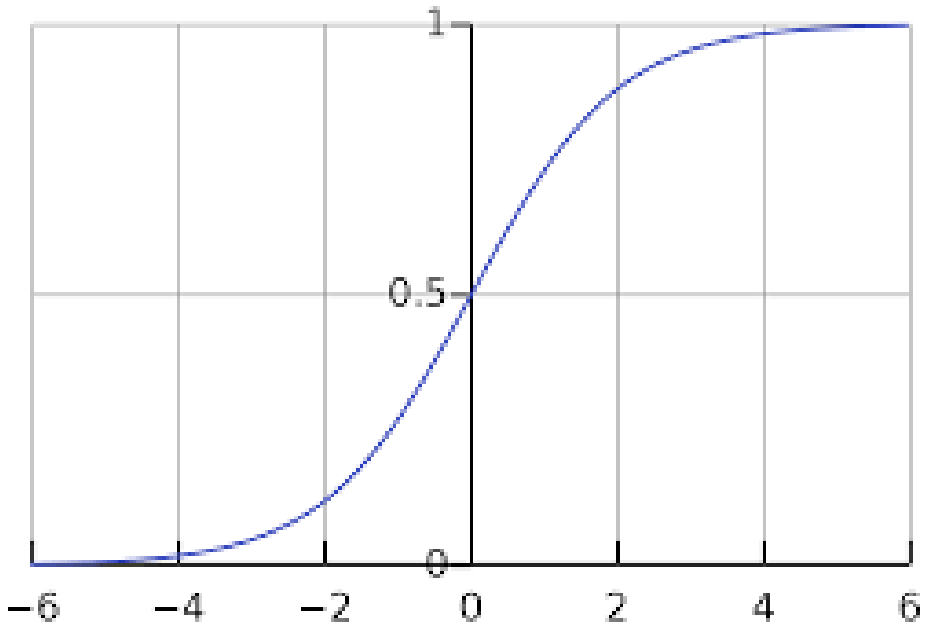
$$g(a) = \sigma(a) = \frac{\exp(a)}{1 + \exp(a)} = \frac{1}{1 + \exp(-a)}$$

- Always positive and strictly increasing

- Naturally suitable for probability

  - Logistic Regression

$$\hat{y} = P(Y = 1|X) = \frac{\exp(w_0 + \sum_i w_i X_i)}{1 + \exp(w_0 + \sum_i w_i X_i)}$$

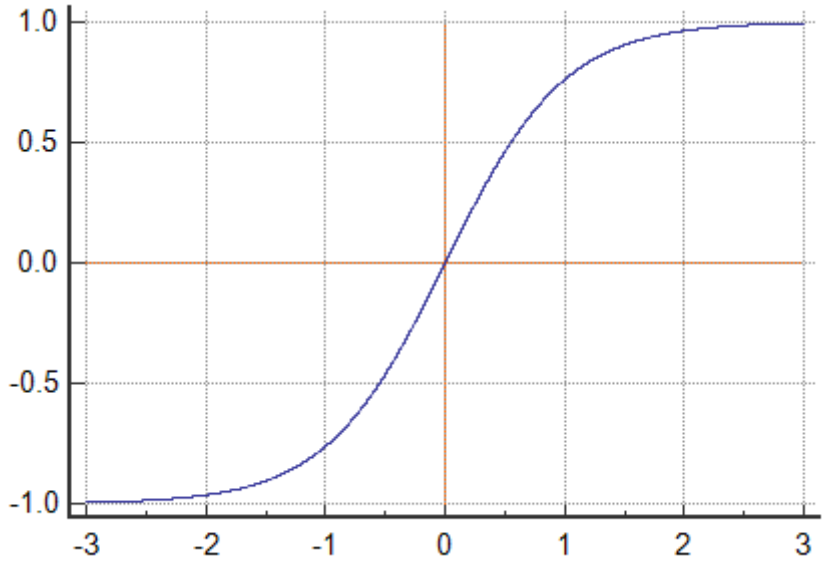


Non-linear Activation Functions

Hyperbolic tangent ("tanh") activation function:

$$g(a) = \tanh(a) = \frac{\exp(a) - \exp(-a)}{\exp(a) + \exp(-a)} = \frac{\exp(2a) - 1}{\exp(2a) + 1}$$

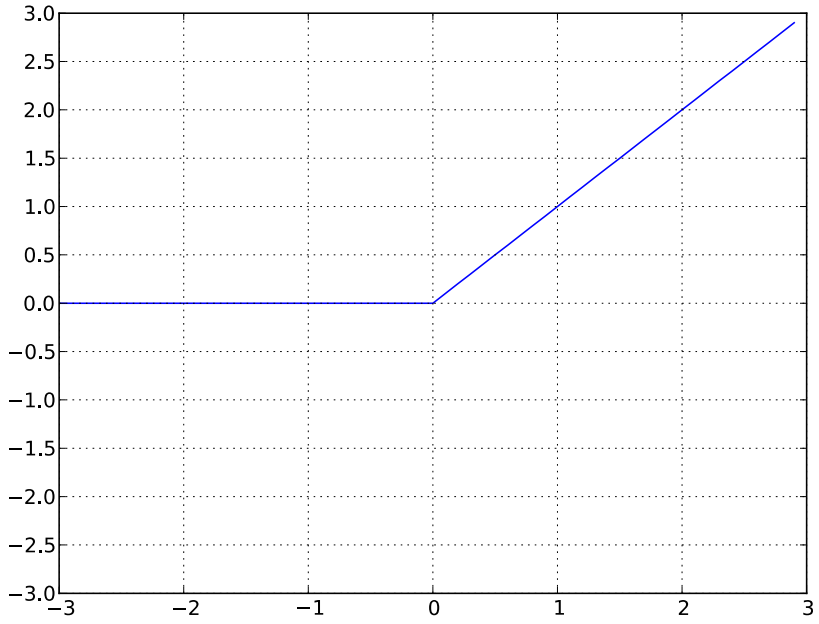
- Squashes the neuron's output between -1 and 1
- Strictly Increasing



Non-linear Activation Functions

Rectified linear Unit (ReLU), activation function:

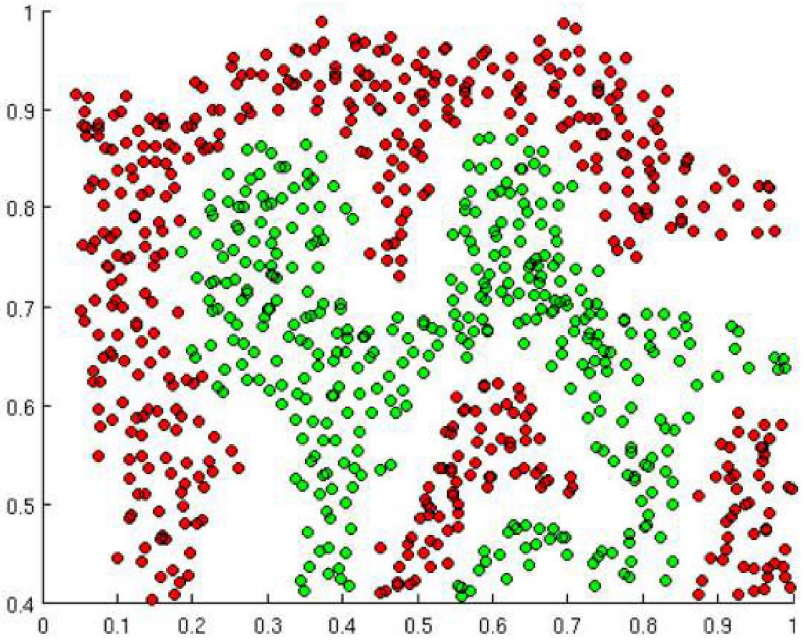
- Rectified linear Unit (ReLU), activation function:
  - Nair and Hinton (2010)
  - Bounded below by 0 (always non-negative)
  - Not upper bounded
  - Not smooth
  - other variants of ReLU (Leaky ReLU)



https://adl1995.github.io/an-overview-of-activation-functions-used-in-neural-networks.html

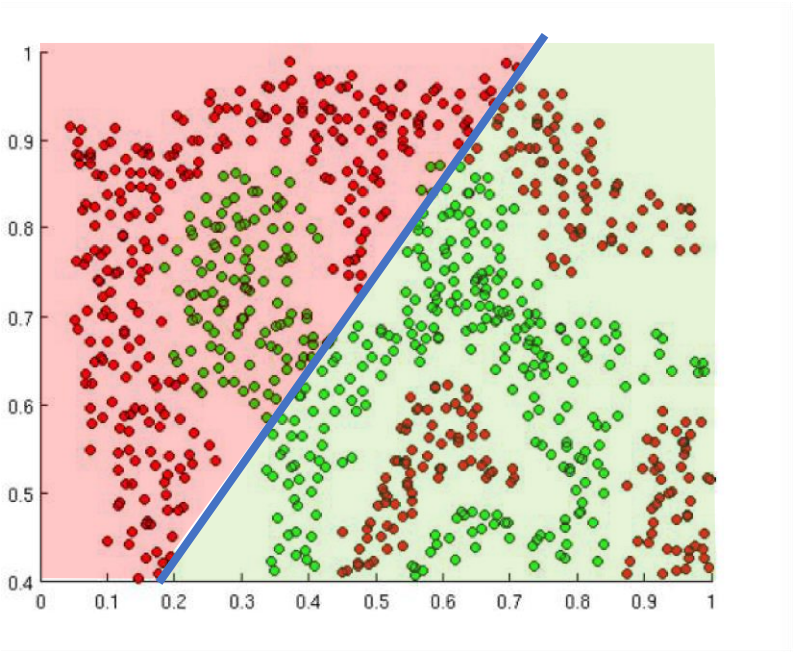
Importance of Activation Function

Non-linearity

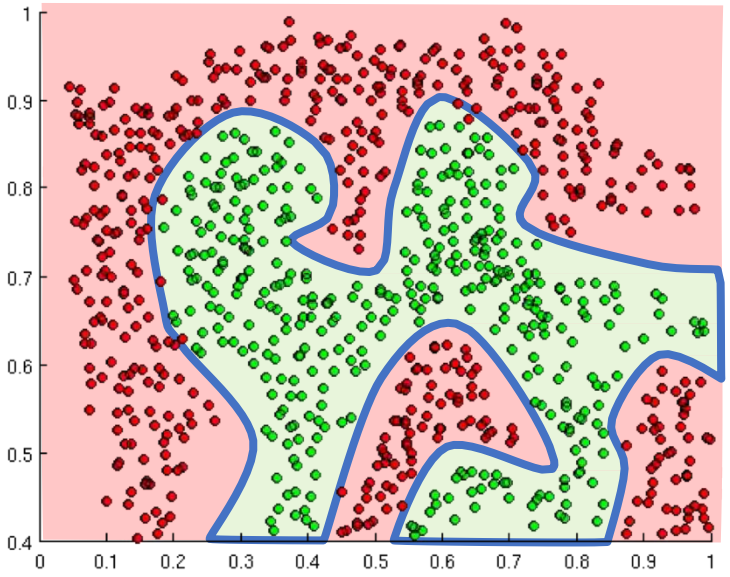


We want to build a Neural Network to distinguish green vs red points

More complex neural networks

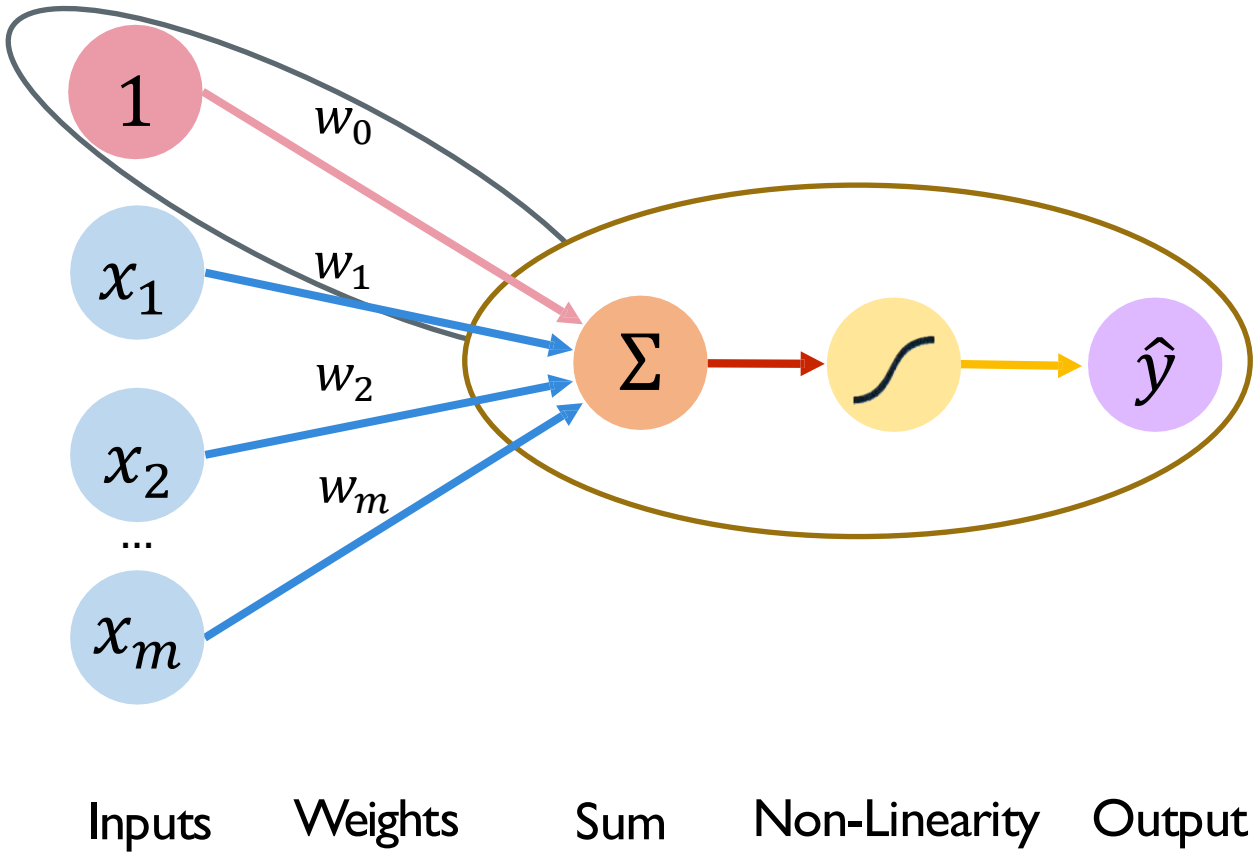


Linear activation function produce linear decision boundaries

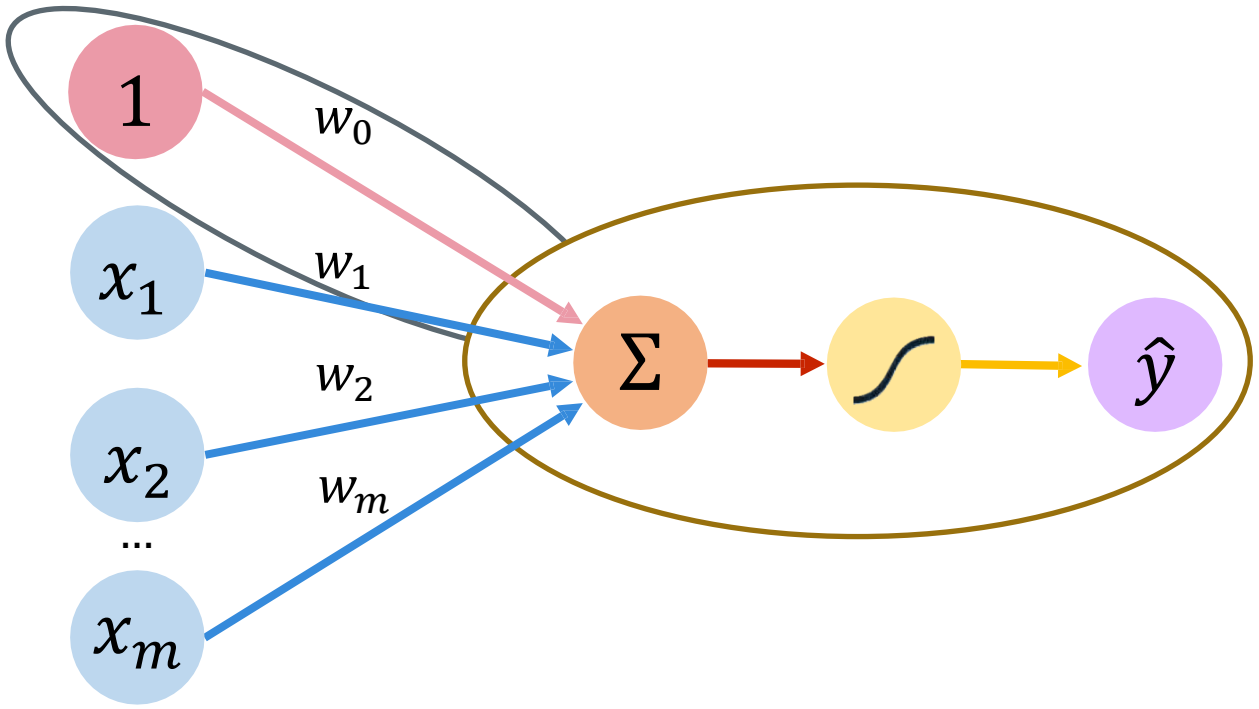


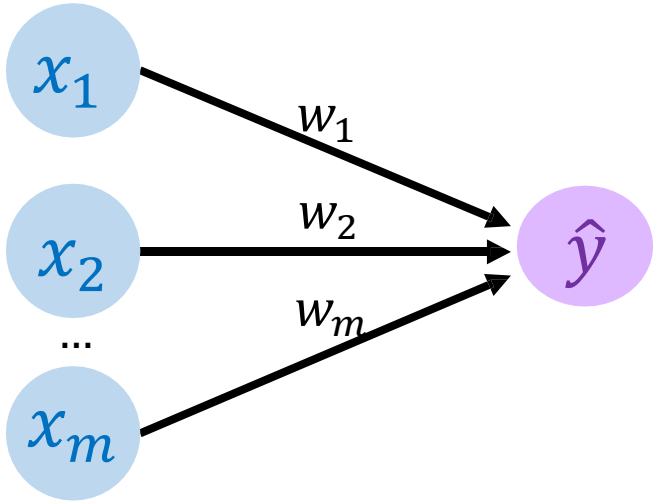
Non-linearity allows us to approximate complex functions

The Perceptron: Simplified



The Perceptron: Simplified

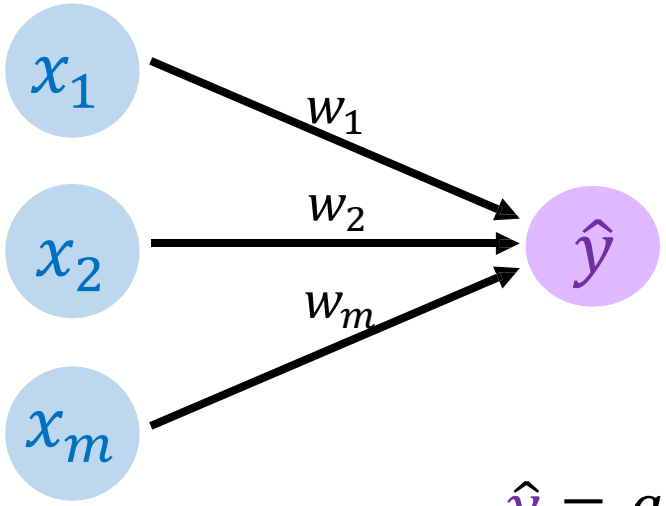




Inputs Weights Sum Non-Linearity Output

$$\hat{y} = g(w_0 + \sum_{i=1}^{m} x_i w_i)$$

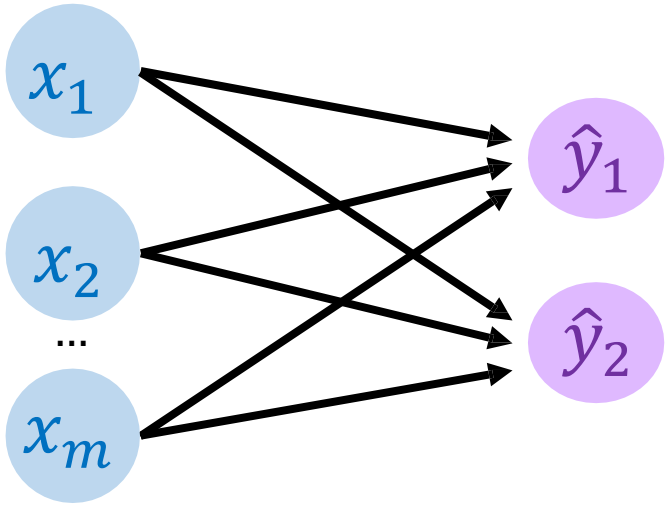
Single Output Perceptron



$$\hat{y} = g(w_0 + \sum_{i=1}^{m} x_i w_i)$$

What if we have multi-output $\hat{y}_1$ and $\hat{y}_2$?

Multi Output Perceptron

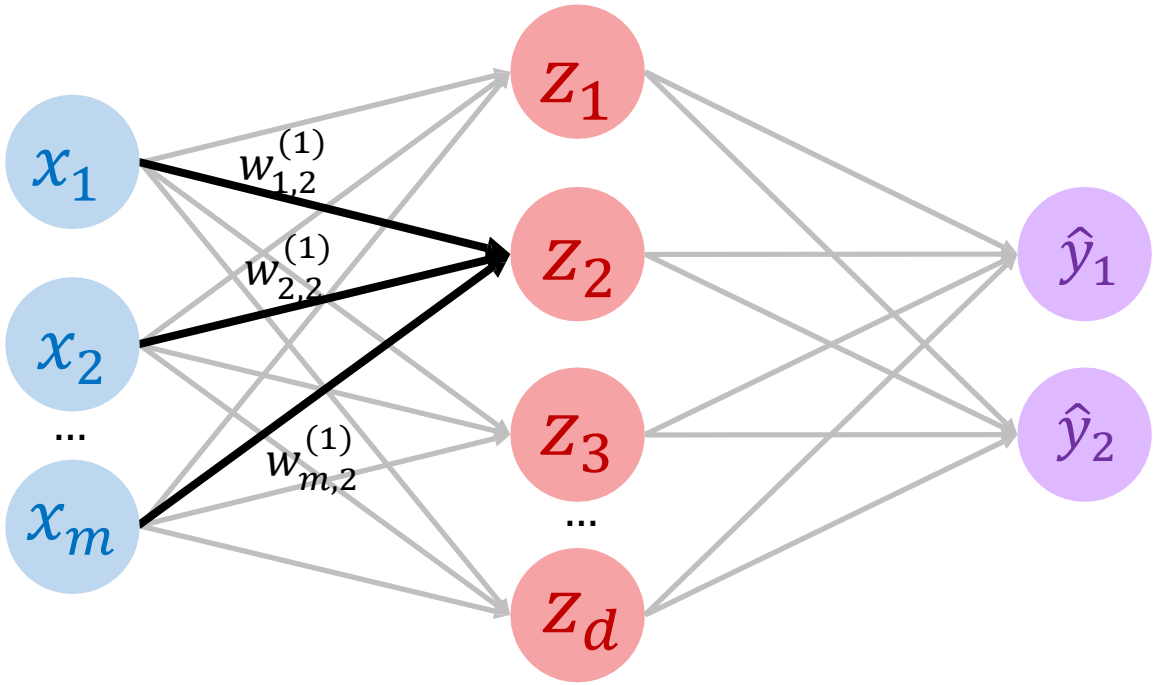


$$= g(w_{0,1} + \sum_{i=1}^{m} x_i w_{i,1})$$

$$= g(w_{0,2} + \sum_{i=1}^{m} x_i w_{i,2})$$

Output $\hat{y}_j = g(w_{0,j} + \sum_{i=1}^{m} x_i w_{i,j})$

Extend to One Hidden Layer Neural Network (Two Layer NN)

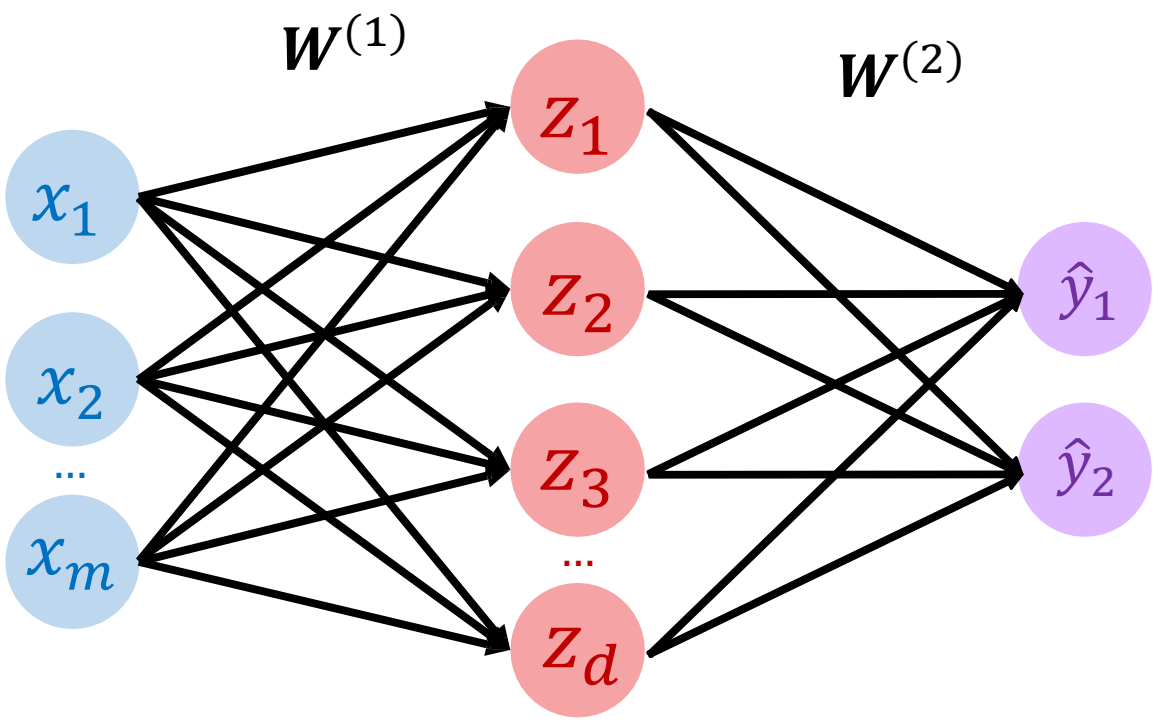


$$z_2 = g(w_{0,2}^{(1)} + \sum_{i=1}^{m} x_i w_{i,2}^{(1)})$$

$$= g(w_{0,2}^{(1)} + x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + \cdots + x_m\, w_{m,2}^{(1)})$$

(1) Indicates the 1st layer

One Hidden Layer Neural Network (Two Layer NN)



Parameters to be learned

$$\boldsymbol{W} = (\boldsymbol{W}^{(1)}, \boldsymbol{W}^{(2)})$$

Inputs

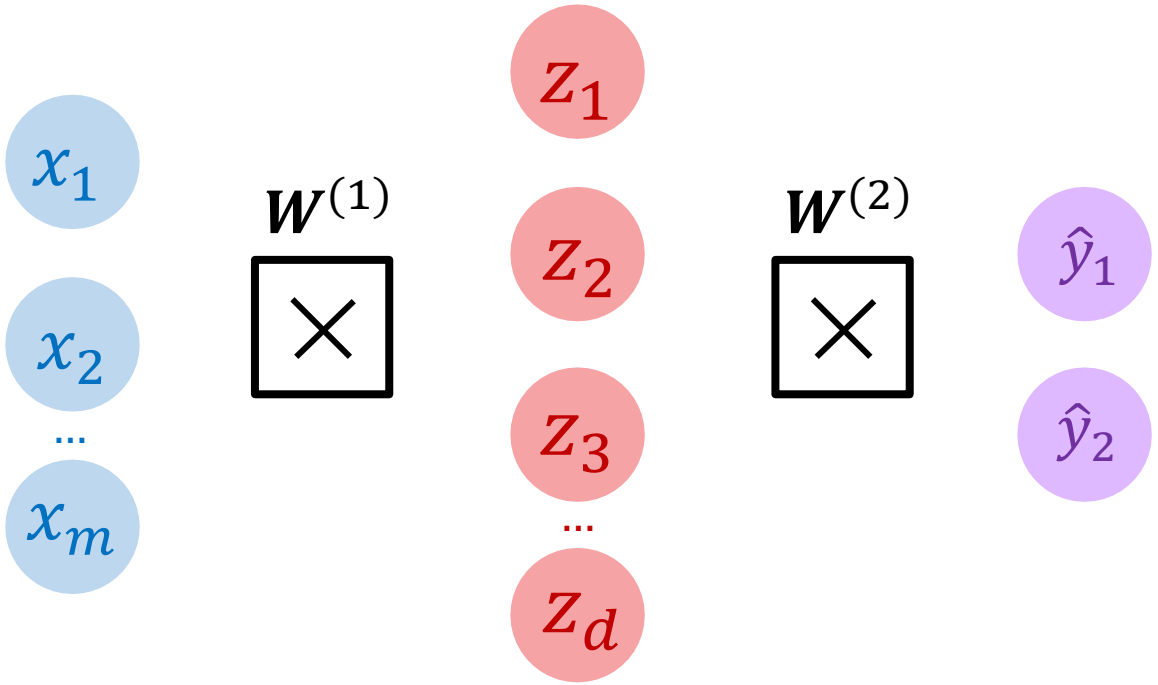
Hidden

Final Output

$$z_j = g(w_{0,j}^{(1)} + \sum_{i=1}^{m} x_i w_{i,j}^{(1)})$$

$$\hat{y}_k = g(w_{0,k}^{(2)} + \sum_{j=1}^{d} z_j w_{j,k}^{(2)})$$

One Hidden Layer Neural Network (Two Layer NN)

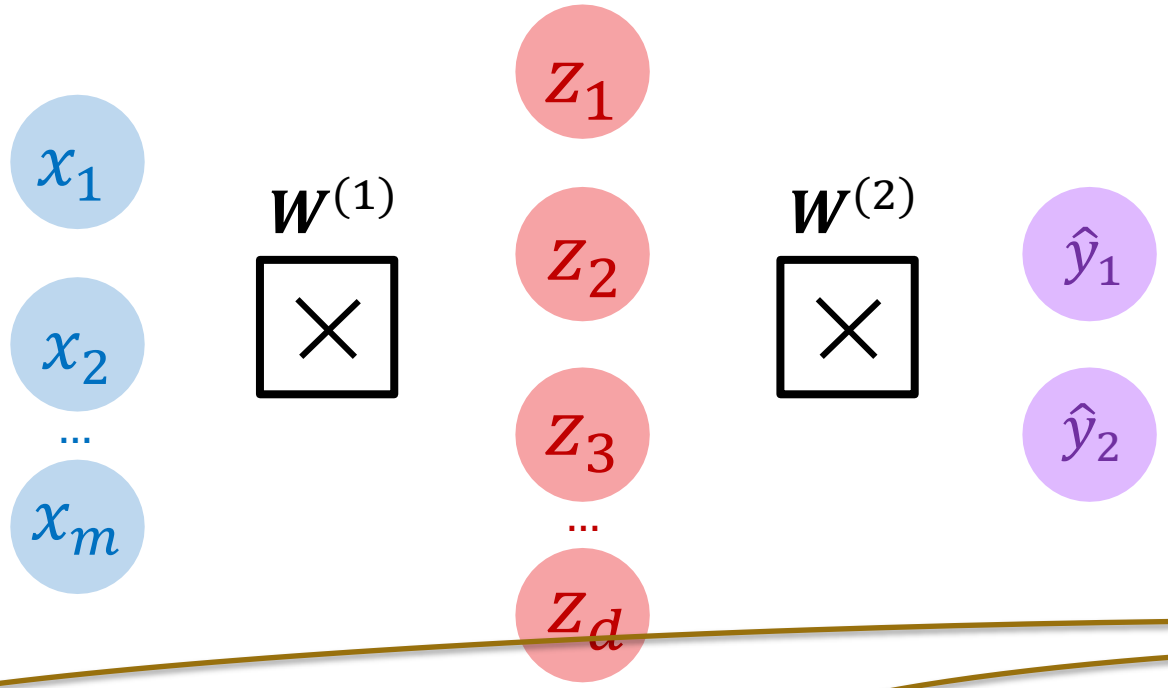


Inputs Hidden Final Output

$$z_j = g(w_{0,j}^{(1)} + \sum_{i=1}^{m} x_i w_{i,j}^{(1)})$$

$$\hat{y}_k = g(w_{0,k}^{(2)} + \sum_{j=1}^{d} z_j w_{j,k}^{(2)})$$

One Hidden Layer Neural Network (Two Layer NN)



Activation functions can be different in different layers

Inputs

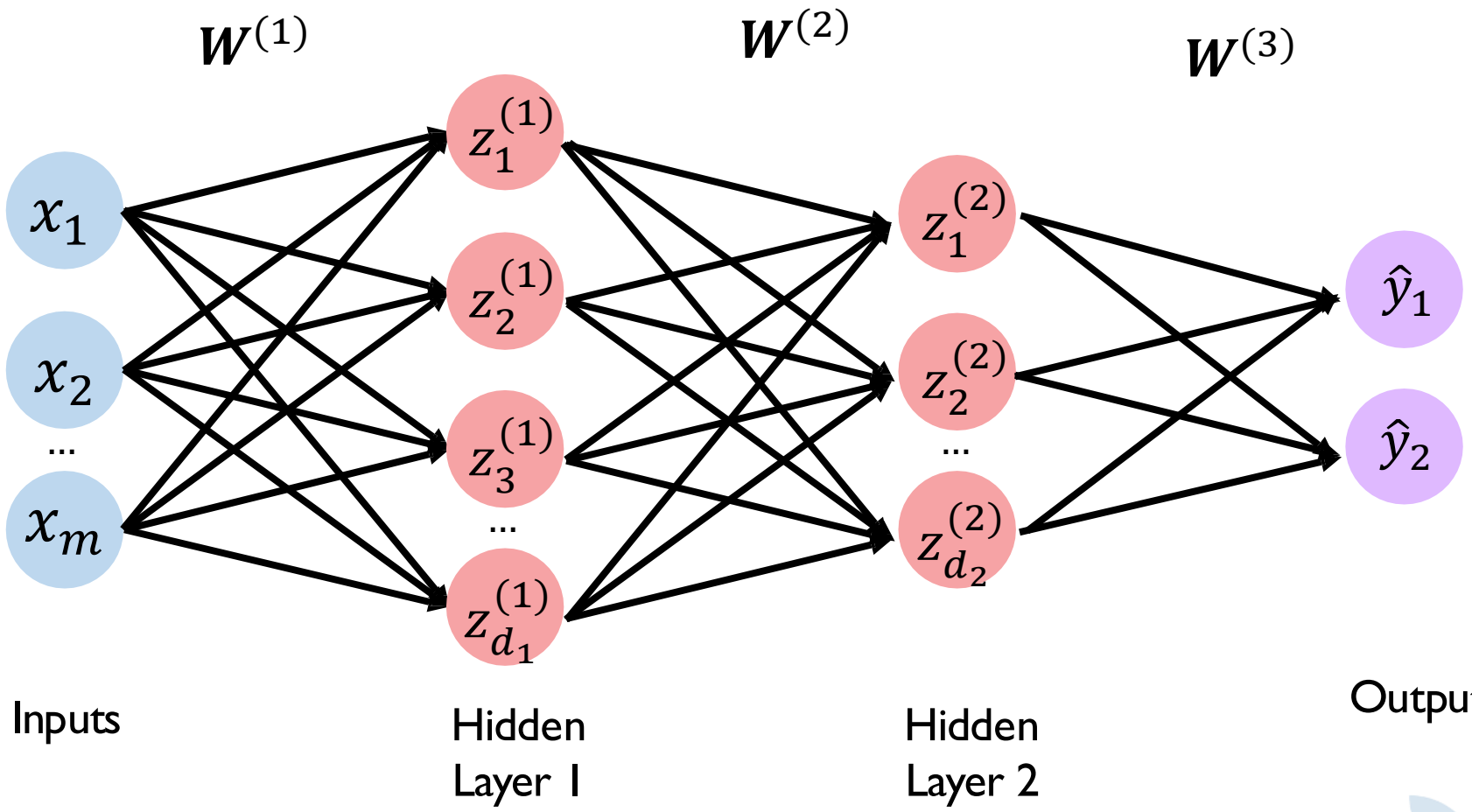
Hidden

Final Output

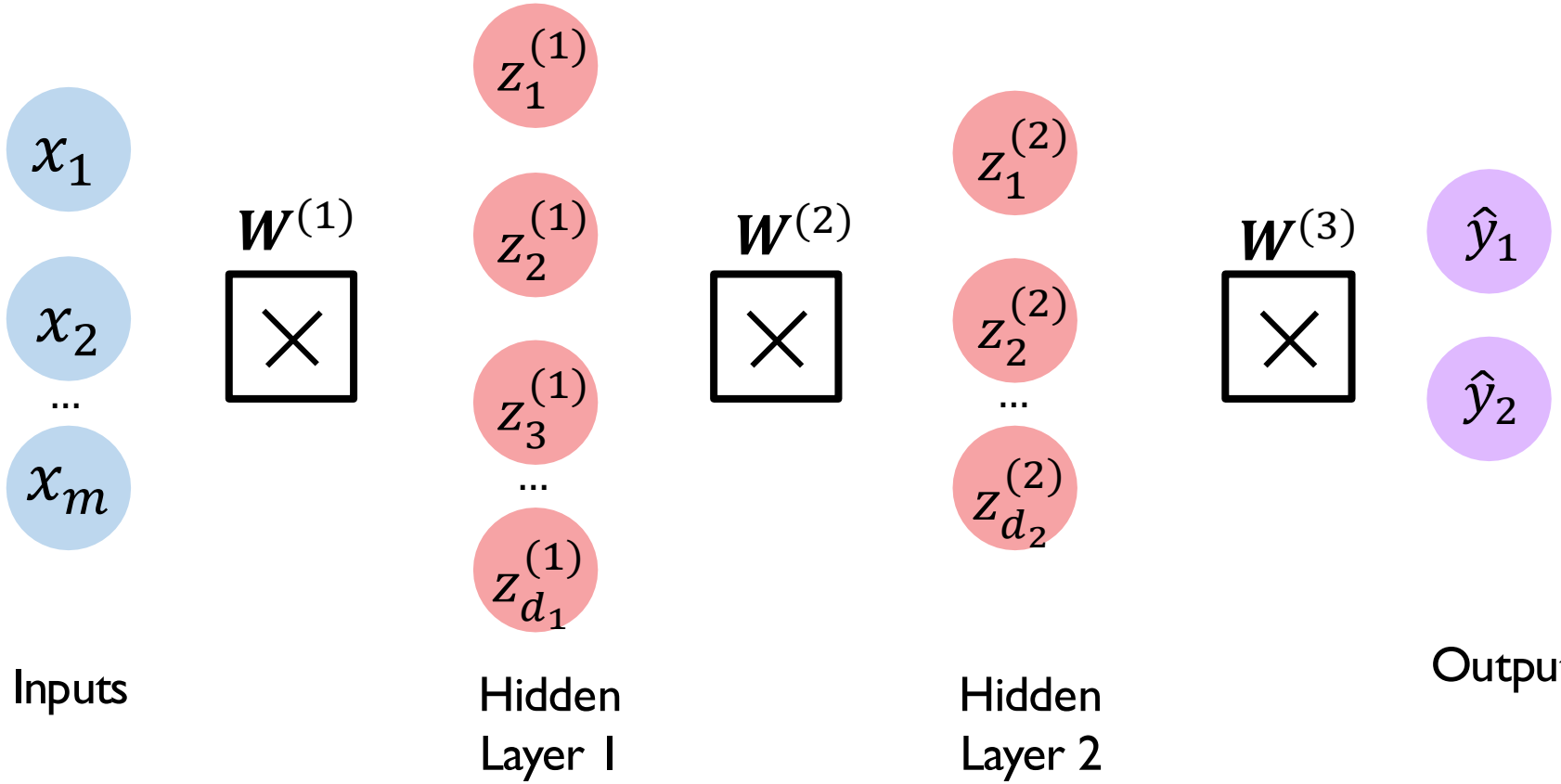
$$z_j = g(w_{0,j}^{(1)} + \sum_{i=1}^{m} x_i w_{i,j}^{(1)})$$

$$\hat{y}_k = g(w_{0,k}^{(2)} + \sum_{j=1}^{d} z_j w_{j,k}^{(2)})$$

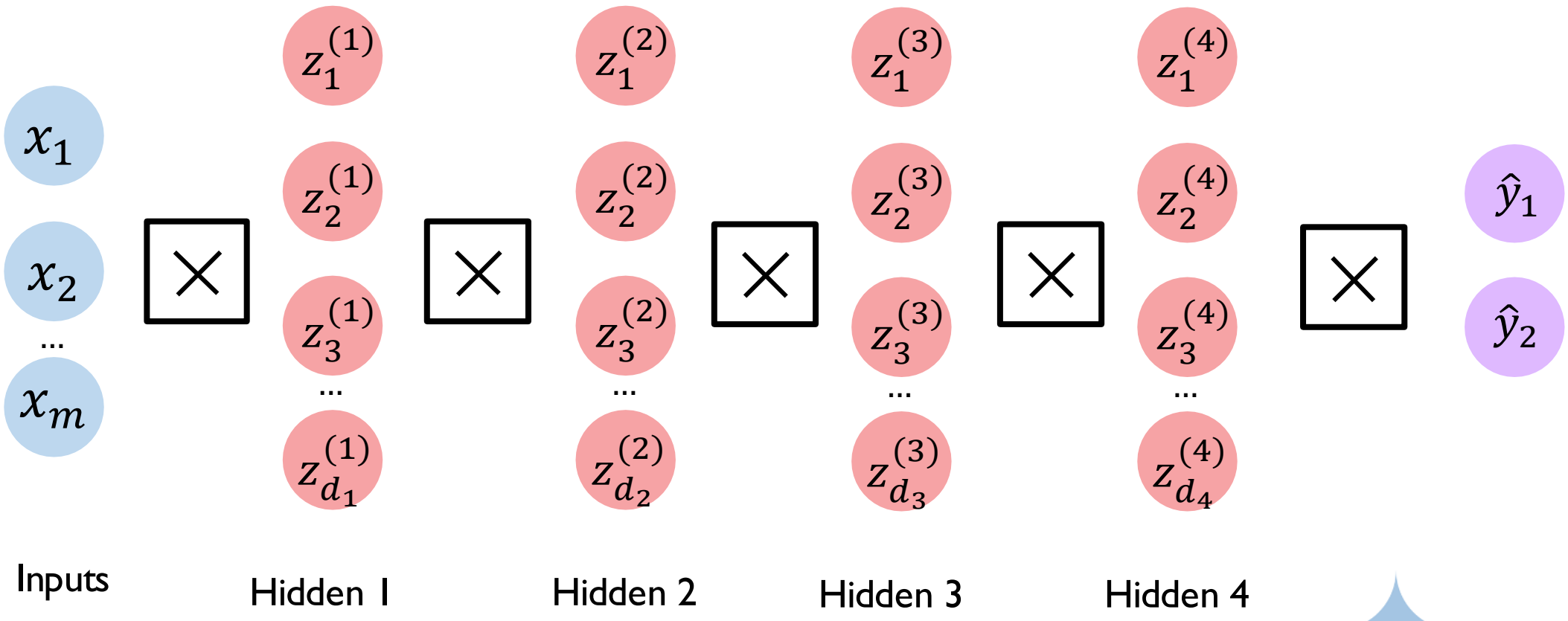
Two Hidden Layer Neural Network (Three Layer NN)



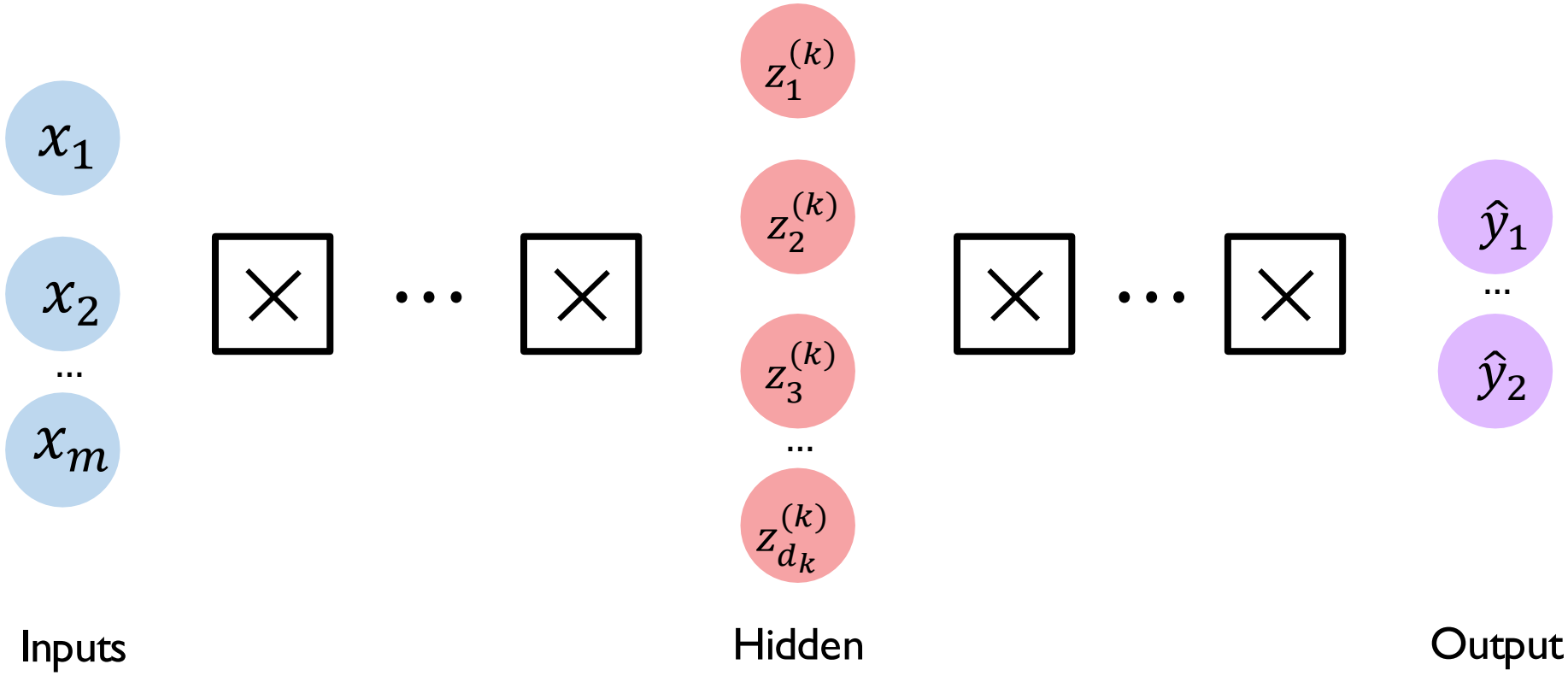
Two Hidden Layer Neural Network (Three Layer NN)



Multi-Layer (Deep) Neural Network

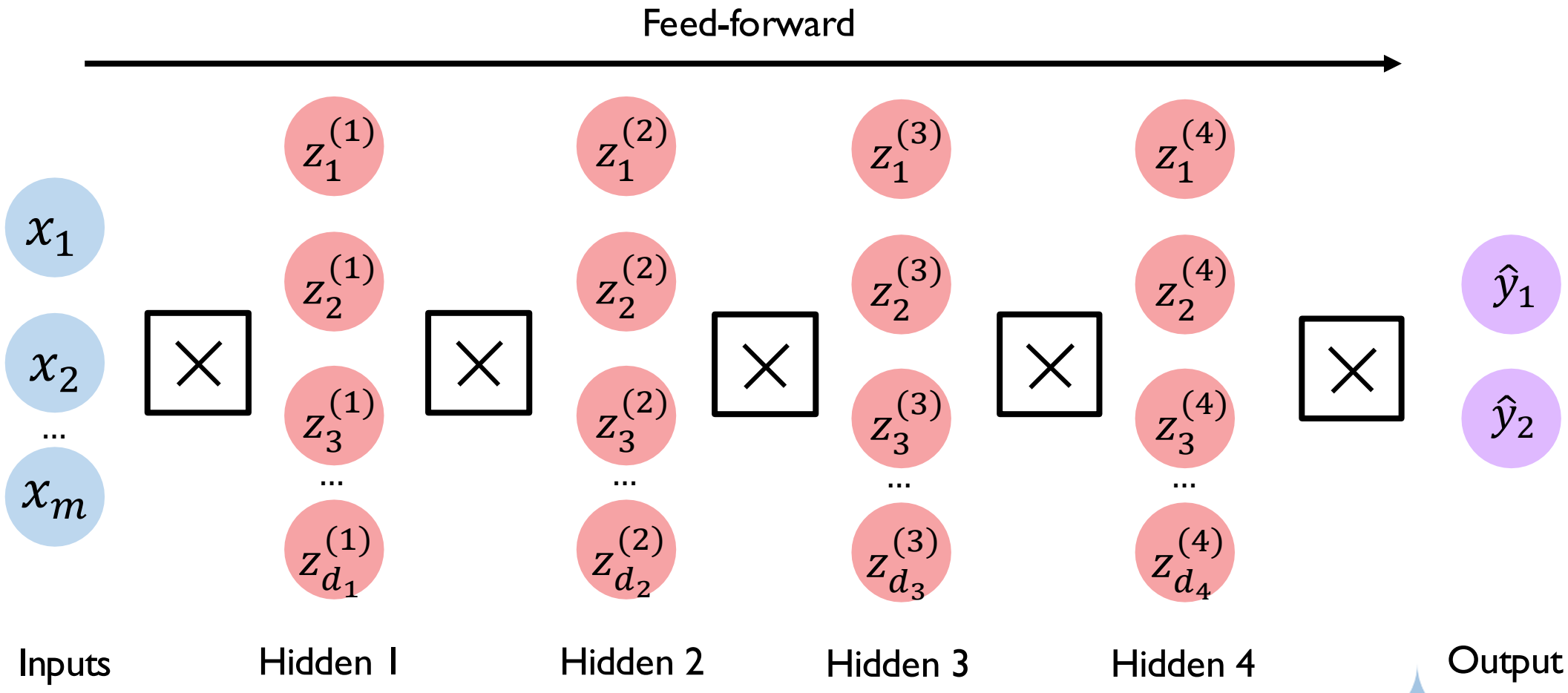


Deep Neural Network (DNN)



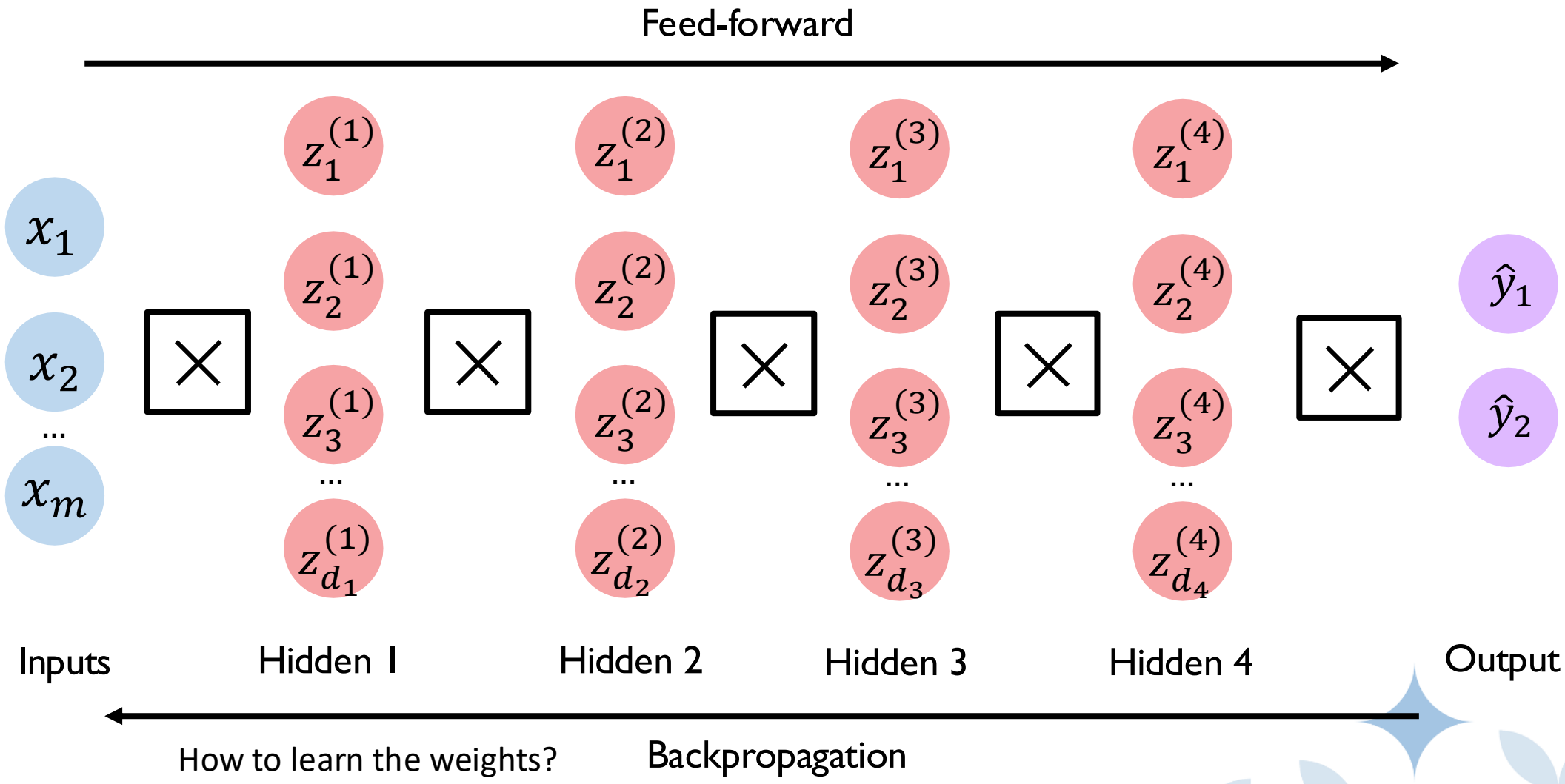
$$z_j^{(k)} = g\left(w_{0,j}^{(k)} + \sum_{i=1}^{d_{k-1}} z_i^{(k-1)} w_{i,j}^{(k)}\right)$$

Feed-forward Networks and Backpropagation



How to learn the weights?

Feed-forward Networks and Backpropagation



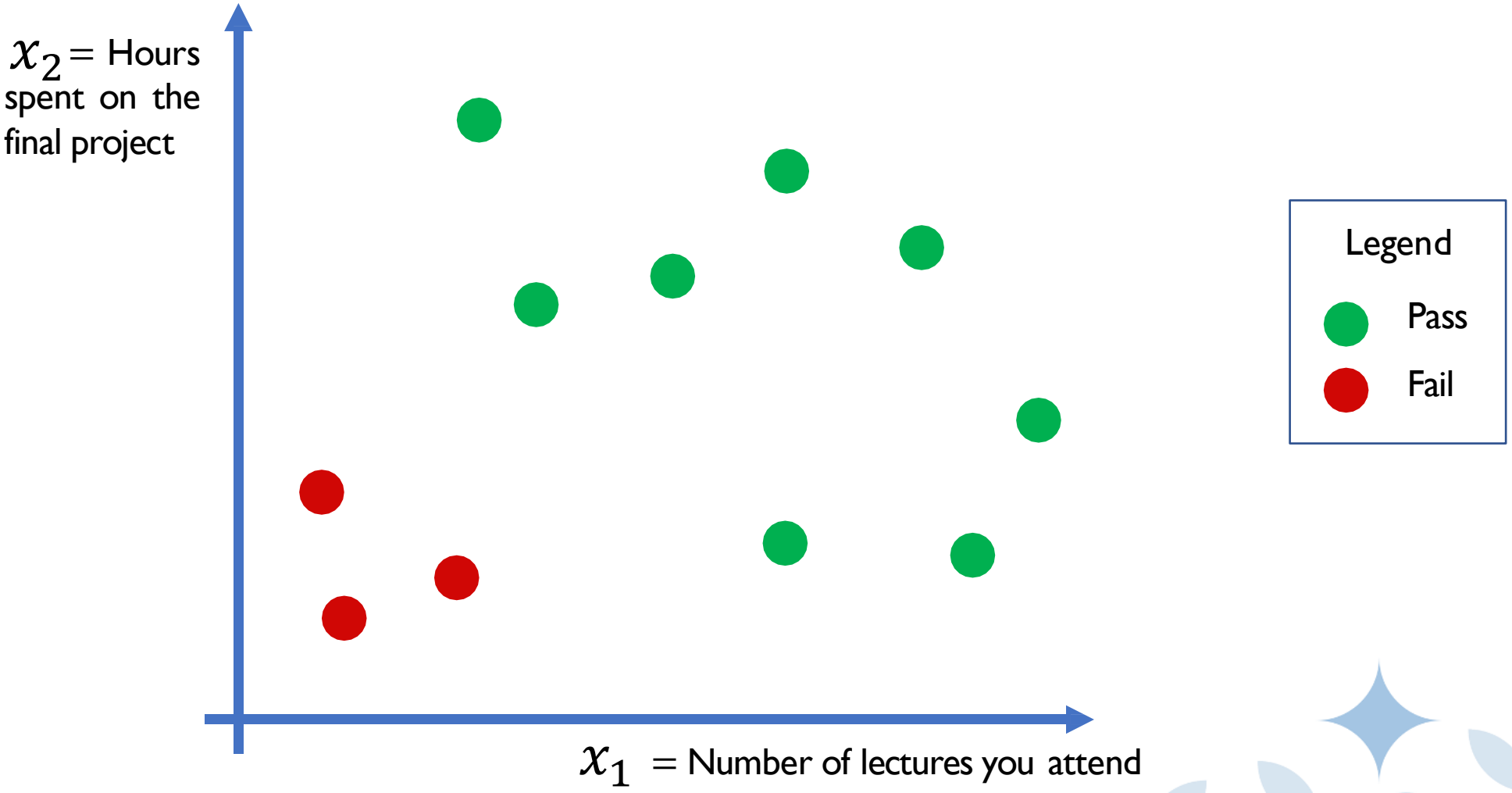
Example Problem

Will I pass the class?

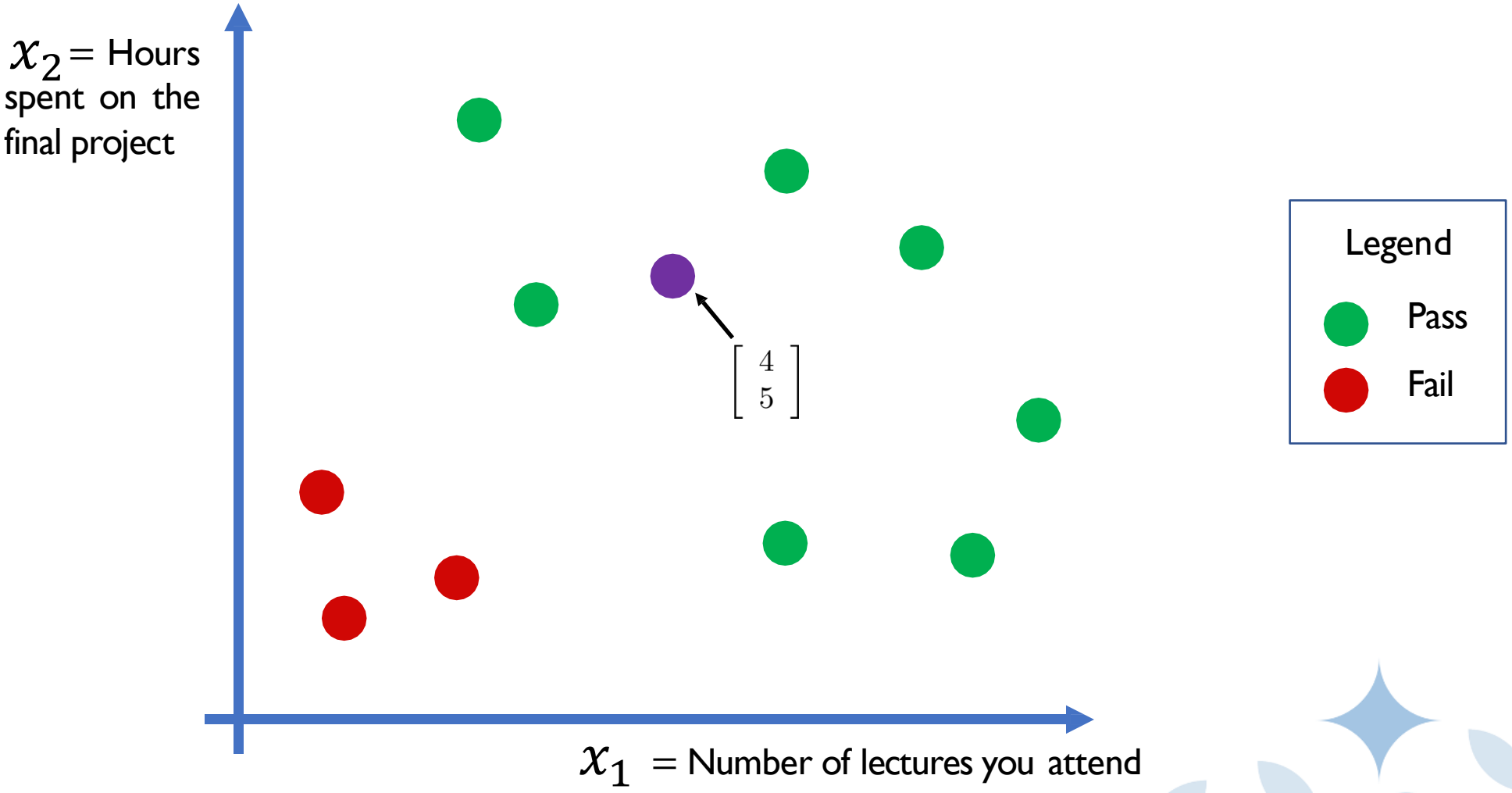
– Let's start with a simple two-feature model

- x_1 = Number of lectures you attend
- x_2 = Hours spent on the final project

Example Problem: Will I pass the class?



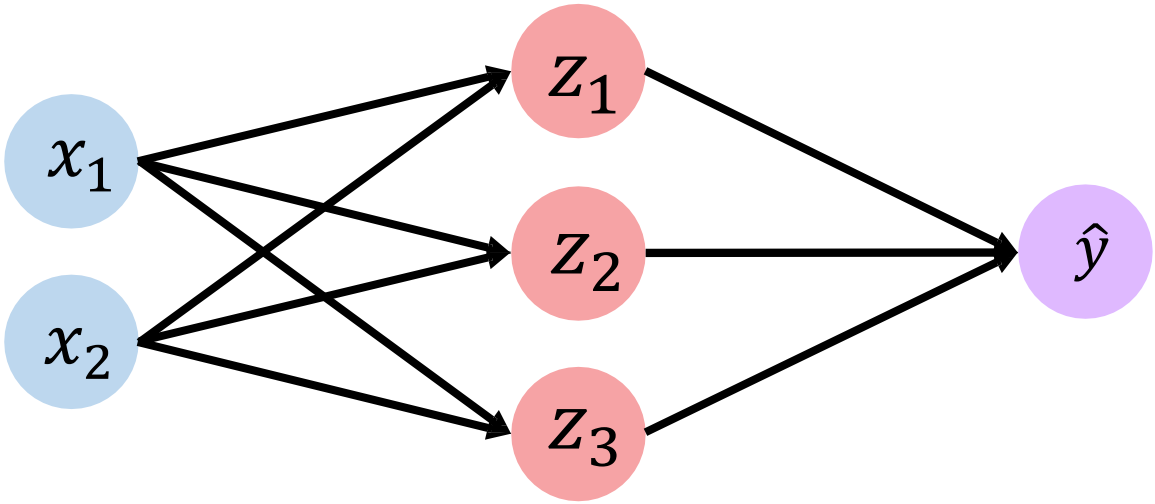
Example Problem: Will I pass the class?



Initialization

Initialize the network to realize the first feed-forward pass

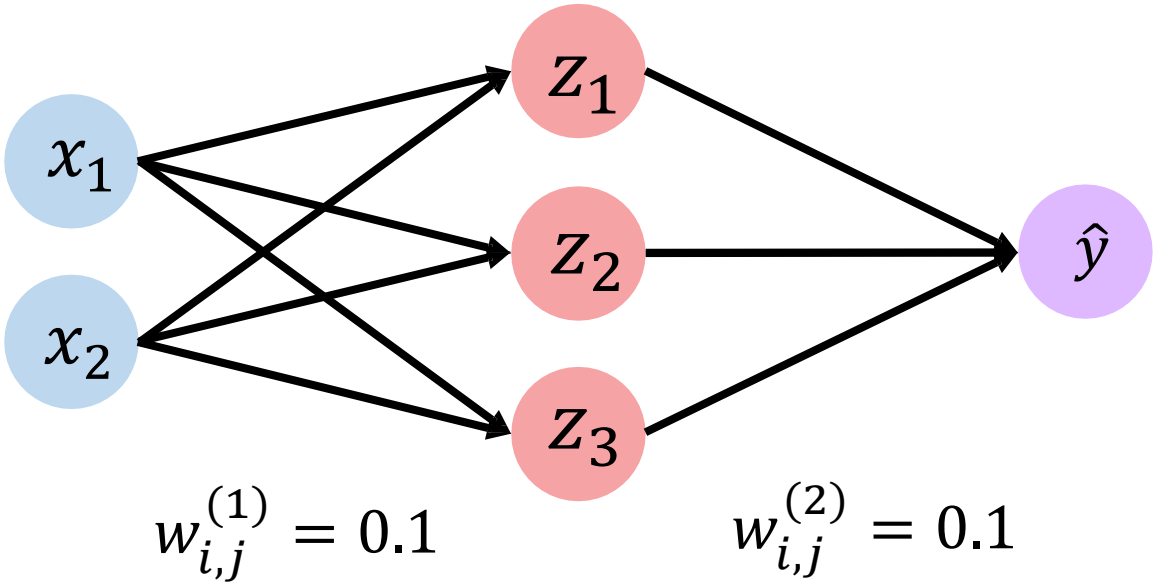
- Step 1: Initialize $W = (W^{(1)}, W^{(2)})$



Initialization

Initialize the network for the first feed-forward pass

- Step 1: Initialize $W = (W^{(1)}, W^{(2)})$

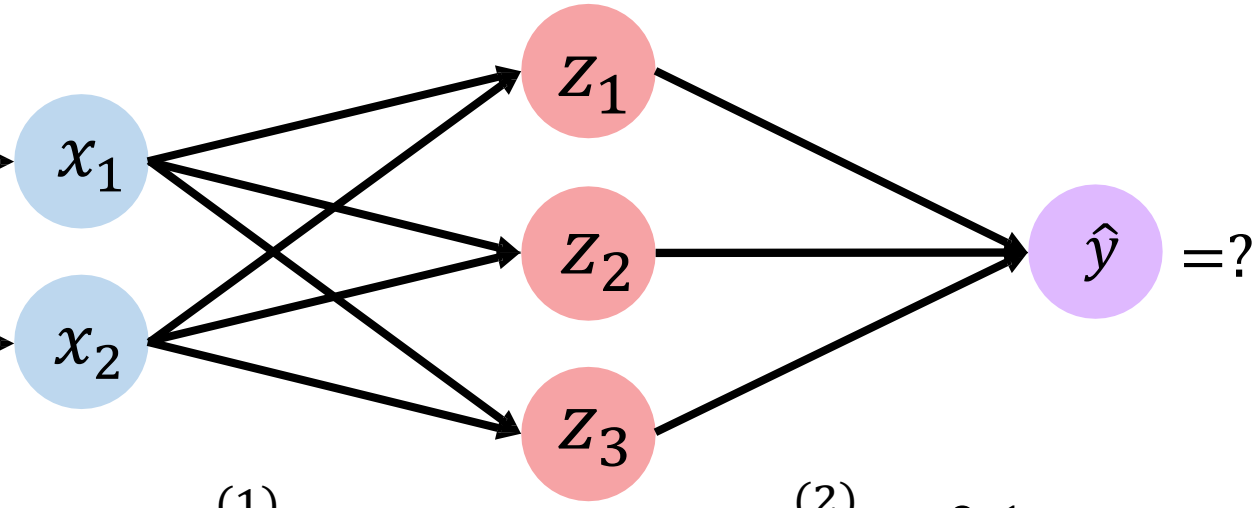


  - For example, set all weights to 0.1

Feed-forward

Data Point:

$$x^{[1]} = [4\,,5\,]$$



$=?$

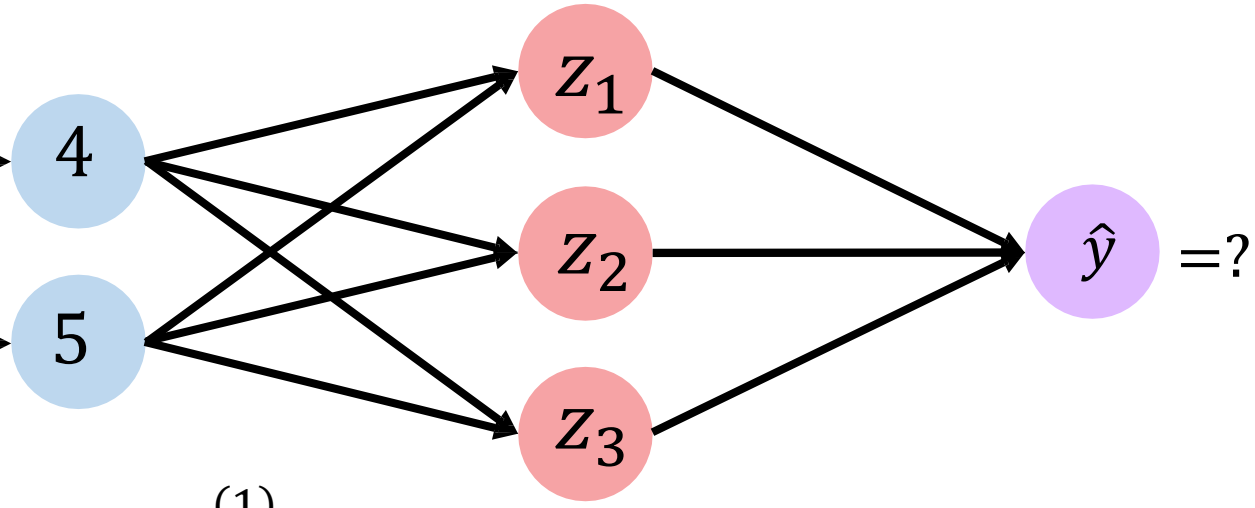
$w_{i,j}^{(1)} = 0.1$

$w_{i,j}^{(2)} = 0.1$

Feed-forward

$$z_1 = g\left(w_{0,1}^{(0)} + x_1 w_{1,1}^{(1)} + x_2 w_{2,1}^{(1)}\right)$$

$$x^{[1]} = [4\,,5\,]$$

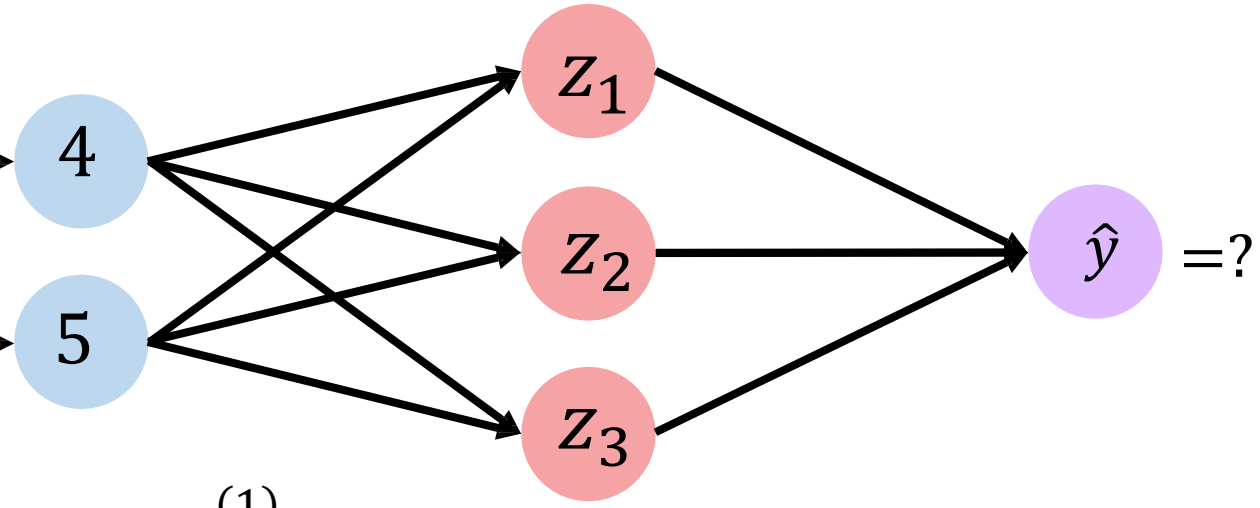


$$w_{i,j}^{(1)} = 0.1$$

Feed-forward

$$z_1 = g\left(w_{0,1}^{(0)} + x_1 w_{1,1}^{(1)} + x_2 w_{2,1}^{(1)}\right)$$
$$= \mathrm{ReLU}(0.1 + 4 \times 0.1 + 5 \times 0.1) = \mathrm{ReLU}(1) = 1.$$

$x^{[1]} = [4\,,5\,]$



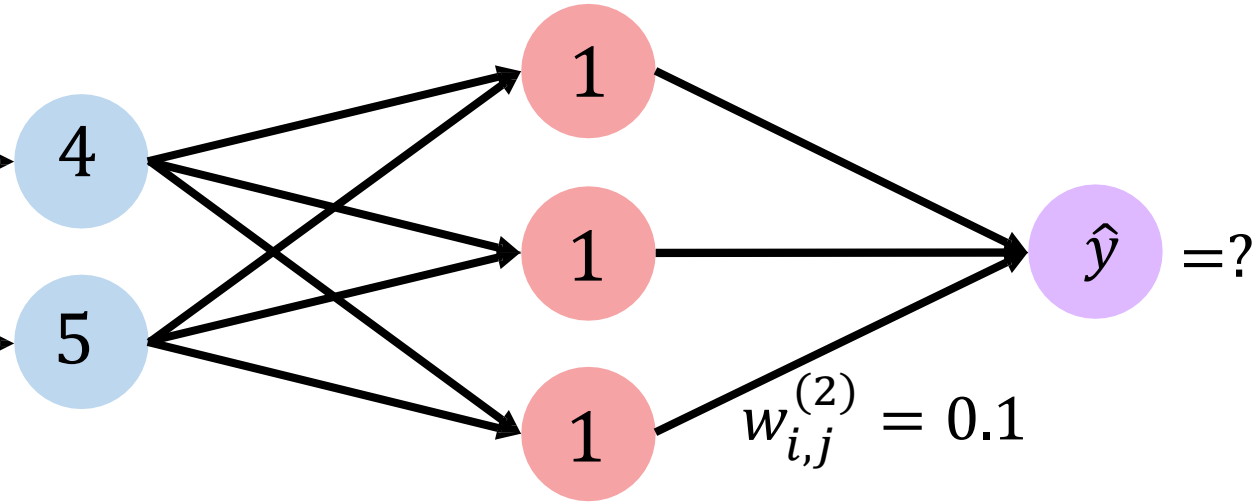
$=?$

$w_{i,j}^{(1)} = 0.1$

Feed-forward

$$z_1 = g\left(w_{0,1}^{(0)} + x_1 w_{1,1}^{(1)} + x_2 w_{2,1}^{(1)}\right)$$
$$= \mathrm{ReLU}(0.1 + 4 \times 0.1 + 5 \times 0.1) = \mathrm{ReLU}(1) = 1.$$

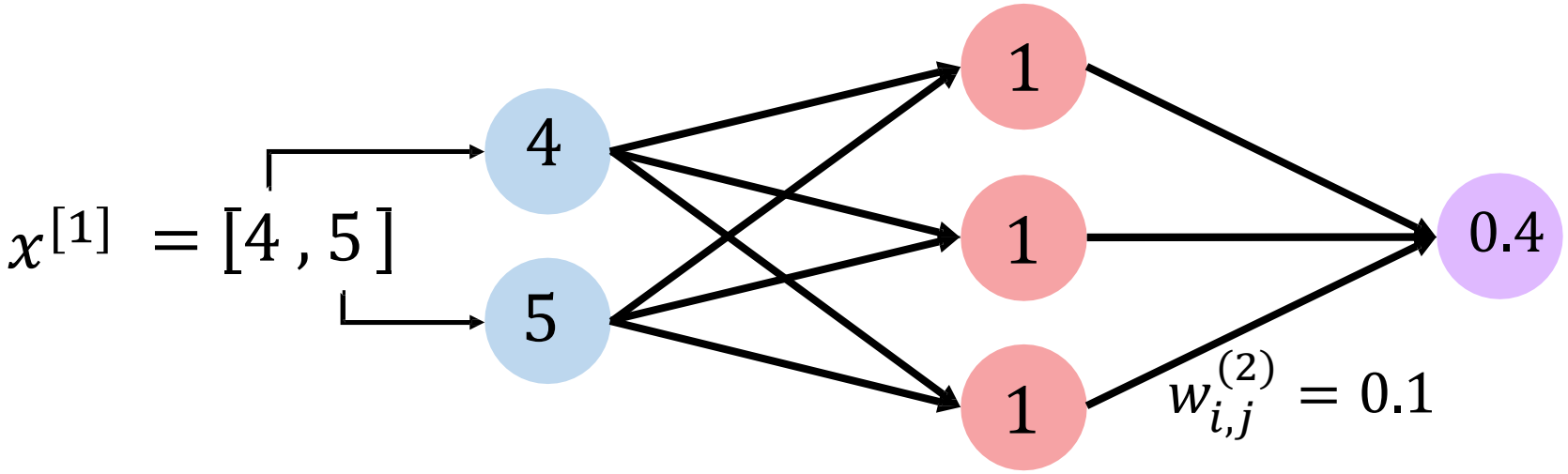
$x^{[1]} = [4\,,5\,]$



$=?$

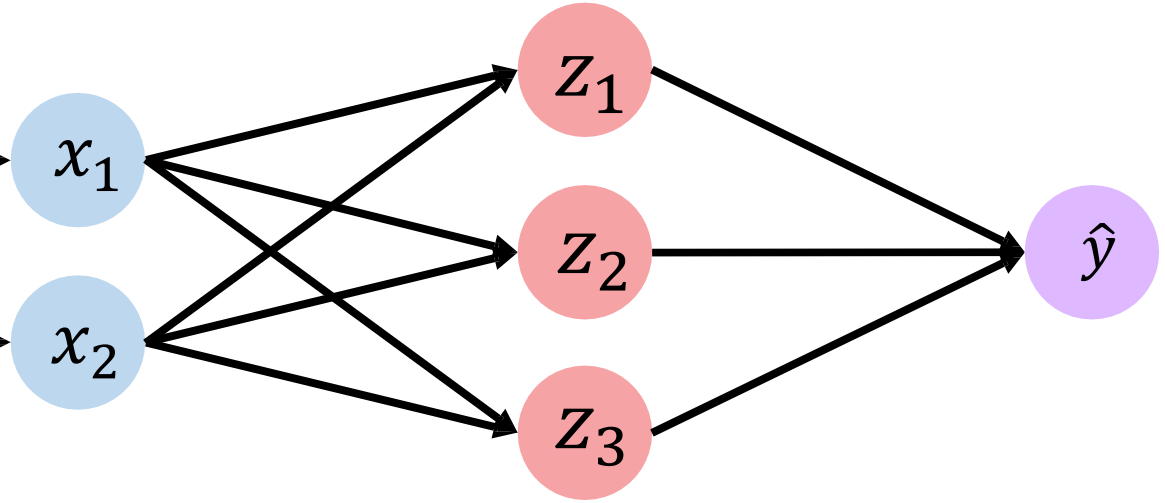
$$\hat{y} = g\left(w_{0,1}^{(2)} + z_1 w_{1,1}^{(2)} + z_2 w_{2,1}^{(2)} + z_3 w_{3,1}^{(2)}\right)$$
$$= \mathrm{ReLU}(0.4) = 0.4$$

Feed-forward



Model Assessment - Quantifying Loss

$$x^{(1)} = [4\,,5\,]$$



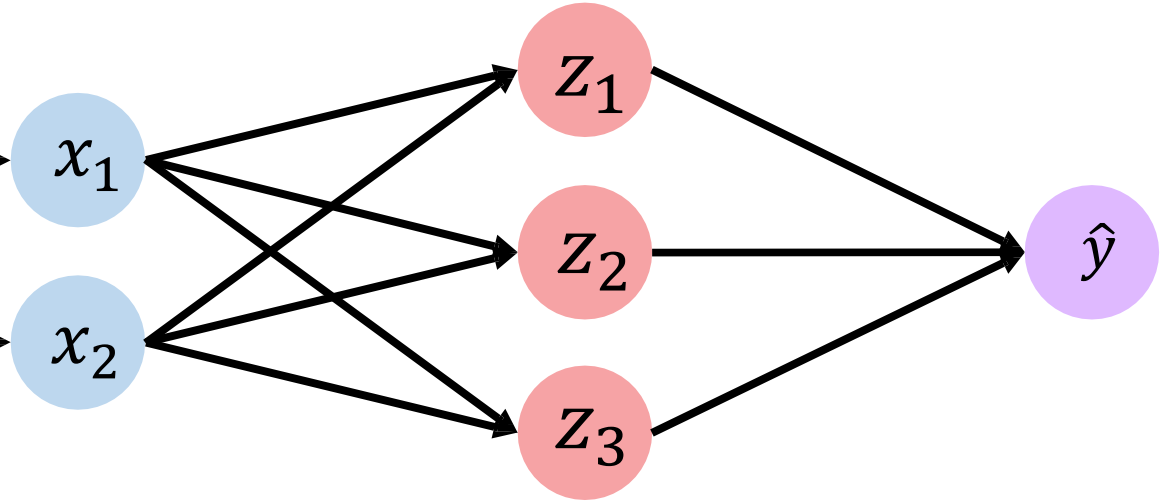
Predicted: 0.4

Actual: 1

$$loss(\underbrace{f(X^{[1]};W)}_{\text{Predicted}},\ \underbrace{Y^{[1]}}_{\text{Actual}})$$

Model Assessment - Quantifying Loss

$$x^{(1)} = [4\,,5\,]$$



Predicted: 0.4

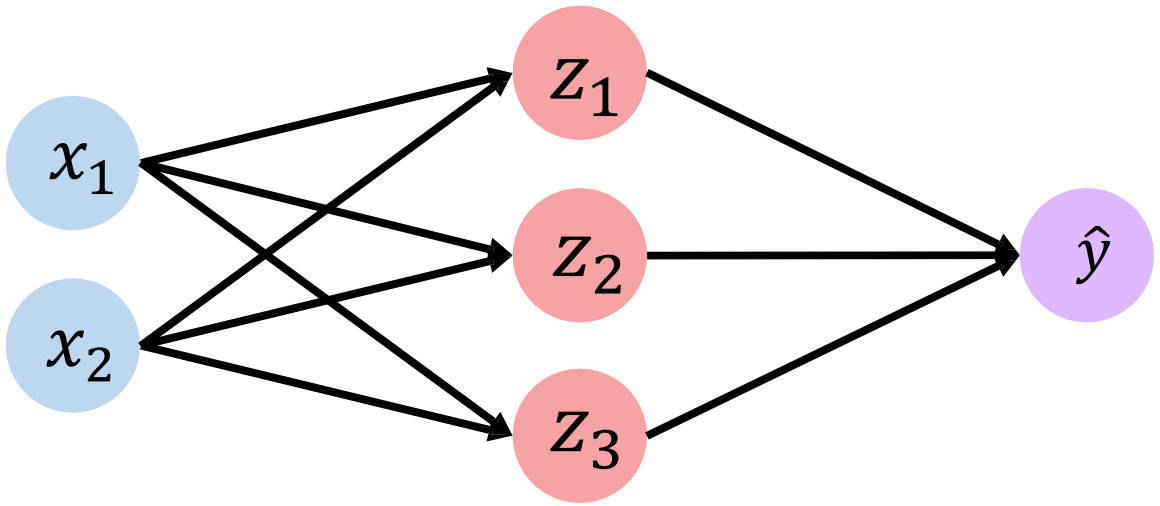
Actual: 1

$$loss(\underbrace{f(X^{[1]};W)}_{\text{Predicted}},\ \underbrace{Y^{[1]}}_{\text{Actual}})$$

Model Assessment – Empirical Risk

The empirical Risk measures the total loss over the training dataset

$$X = \begin{bmatrix} 4, & 5 \\ 2, & 2 \\ 6, & 8 \\ \vdots & \vdots \end{bmatrix}$$



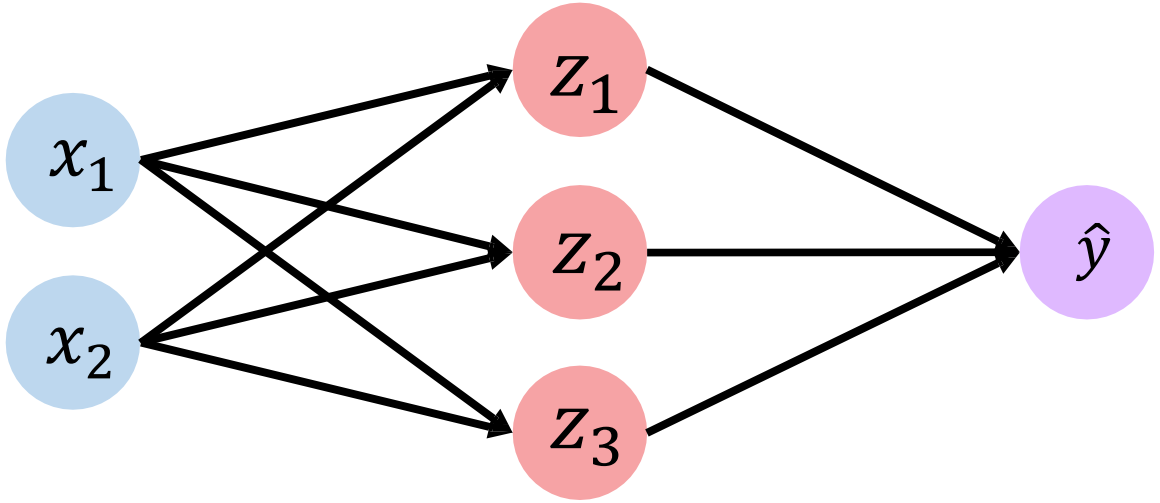
$$f(x) = \begin{bmatrix} 0.4 \\ 0.25 \\ 0.55 \\ \vdots \end{bmatrix} \qquad y = \begin{bmatrix} 1 \\ 0 \\ 1 \\ \vdots \end{bmatrix}$$

$$\min \quad J(W) = \frac{1}{n} \sum_{i=1}^{n} loss(\underbrace{f(X^{[i]}; W)}_{\text{Predicted}}, \underbrace{Y^{[i]}}_{\text{Actual}})$$

Model Assessment – Empirical Risk

The empirical Risk measures the total loss over the training dataset

$$X = \begin{bmatrix} 4, & 5 \\ 2, & 2 \\ 6, & 8 \\ \vdots & \vdots \end{bmatrix}$$



$$\begin{array}{cc} f(x) & y \\ \begin{bmatrix} 0.4 \\ 0.25 \\ 0.55 \\ \vdots \end{bmatrix} & \begin{bmatrix} 1 \\ 0 \\ 1 \\ \vdots \end{bmatrix} \end{array}$$

Also known as:

- Objective function
- Cost function

$$\min \quad J(W) = \frac{1}{n} \sum_{i=1}^{n} loss(\underbrace{f(X^{[i]}; W)}_{\text{Predicted}}, \underbrace{Y^{[i]}}_{\text{Actual}})$$

Quantifying Loss – Binary Classification

- 0/1 Loss: $loss(Y, f(X)) = 1_{Y \neq f(X)}$
  - $loss = 0$ when correctly classified, 1 when misclassified
  - Intuitive but hard to train

$$\min \quad J(W) = \frac{1}{n} \sum_{i=1}^{n} loss(\underbrace{f(X^{[i]}; W)}_{\text{Predicted}}, \underbrace{Y^{[i]}}_{\text{Actual}})$$

Quantifying Loss – Binary Classification

- 0/1 Loss: $loss(Y, f(X)) = 1_{Y \neq f(X)}$
  - $loss = 0$ when correctly classified, 1 when misclassified
  - Intuitive but hard to train

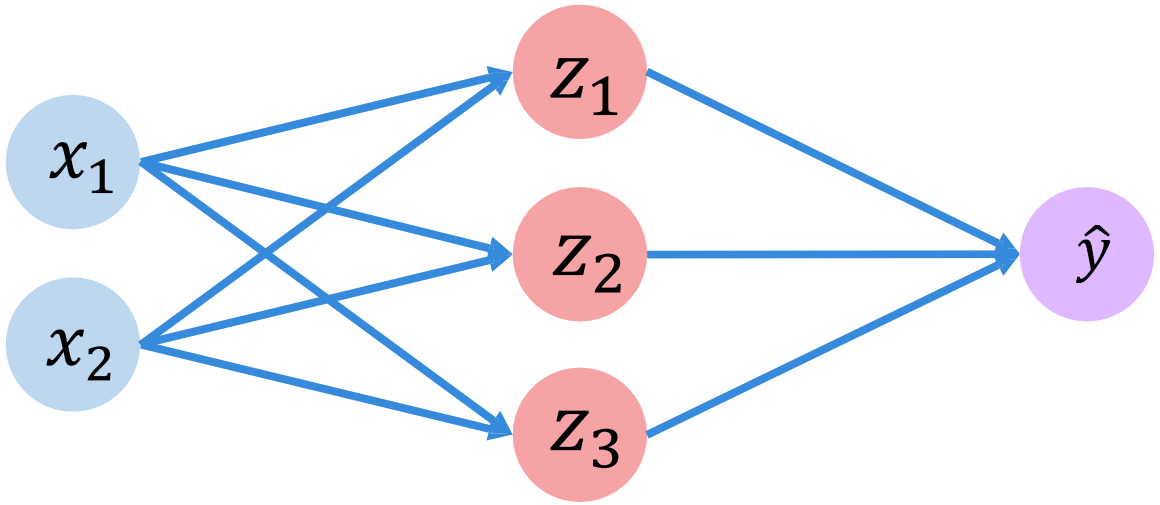
- Binary Cross Entropy Loss: $loss(Y, f(X)) = -Y \log f(X) - (1 - Y) \log(1 - f(X))$
  - measure of the difference between two probability distributions
  - Can be used when output a probability from 0 to 1

$$\min \quad J(W) = \frac{1}{n} \sum_{i=1}^{n} loss(\underbrace{f(X^{[i]}; W)}_{\text{Predicted}}, \underbrace{Y^{[i]}}_{\text{Actual}})$$

Classification: Binary Cross Entropy Loss

Using Binary Cross Entropy Loss for binary classification

$$x = \begin{bmatrix} 4, & 5 \\ 2, & 2 \\ 6, & 8 \\ \vdots & \vdots \end{bmatrix}$$

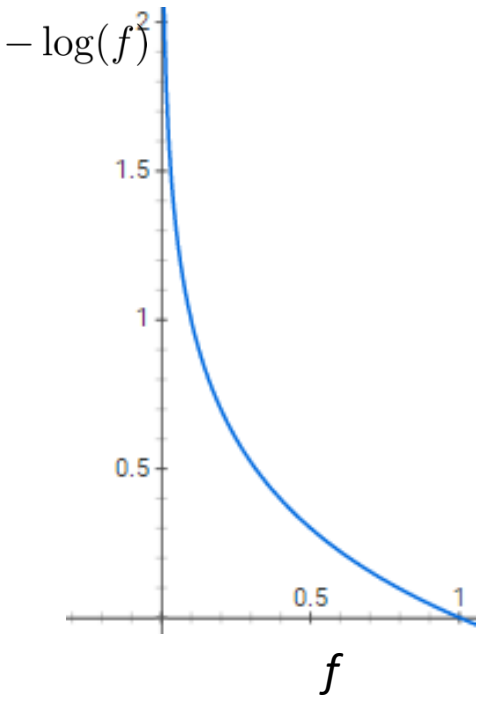


$$\begin{array}{cc} f(x) & y \\ \begin{bmatrix} 0.4 \\ 0.25 \\ 0.55 \\ \vdots \end{bmatrix} & \begin{bmatrix} 1 \\ 0 \\ 1 \\ \vdots \end{bmatrix} \end{array}$$

$$\min \quad J(W) = \frac{1}{n}\sum_{i=1}^{n} -\underbrace{Y^{[i]}}_{\text{Actual}} \log \underbrace{f\left(X^{[i]}; W\right)}_{\text{Predicted}} - \left(1 - \underbrace{Y^{[i]}}_{\text{Actual}}\right) \log\left(1 - \underbrace{f\left(X^{[i]}; W\right)}_{\text{Predicted}}\right)$$

Classification: Binary Cross Entropy Loss

True label
$Y^{[i]} = 1$



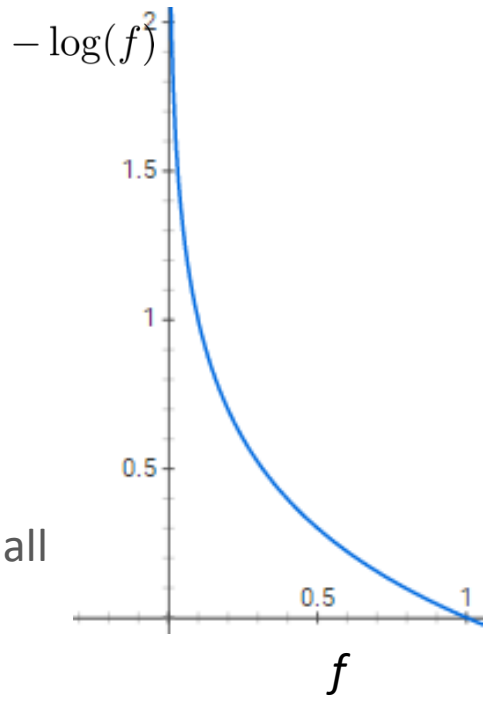
f

$$\min \quad J(W) = \frac{1}{n}\sum_{i=1}^{n} -\underbrace{Y^{[i]}}_{\text{Actual}} \log \underbrace{f\left(X^{[i]};W\right)}_{\text{Predicted}} - \left(1-\underbrace{Y^{[i]}}_{\text{Actual}}\right)\log\left(1-\underbrace{f\left(X^{[i]};W\right)}_{\text{Predicted}}\right)$$

Classification: Binary Cross Entropy Loss

True label
$Y^{[i]} = 1$

Hope $-\log(f)$ to be small such that f is close to 1

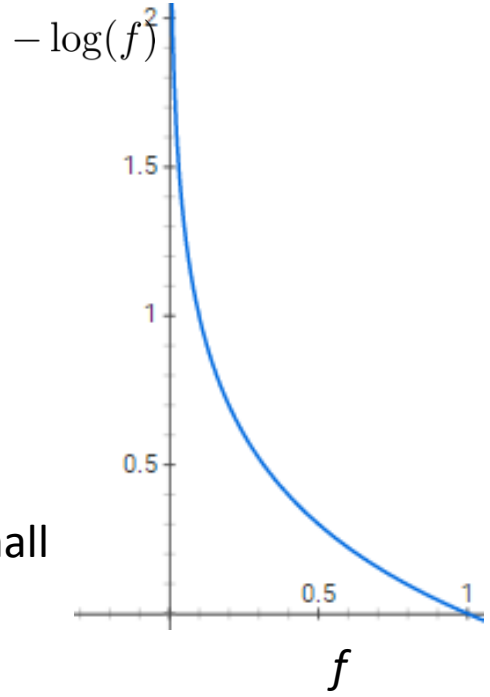


$$\min \quad J(W) = \frac{1}{n}\sum_{i=1}^{n} -\underbrace{Y^{[i]}}_{\text{Actual}} \log \underbrace{f\left(X^{[i]}; W\right)}_{\text{Predicted}} - \left(1 - \underbrace{Y^{[i]}}_{\text{Actual}}\right) \log\left(1 - \underbrace{f\left(X^{[i]}; W\right)}_{\text{Predicted}}\right)$$

Classification: Binary Cross Entropy Loss

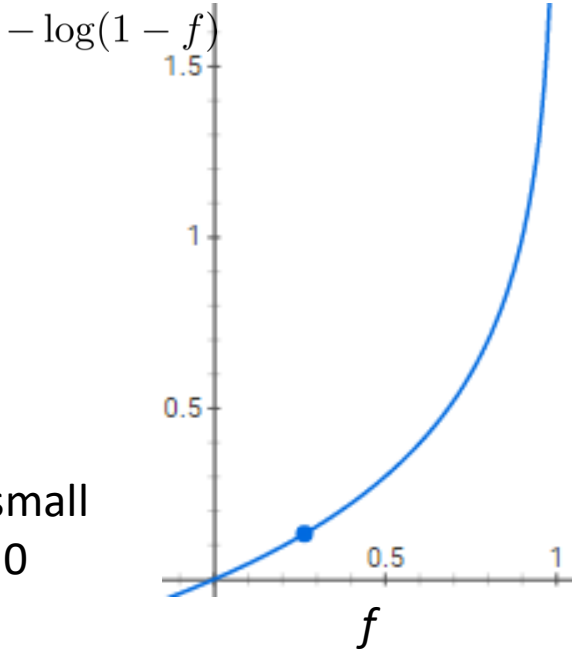
True label
$Y^{[i]} = 1$

Hope $-\log(f)$ to be small such that f is close to 1



True label
$Y^{[i]} = 0$

Hope $-\log(1-f)$ to be small such that f to be close to 0



$$\min \quad J(W) = \frac{1}{n}\sum_{i=1}^{n} -\underbrace{Y^{[i]}}_{\text{Actual}} \log \underbrace{f\left(X^{[i]};W\right)}_{\text{Predicted}} - \left(1-\underbrace{Y^{[i]}}_{\text{Actual}}\right) \log\left(1-\underbrace{f\left(X^{[i]};W\right)}_{\text{Predicted}}\right)$$

Classification: Binary Cross Entropy Loss

- Binary Cross Entropy Loss

$$loss = -Y \log f(X;W) - (1-Y)\log(1-f(X;W))$$

- Vectorize True Label as (Y_1, Y_0): $Y_1 = Y$ and $Y_0 = 1 - Y$
- Prediction: $\hat{Y}_1 = f(X;W)$ versus $\hat{Y}_0 = 1 - f(X;W)$

$$loss = -Y_1 \log \hat{Y}_1 - Y_0 \log \hat{Y}_0$$

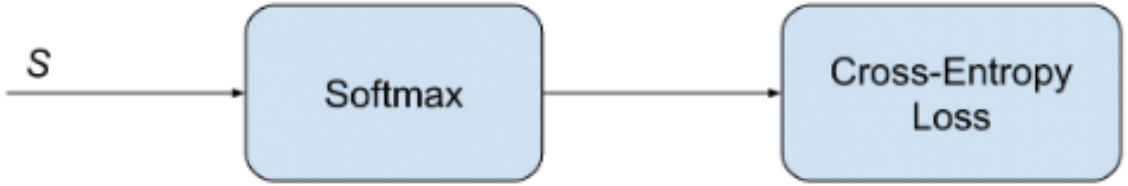
  – Note that $Y_1 + Y_0 = 1$ and $\hat{Y}_1 + \hat{Y}_0 = 1$

- Extend the definition Multi-Class?

Classification: Multi-class Cross Entropy Loss

- Binary Cross-entropy $loss = -Y_1 \log \hat{Y}_1 - Y_0 \log \hat{Y}_0$
- K Classes: $loss = -\sum_{k=1,\dots,K} Y_k \log(\hat{Y}_k)$
- Vectorize $Y = (Y_1, Y_2, Y_3, \dots)^\top$: $loss = -Y^T \log(\hat{Y})$

- The true label vector $\sum_k y_k = 1$. How to make sure $\sum_k \hat{y}_k = 1$?
  - Apply a Softmax operator to the output
  - Use it as the activation function of the last layer
  - When no hidden layer: logistic regression

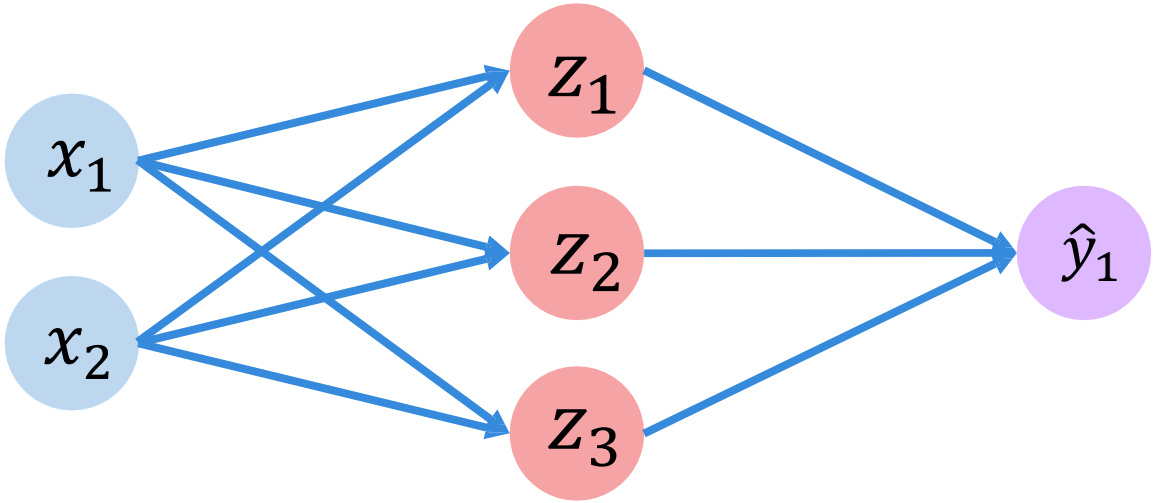


$$\hat{y}_k = \frac{\exp(s_k)}{\sum \exp(s_i)}$$

Regression: Mean Squared Error Loss

Used with regression models that output continuous real numbers

$$\boldsymbol{x} = \begin{bmatrix} 4, & 5 \\ 2, & 2 \\ 6, & 8 \\ \vdots & \vdots \end{bmatrix}$$



$$\begin{array}{cc} f(x) & y \\ \begin{bmatrix} 30 \\ 80 \\ 85 \\ \vdots \end{bmatrix} & \begin{bmatrix} 90 \\ 20 \\ 95 \\ \vdots \end{bmatrix} \end{array}$$

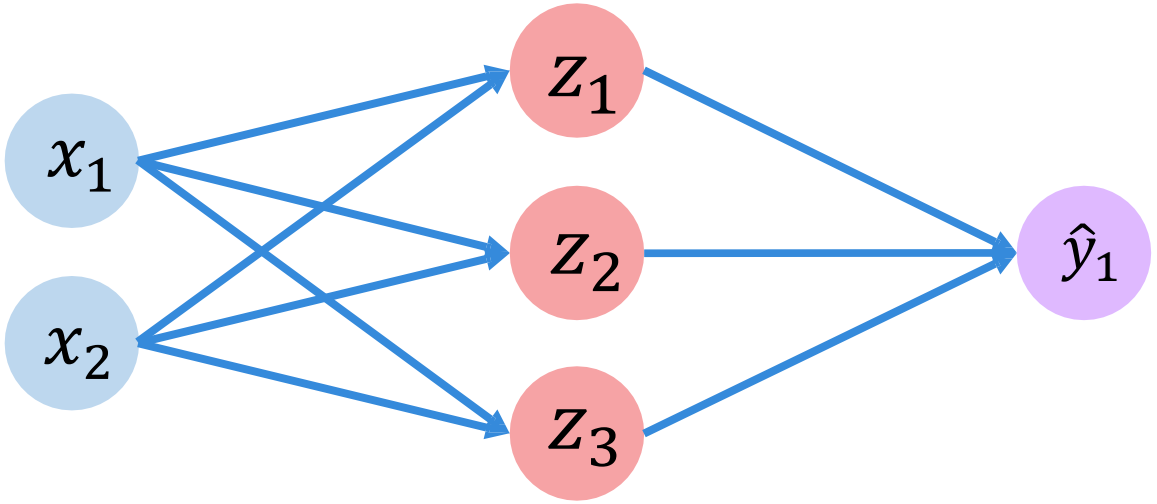
Final Grades (percentage)

$$\min \quad J(W) = \frac{1}{n} \sum_{i=1}^{n} loss(\underbrace{f(X^{[i]}; W)}_{\text{Predicted}}, \underbrace{Y^{[i]}}_{\text{Actual}})$$

Regression: Mean Squared Error Loss

Used with regression models that output continuous real numbers

$$\boldsymbol{x} = \begin{bmatrix} 4, & 5 \\ 2, & 2 \\ 6, & 8 \\ \vdots & \vdots \end{bmatrix}$$



$$\begin{array}{cc} f(x) & y \\ \begin{bmatrix} 30 \\ 80 \\ 85 \\ \vdots \end{bmatrix} & \begin{bmatrix} 90 \\ 20 \\ 95 \\ \vdots \end{bmatrix} \end{array}$$

Final Grades (percentage)

$$\min \quad J(W) = \frac{1}{n} \sum_{i=1}^{n} \Big(\underbrace{f\big(X^{[i]}; W\big)}_{\text{Predicted}} - \underbrace{Y^{[i]}}_{\text{Actual}} \Big)^2$$

Loss Optimization

We want to find the network weights that achieve the lowest loss

$$W^* = \underset{W}{\operatorname{argmin}}\, J(W) = \frac{1}{n}\sum_{i=1}^{n} loss(\, f(X^{[i]}; W),\ Y^{[i]})$$

$$\boldsymbol{W} = \left\{\boldsymbol{W}^{(1)}, \boldsymbol{W}^{(2)}, \cdots\right\}$$

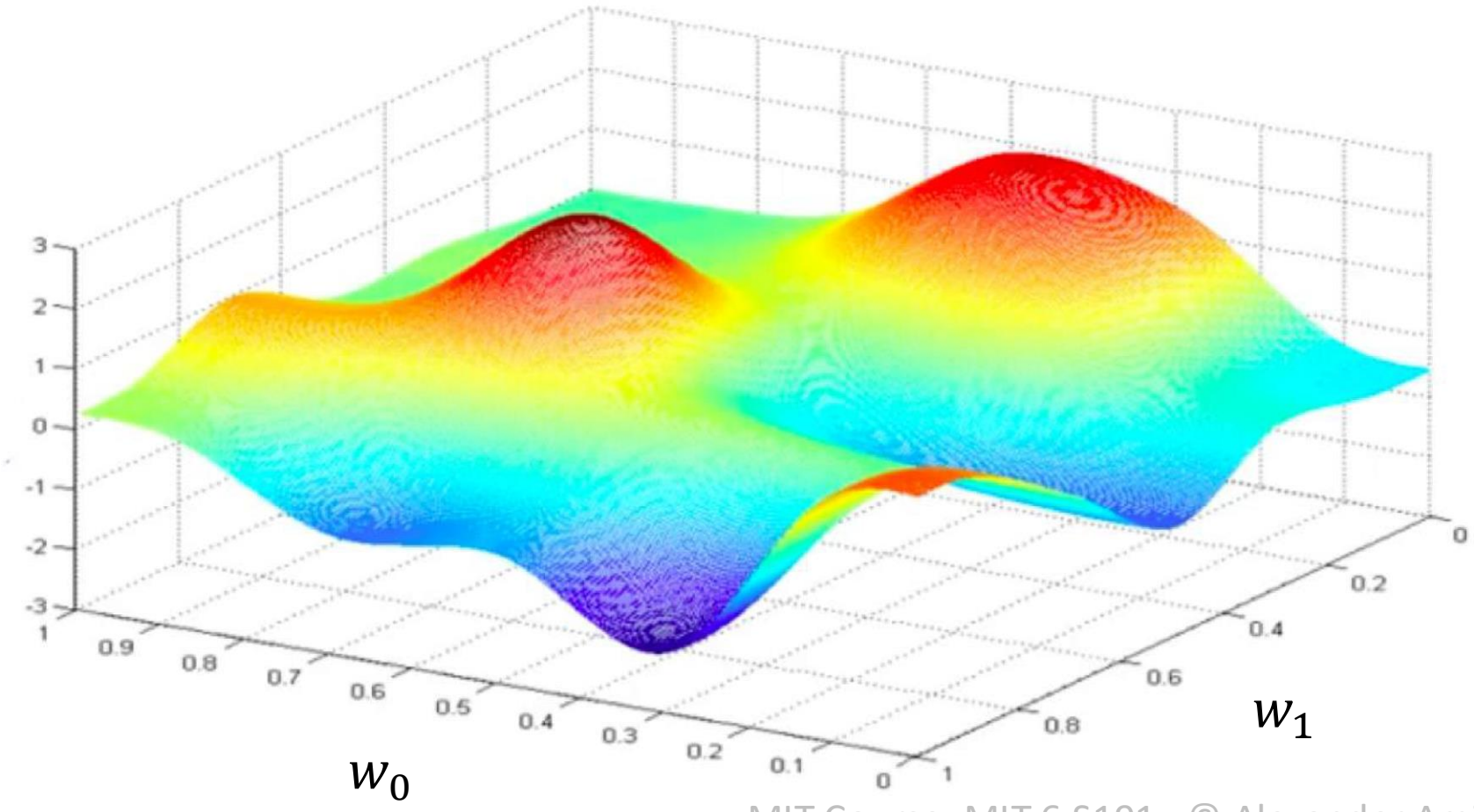
Remember:

Our loss is a function of the network weights!

Loss Optimization - Gradient Descent

$$W^* = \underset{W}{\operatorname{argmin}}\, J(W)$$

$J(w_0, w_1)$



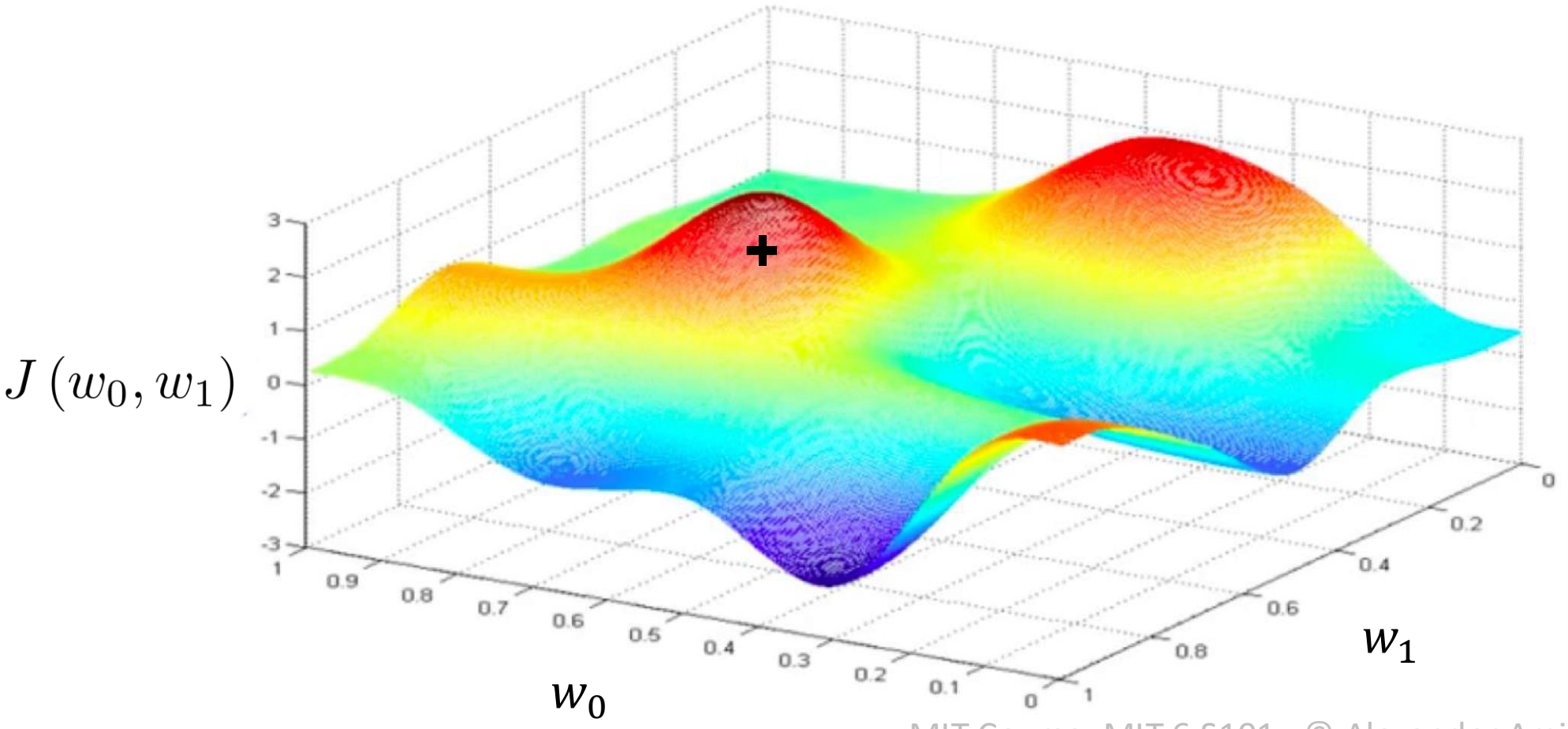
w_0

w_1

MIT Course: MIT 6.S191: © Alexander Amini and Ava Soleimany

Loss Optimization - Gradient Descent

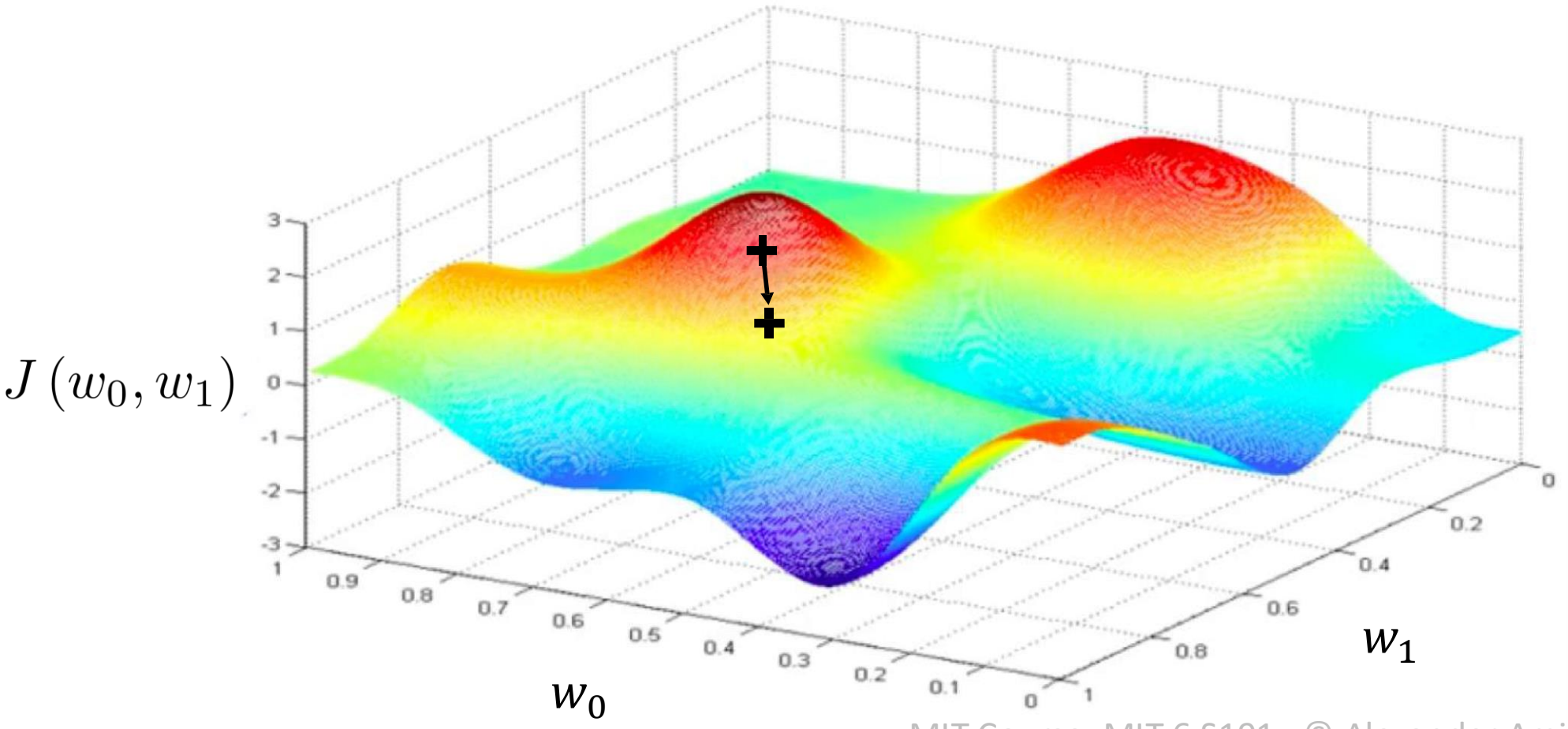
Randomly pick an initial (w_0, w_1)



MIT Course: MIT 6.S191: © Alexander Amini and Ava Soleimany

Loss Optimization - Gradient Descent

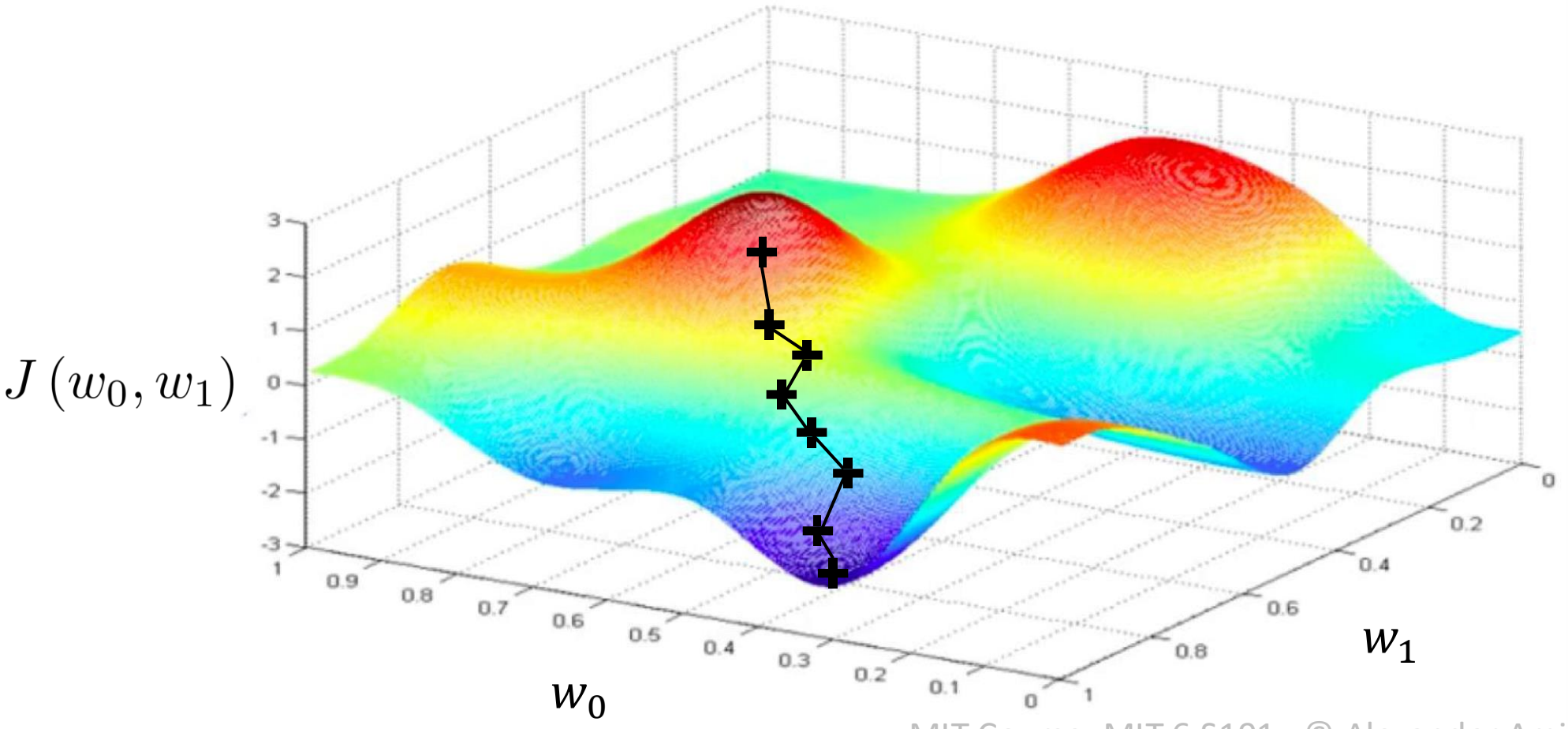
Compute gradient and move towards negative gradient direction



MIT Course: MIT 6.S191: © Alexander Amini and Ava Soleimany

Loss Optimization - Gradient Descent

Repeat until convergence



MIT Course: MIT 6.S191: © Alexander Amini and Ava Soleimany

Loss Optimization - Gradient Descent

1. Initialize weights
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $W^{t+1} \leftarrow W^{t} - \eta \frac{\partial J(W)}{\partial W^{\top}}$
5. Return weights

Loss Optimization - Gradient Descent

1. Initialize weights
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $W^{t+1} \leftarrow W^{t} - \eta \frac{\partial J(W)}{\partial W^{\top}}$
5. Return weights

Question 1: How to set initial weights?

Loss Optimization - Gradient Descent

1. Initialize weights
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $W^{t+1} \leftarrow W^{t} - \eta \frac{\partial J(W)}{\partial W^{\top}}$
5. Return weights

Question 1: How to set initial weights?

Question 2: How to compute gradient?

Loss Optimization - Gradient Descent

1. Initialize weights
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $W^{t+1} \leftarrow W^{t} - \eta \frac{\partial J(W)}{\partial W^{\mathsf{T}}}$
5. Return weights

Question 1: How to set initial weights?

Question 2: How to compute gradient?

Question 3: How to choose the stepsize?

η: Stepsize (Learning Rates)

Loss Optimization - Gradient Descent

1. Initialize weights
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $W^{t+1} \leftarrow W^{t} - \eta \frac{\partial J(W)}{\partial W^{\mathsf{T}}}$
5. Return weights

Question 1: How to set initial weights?

Question 4: When to stop?

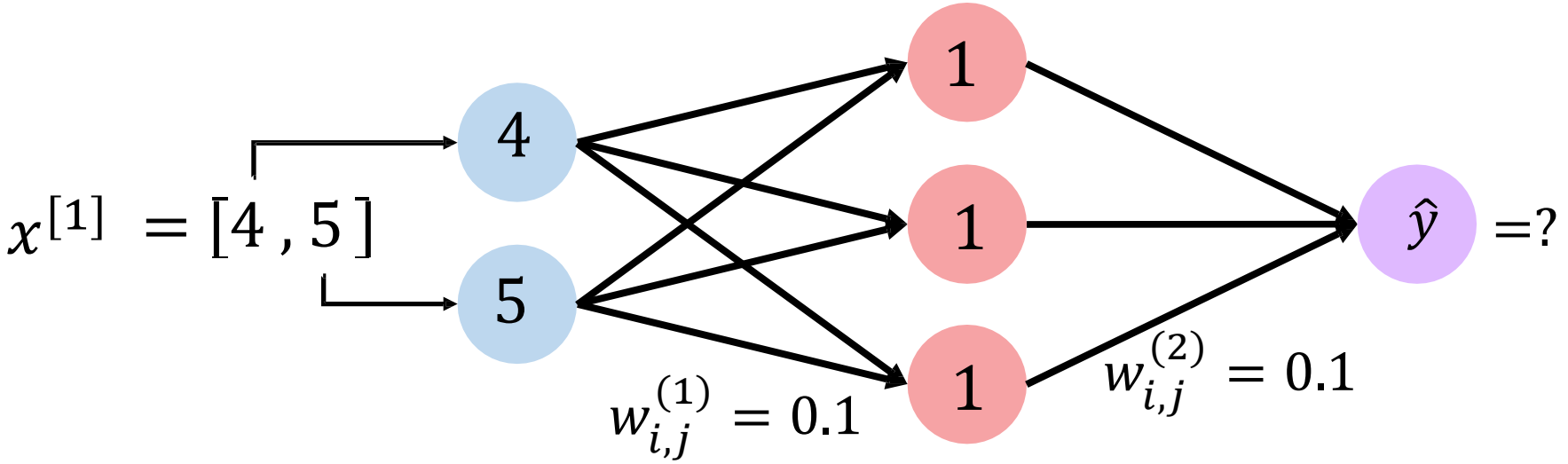
Question 2: How to compute gradient?

Question 3: How to choose the stepsize?

η: Stepsize (Learning Rates)

Question 1: How to set initial weights?

Deterministic Initialization is poor



Question 1: How to set initial weights?

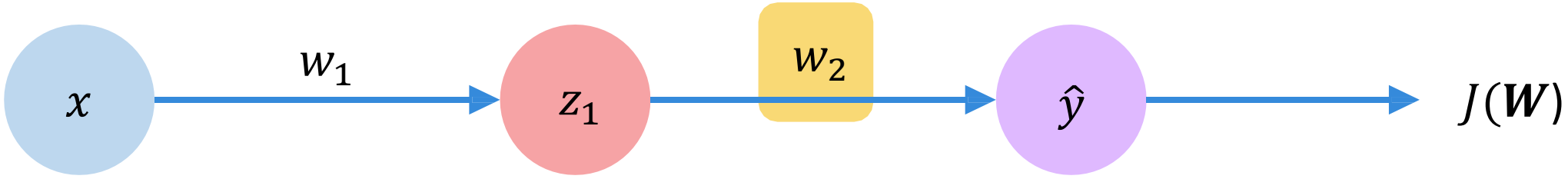
- Deterministic Initialization is poor
  - Set initial weights randomly $w_{i,j}^{(k)} \sim N(0, \sigma^2)$
  - How to choose σ^2
    - Carefully, not always working
  - Readings:
    - https://towardsdatascience.com/weight-initialization-in-neural-networks-a-journey-from-the-basics-to-kaiming-954fb9b47c79

Question 1: How to set initial weights?

Common Initializers

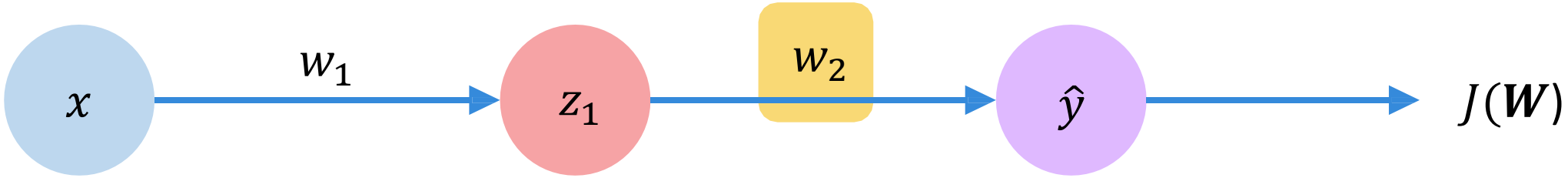
- Random Normal (mean, std)
- Random Uniform (min, max)
- Truncated Normal (mean, std), values more than 2std from the mean is redrawn
- Zeros
- Ones
- Glorot Normal: truncated normal with mean 0, std = sqrt (2/(fan_in+fan_out))
- Glorot Uniform: max=-min=sqrt (6/(fan_in+fan_out))
- HeNormal: truncated normal with mean 0, std = sqrt (2/ fan_in)
- HeUniofrm: max=-min=sqrt (6/fan_in)

Question 2: How to compute Gradient



How does a small change in one weight (ex. w_2) affect the final loss $J(\boldsymbol{W})$?

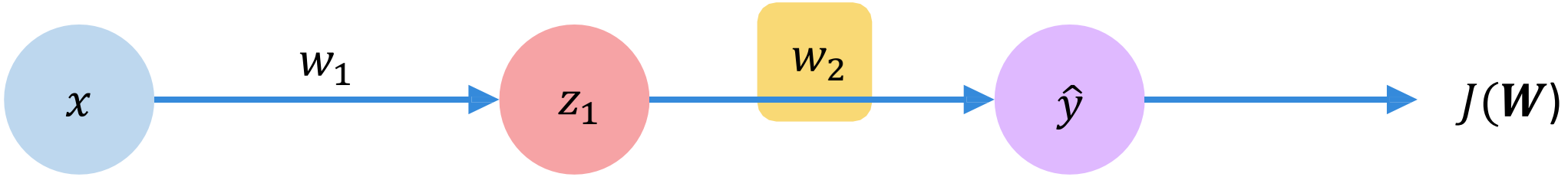
Question 2: How to compute Gradient



How does a small change in one weight (ex. w_2) affect the final loss $J(\boldsymbol{W})$?

Backpropagation

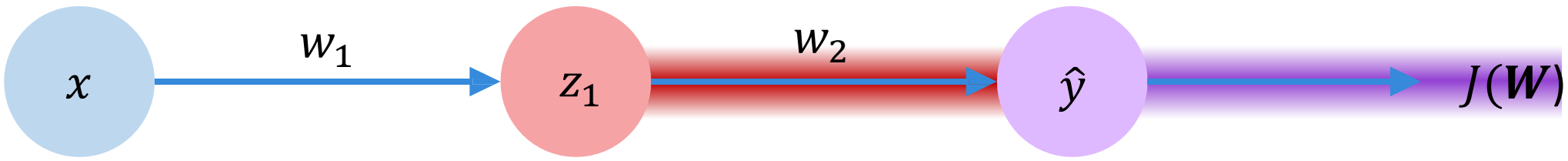
Computing Gradients: Backpropagation



$$\frac{\partial J(\boldsymbol{W})}{\partial w_2} =$$

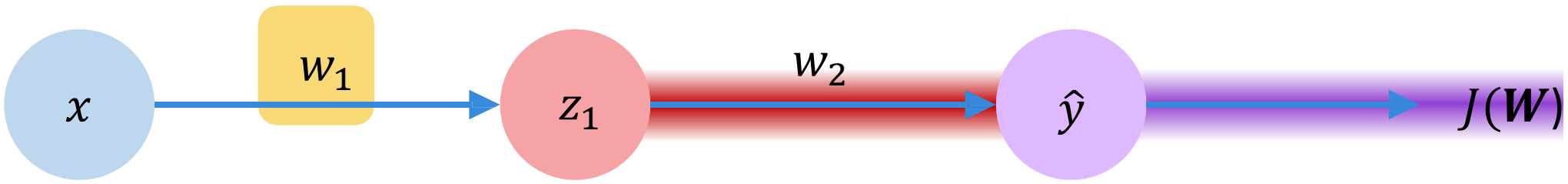
Let's use the chain rule!

Computing Gradients: Backpropagation



$$\frac{\partial J(\boldsymbol{W})}{\partial w_2} = \frac{\partial J(\boldsymbol{W})}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_2}$$

Computing Gradients: Backpropagation

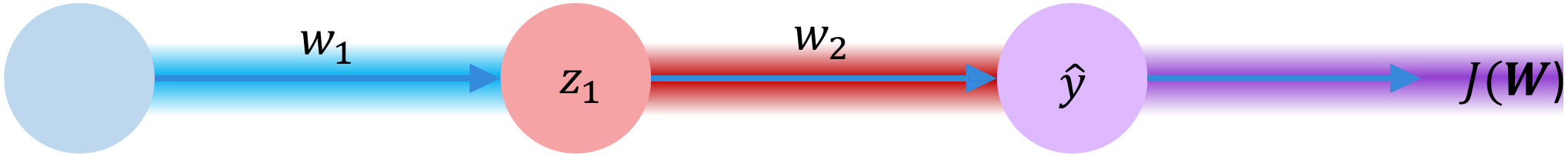


$$\frac{\partial J(\boldsymbol{W})}{\partial w_1} = \frac{\partial J(\boldsymbol{W})}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_1}$$

Apply chain rule!

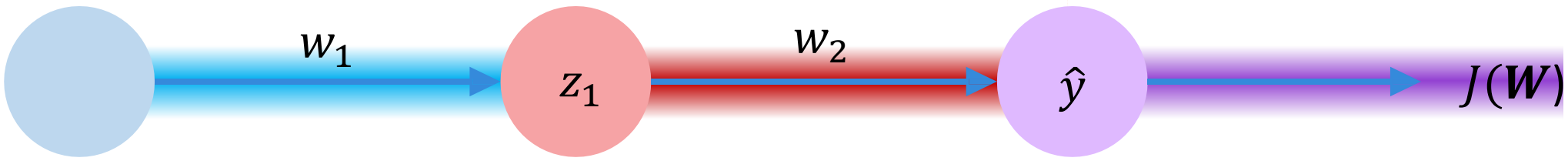
Apply chain rule!

Computing Gradients: Backpropagation



$$\frac{\partial J(\boldsymbol{W})}{\partial w_1} = \frac{\partial J(\boldsymbol{W})}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial z_1} * \frac{\partial z_1}{\partial w_1}$$

Computing Gradients: Backpropagation



$$\frac{\partial J(\boldsymbol{W})}{\partial w_1} = \frac{\partial J(\boldsymbol{W})}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial z_1} * \frac{\partial z_1}{\partial w_1}$$

Repeat this for every weight in the network using gradients from later layers

Loss Optimization - Gradient Descent

1. Initialize weights
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $W^{t+1} \leftarrow W^{t} - \eta \frac{\partial J(W)}{\partial W}$
5. Return weights

Question 2: How to compute gradient?

Backpropagation

Loss Optimization - Gradient Descent

1. Initialize weights
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $W^{t+1} \leftarrow W^{t} - \eta \frac{\partial J(W)}{\partial W}$
5. Return weights

Question 2: How to compute gradient?

Backpropagation

Good! But

$$J(W) = \frac{1}{n} \sum_{i=1}^{n} loss(\, f(X^{[i]}; W),\ Y^{[i]})$$

Slow with big data !

Loss Optimization – Stochastic Gradient Descent

- Gradient of $J(W)$ is very slow to compute: You need to sum over all data

$$J(W) = \frac{1}{n} \sum_{i=1}^{n} \underbrace{loss\left(f\left(X^{[i]}; W\right),\ Y^{[i]}\right)}_{J_i(W)}$$

- Replace loss $J(W)$ by stochastic loss $J_i(W)$

Loss Optimization – Stochastic Gradient Descent

1. Initialize weights
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $W^{t+1} \leftarrow W^{t} - \eta \frac{\partial J(W)}{\partial W^{\top}}$
5. Return weights

Loss Optimization – Stochastic Gradient Descent

1. Initialize weights
2. Loop until convergence:
3. Pick a single data point i, and compute stochastic gradient, $\frac{\partial J_i(\boldsymbol{W})}{\partial \boldsymbol{W}}$
4. Update weights, $W^{t+1} \leftarrow W^t - \eta \frac{\partial J_i(W)}{\partial W}$
5. Return weights

Loss Optimization – Stochastic Gradient Descent

1. Initialize weights
2. Loop until convergence:
3. Pick a single data point i, and compute stochastic gradient, $\frac{\partial J_i(\boldsymbol{W})}{\partial \boldsymbol{W}}$
4. Update weights, $W^{t+1} \leftarrow W^t - \eta \frac{\partial J_i(W)}{\partial W}$
5. Return weights

Fast to compute but very noisy (stochastic)

Loss Optimization – Mini-Batch Gradient Descent

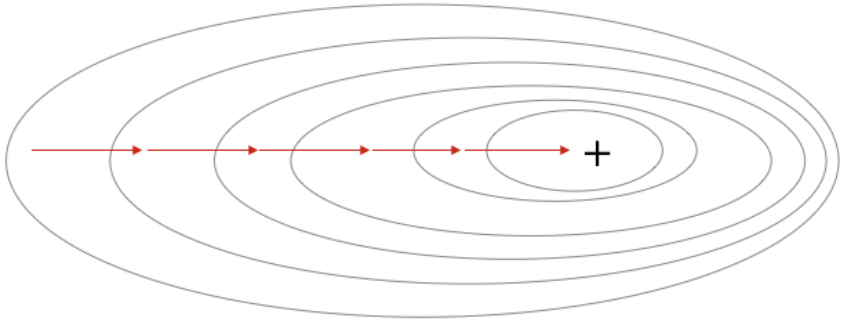
1. Initialize weights
2. Loop until convergence:
3. Pick a batch of data points B, and compute stochastic gradient, $\frac{1}{B}\sum_{i=1}^{B}\frac{\partial J_i(\boldsymbol{W})}{\partial \boldsymbol{W}}$
4. Update weights, $W^{t+1} \leftarrow W^t - \eta\frac{1}{B}\sum_{i=1}^{B}\frac{\partial J_i(\boldsymbol{W})}{\partial \boldsymbol{W}}$
5. Return weights

Choose the batch size B as a tradeoff of fast training vs accurate gradient estimation

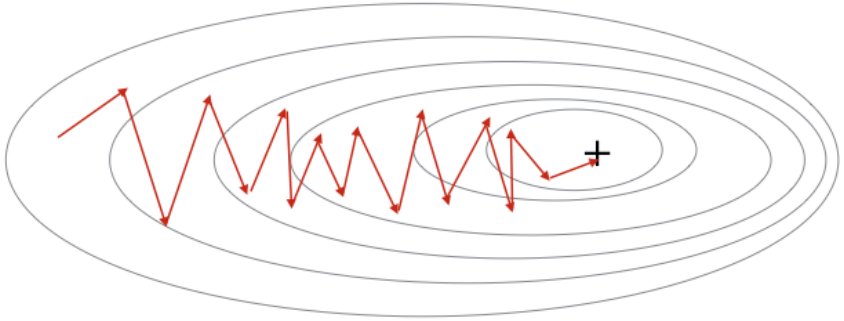
Loss Optimization

Path towards the optimum

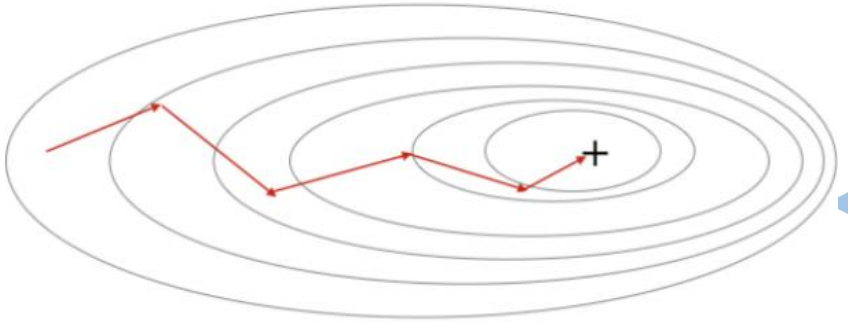
Gradient Descent



Stochastic Gradient Descent



Mini-Batch Gradient Descent



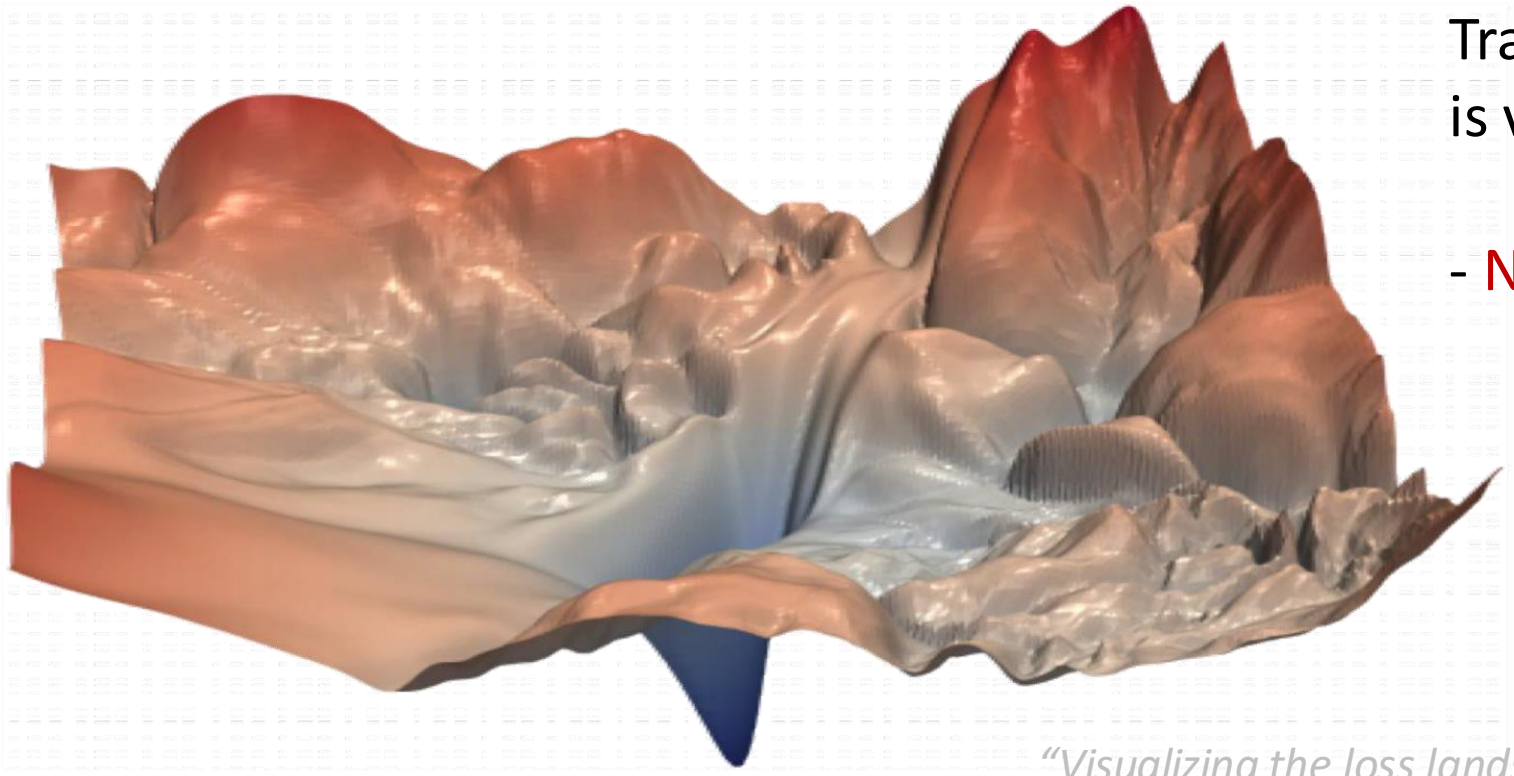
Loss Optimization

1. Initialize weights
2. Loop until convergence: Question 4: When to stop?
3. Compute (stochastic / mini-batch) gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $W^{t+1} \leftarrow W^{t} - \eta \frac{\partial J(W)}{\partial W}$ Question 3: How to choose the stepsize?
5. Return weights

Question 3: How to choose Learning Rates?

$$W^{t+1} \leftarrow W^t - \eta \nabla_w J(W^t)$$

How can we set the learning rate?



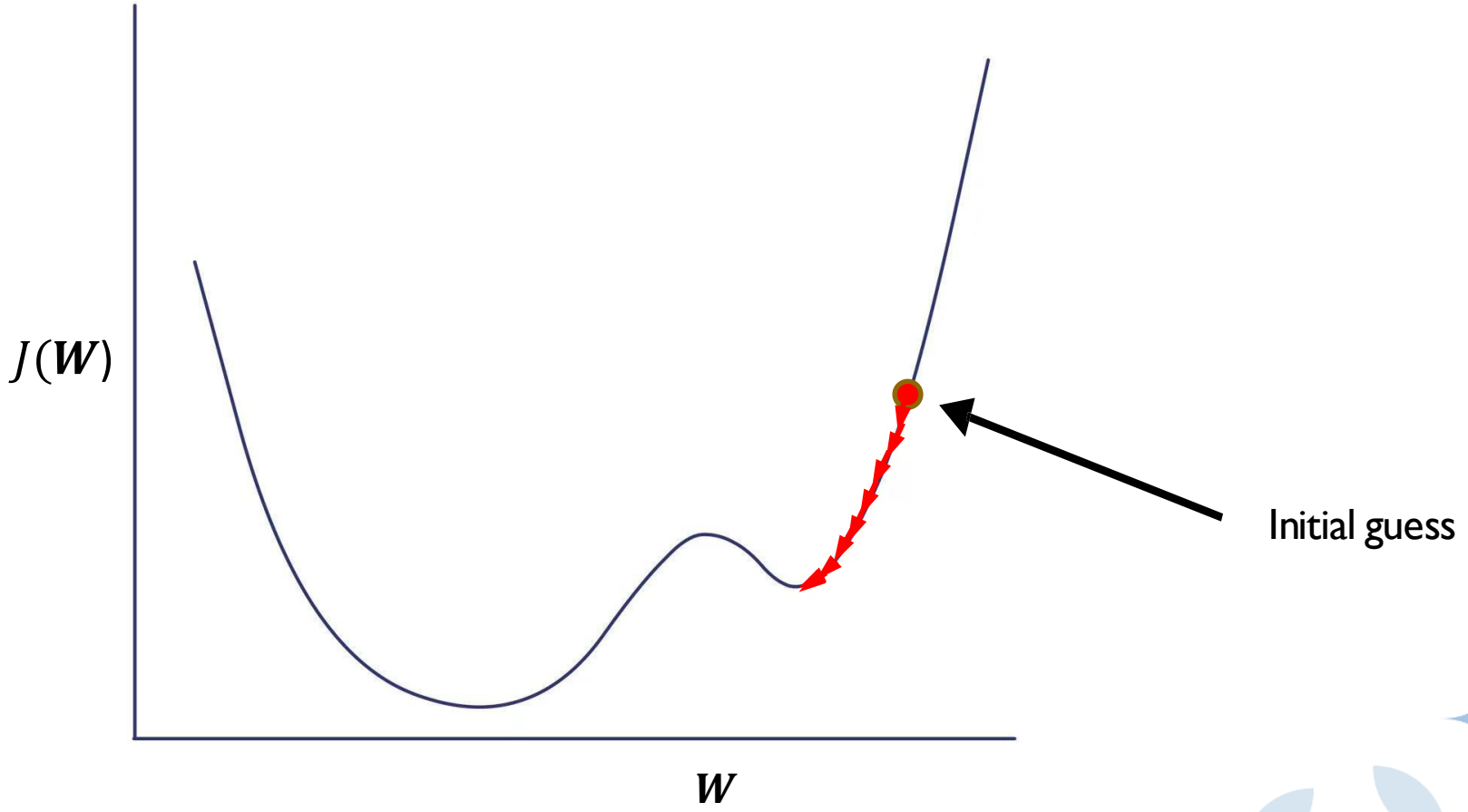
Training DNN is very difficult

- Nonconvex loss functions

"Visualizing the loss landscape of neural nets". Dec 2017.

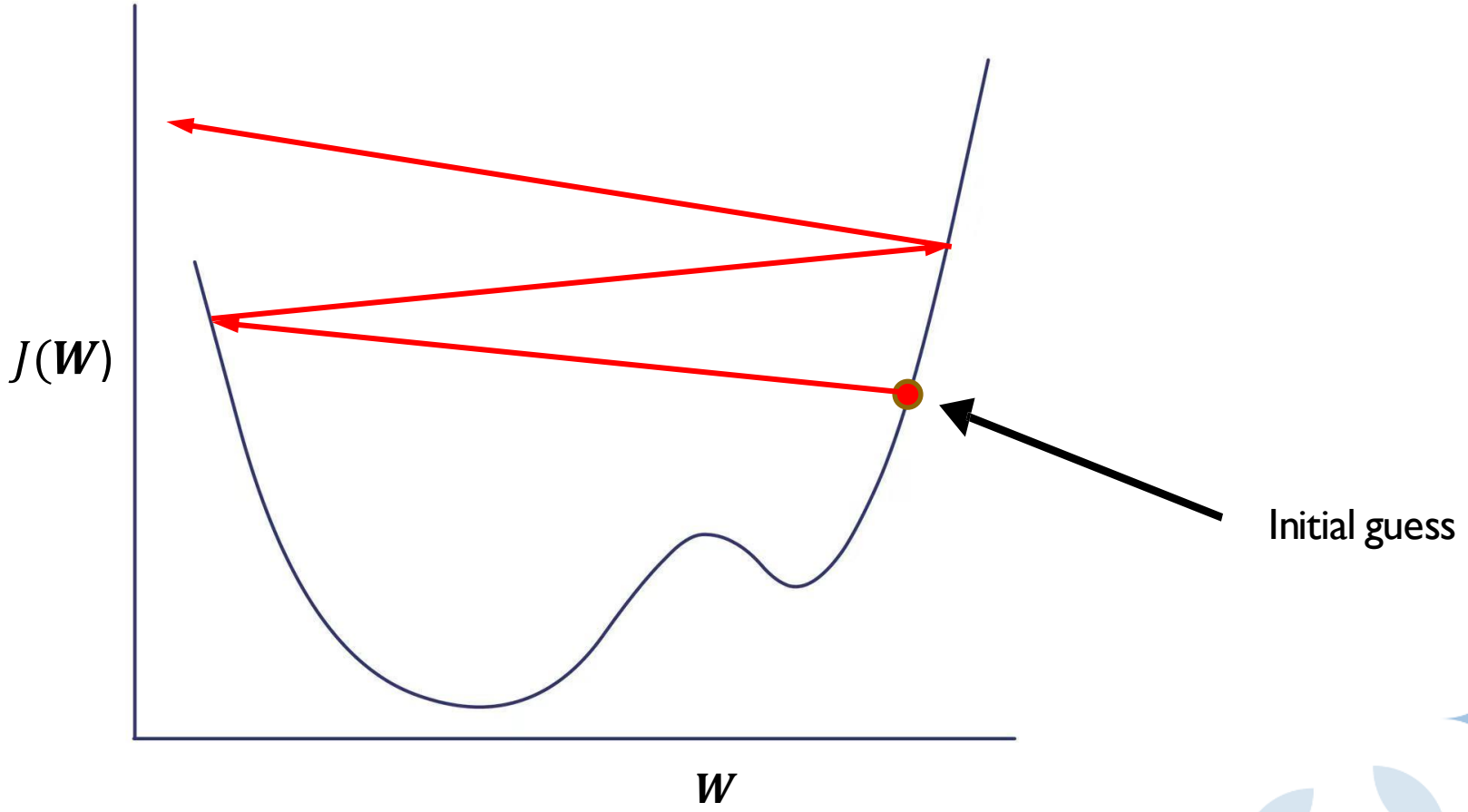
Question 3: How to choose Learning Rates?

Small learning rate converges slowly and gets stuck in false local minima



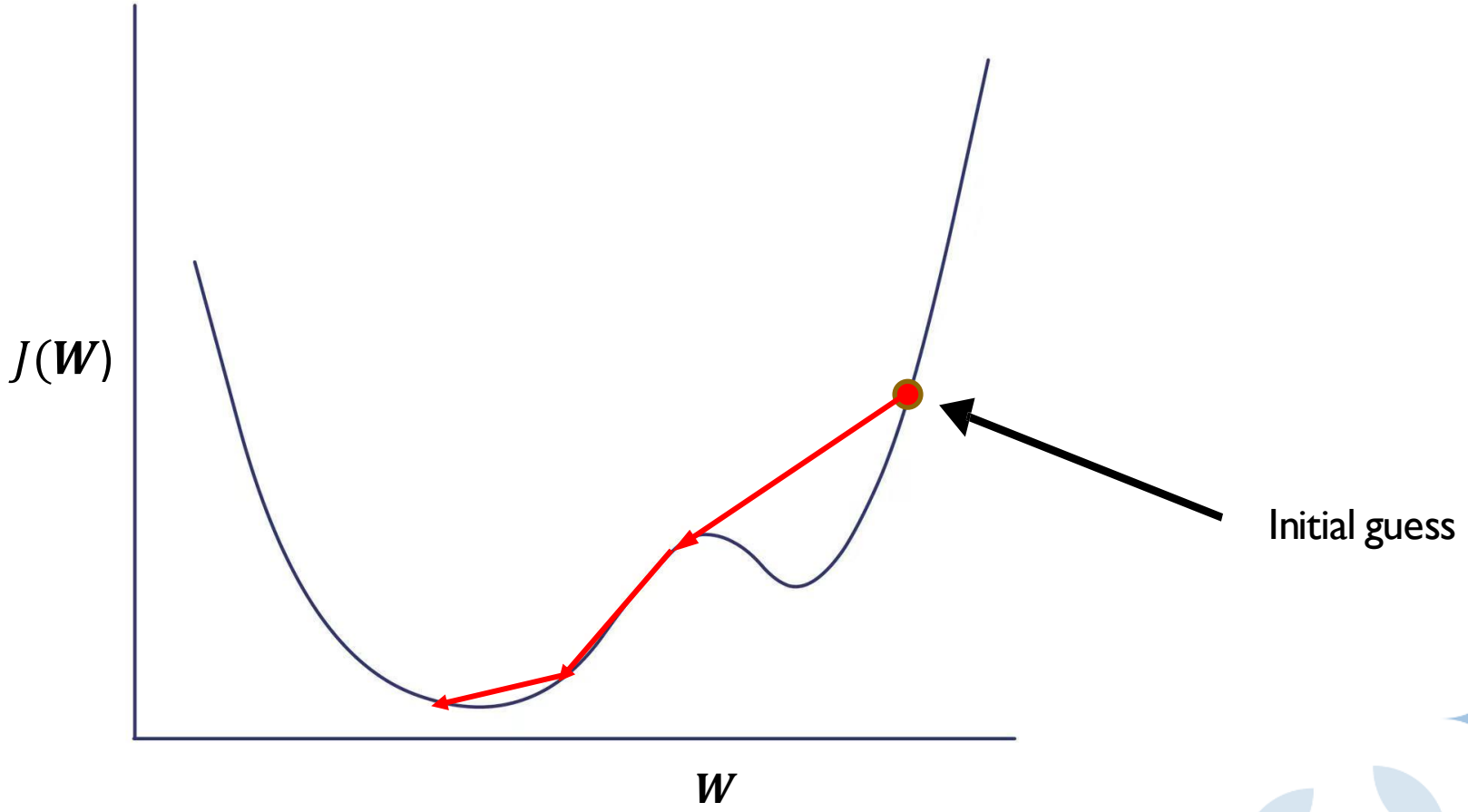
Question 3: How to choose Learning Rates?

Large learning rates overshoot, become unstable and diverge



Question 3: How to choose Learning Rates?

Stable learning rates converge smoothly and avoid local minima



Question 3: How to choose Learning Rates?

- Too large / small learning rates: bad performance
  - How to choose the best Learning Rates?
- Try a lot of different learning rates and see what is “just right”

Question 3: How to choose Learning Rates?

- Too large / small learning rates: bad performance
  - How to choose the best Learning Rates?
- Try a lot of different learning rates and see what is “just right”

- Or Do something smarter!
  - Design an adaptive learning rate that “adapts” to the landscape

Adaptive Learning Rates

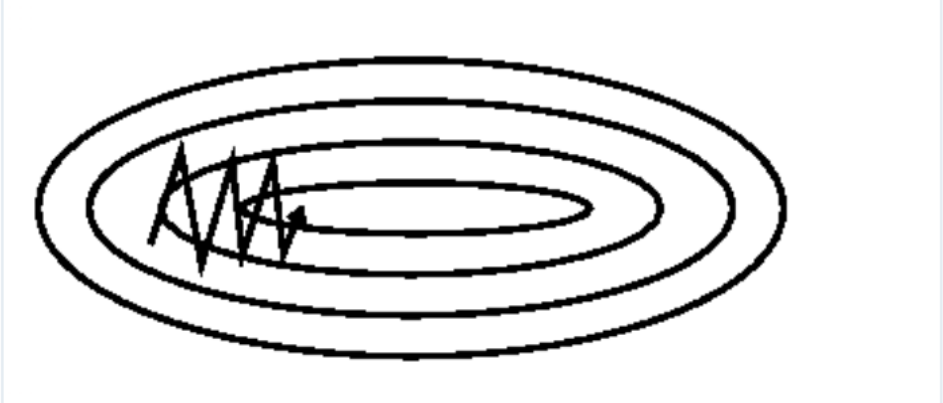
- Goal :Design an adaptive learning rate that “adapts” to the landscape
- Learning rate increases or decrease depend on
  - How large the gradient is
  - How fast learning is happening

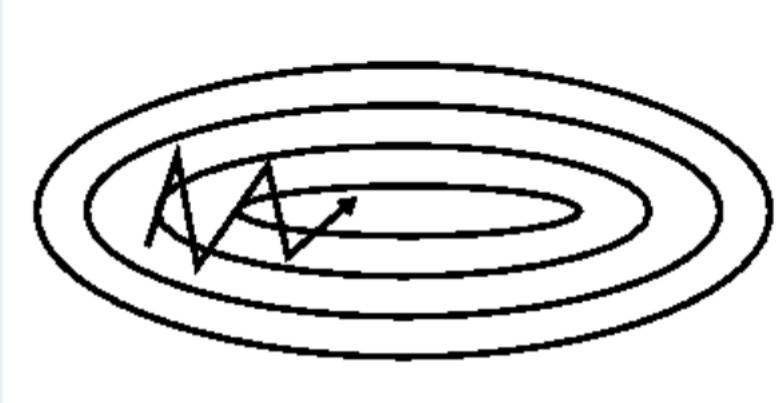
Adaptive Learning Rates

- Goal :Design an adaptive learning rate that “adapts” to the landscape
- Learning rate increases or decrease depend on
  - How large the gradient is
  - How fast learning is happening
- Popular algorithms:
  - Momentum / Nesterov accelerated gradient
  - Adagrad / RMSprop / Adam

Momentum / Nesterov accelerated gradient

- Stochastic Gradient Descent (SGD): $W_{t+1} \leftarrow W_t - \eta \nabla_w J(W_t)$
- Momentum (Qian,1999): $W_{t+1} \leftarrow W_t - v_t$,
  - where $v_t = \gamma v_{t-1} + \eta \nabla_w J(W_t)$





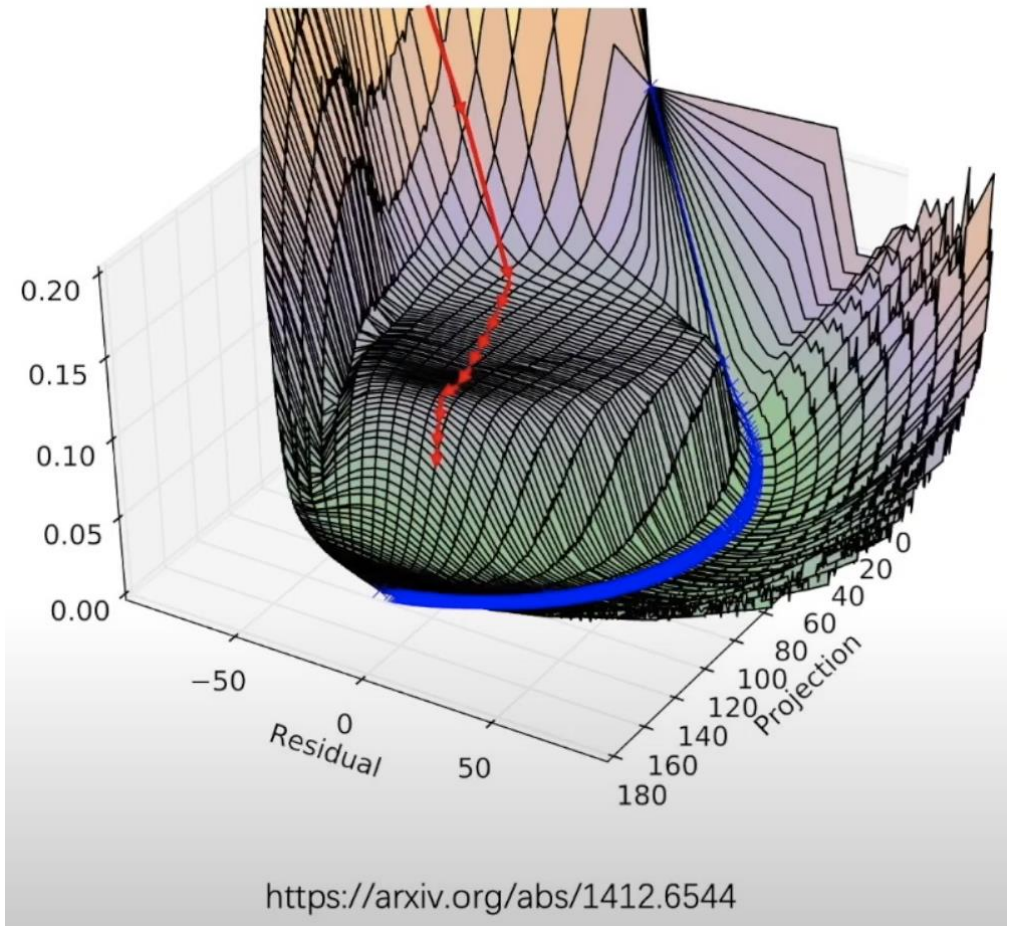
- Nesterov accelerated gradient (NAG), Yurii Nesterov, 1983

  $W_{t+1} \leftarrow W_t - v_t$, where $v_t = \gamma v_{t-1} + \eta \nabla_w J(W_t - \gamma v_{t-1})$

Adagrad / RMSProp

- SGD: $W_{t+1} \leftarrow W_t - \eta g_t$, where $g_t = \nabla_w J(W_t)$
- Adagrad (Duchi, Hazan, Singer, 2011):
  - $W_{t+1} \leftarrow W_t - \frac{\eta}{\sqrt{G_t + \varepsilon}} g_t$
  - $G_t = \sqrt{g_1^2 + g_2^2 + \cdots + g_t^2}$
  - Multi-dimension, $W_{t+1,d} \leftarrow W_{t,d} - \frac{\eta}{\sqrt{G_{t,dd} + \varepsilon}} g_{t,d}$

RMSProp



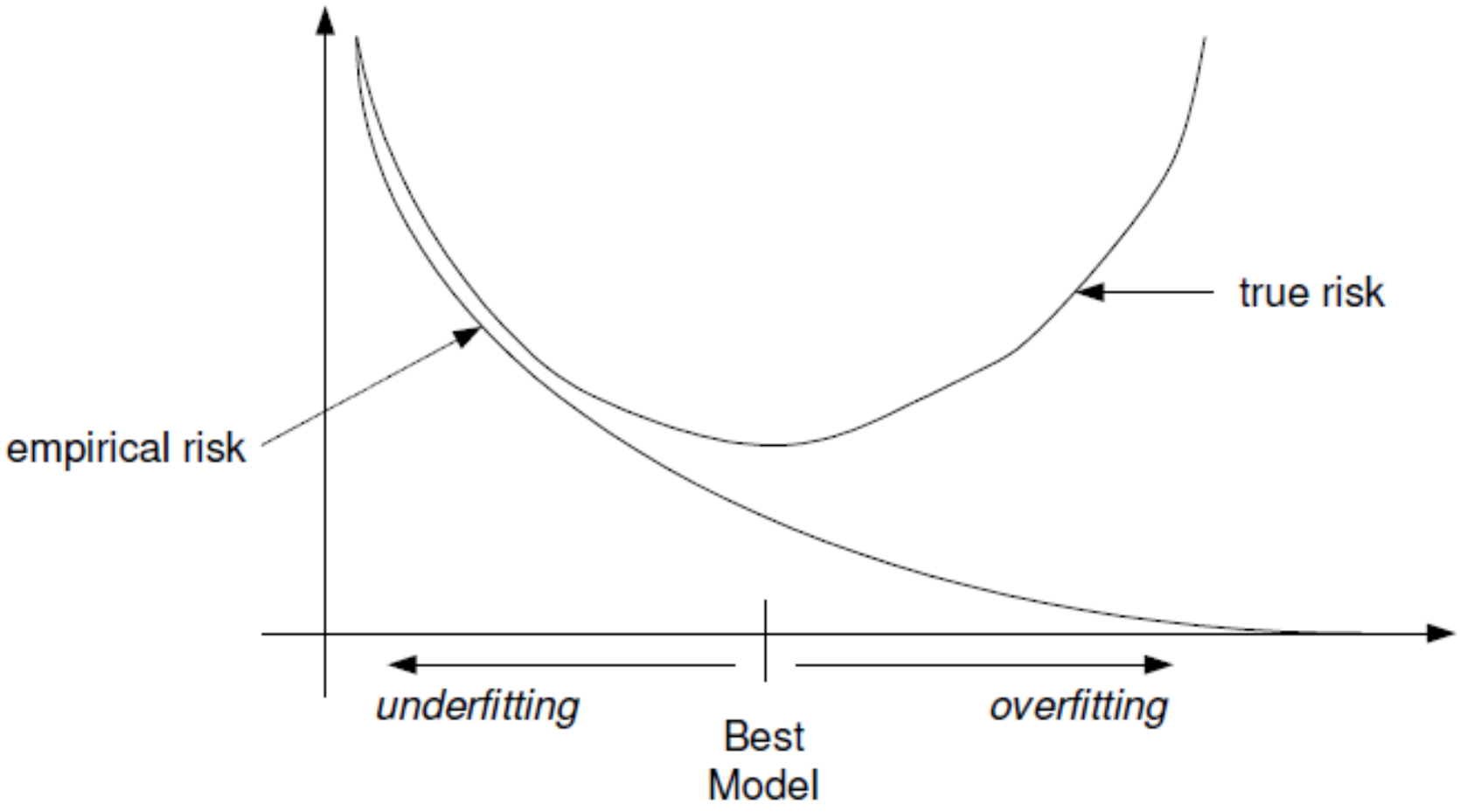
https://www.youtube.com/watch?v=spbBQshdhL4

- Adadelta (Zeiler, 2012)
- RMSProp (Hinton, unpublished)
  - extension to Adagrad
  - $G_t = \gamma G_{t-1} + (1-\gamma) g_t$
  - restricts the window of accumulated past gradients

Adam

- adds bias-correction and momentum to RMSprop
  - Momentum $v_t = \gamma v_{t-1} + \eta \nabla_w J(W_t)$
  - RMSProp $G_t = \gamma G_{t-1} + (1-\gamma) g_t$
  - Bias-correction $\hat{v}_t = v_t/(1-\eta)$, $\hat{G}_t = G_t/(1-\gamma)$
  - $W_{t+1} \leftarrow W_t - \frac{\eta}{\sqrt{\hat{G}_t + \varepsilon}} \hat{v}_t$
- Other Variations
  - AdaMax (Kingma and Ba, 2015)
  - Nadam (Dozat, 2016) Nesterov Momentum + Adam
  - AMSGrad (Reddi et al., 2018)
  - Vanilla SGD with annealing schedule
  - https://www.ruder.io/optimizing-gradient-descent
  - https://github.com/lilipads/gradient_descent_viz

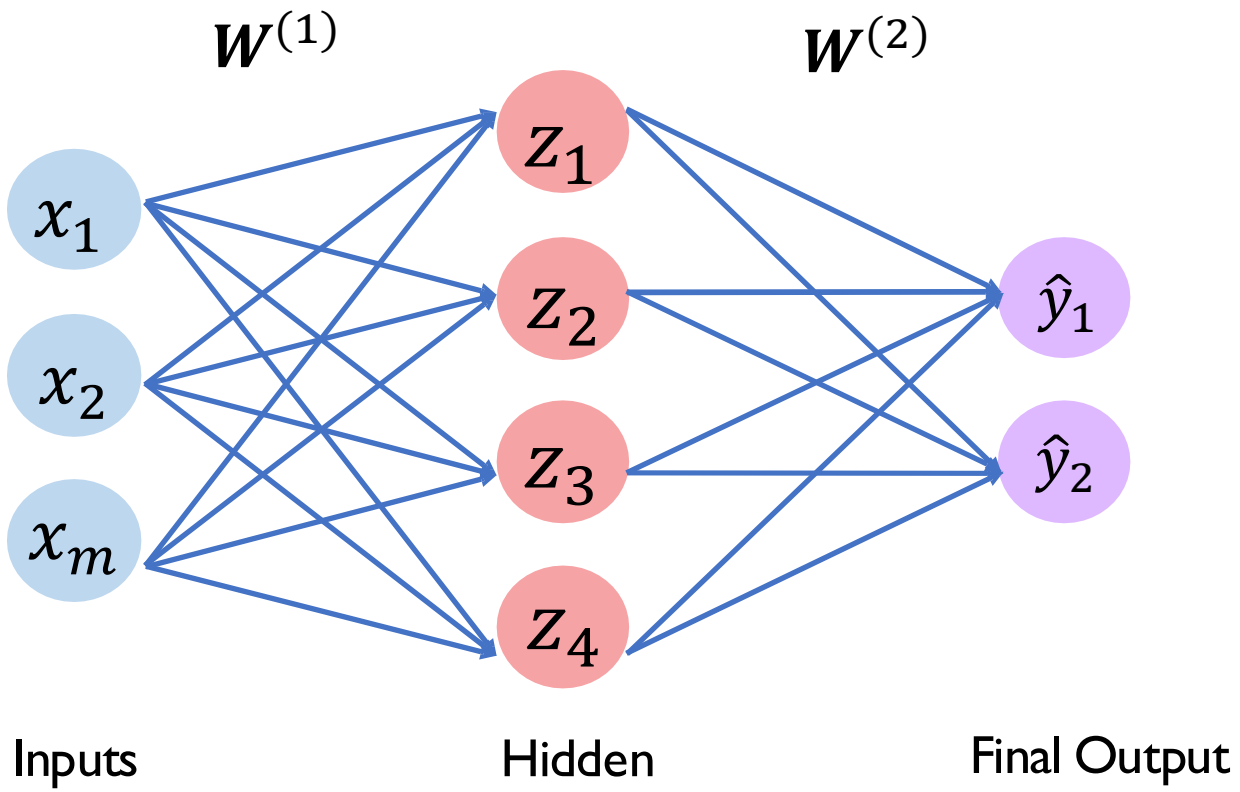
Question 4 When to stop?



Regularization

- Constrain the optimization problem to discourage complex models
- To Improve generalization of our model on unseen data

Regularization 1: Norm-Regularization



l_2-penalty, Ridge

$$\mathrm{Pen}(W) = \|W\|_2 = \sqrt{\sum_j |w_j|^2}$$

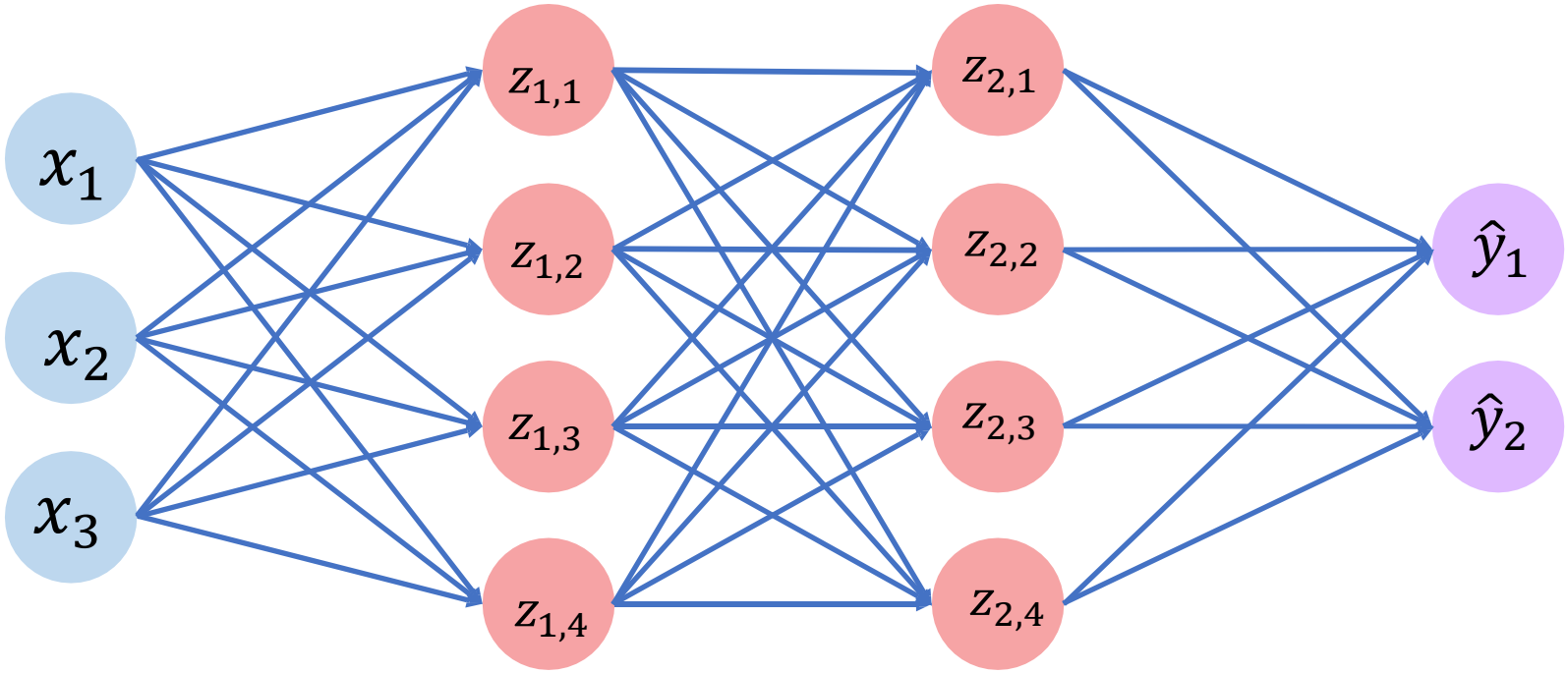
l_1-penalty, Lasso

$$\mathrm{Pen}(W) = \|W\|_1 = \sum_j |w_j|$$

$$J(W) = \frac{1}{n}\sum_{i=1}^{n} loss\left(f\left(X^{[i]}; W\right),\ Y^{[i]}\right) + \lambda\,\mathrm{Pen}(W)$$

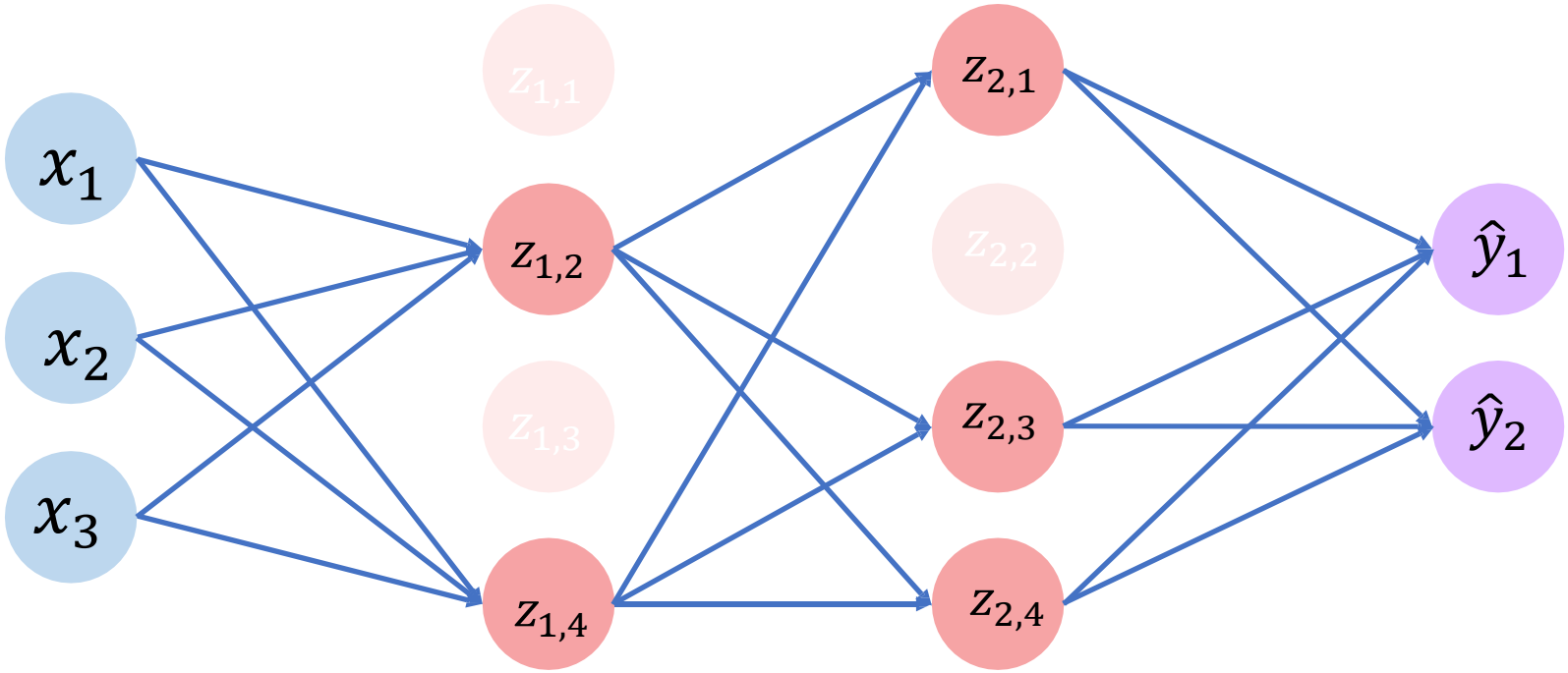
Regularization 2: Dropout

- During training, randomly set some activations to 0
  - *Improving neural networks by preventing co-adaptation of feature detectors*, (Hinton et al, 2012)



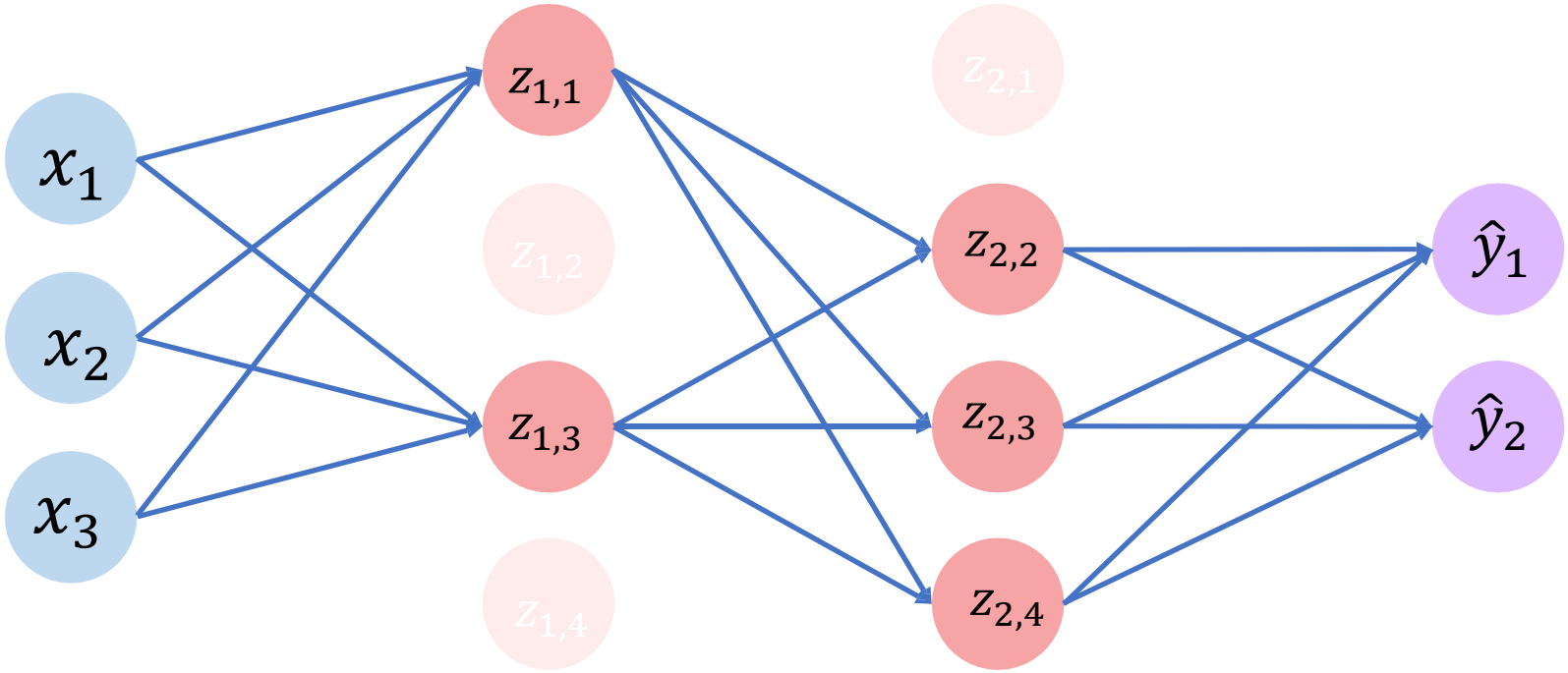
Regularization 2: Dropout

- During training, at each iteration, randomly set some activations to 0
  - Typically ‘drop’ 50% of activations in layer



Regularization 2: Dropout

- During training, at each iteration, randomly set some activations to 0
  - Typically ‘drop’ 50% of activations in layer

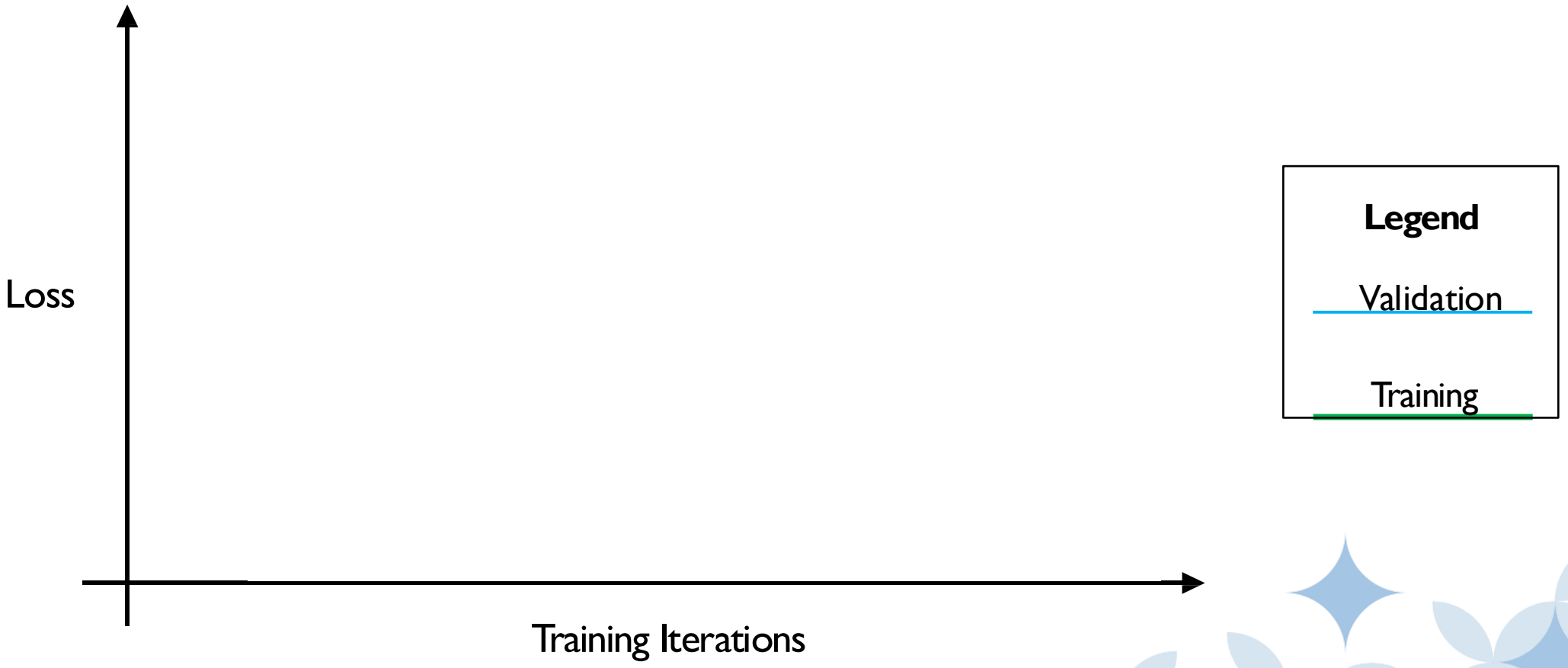


Regularization 2:Why Dropout work?

Dropout forces the network to make decision based on part of the features

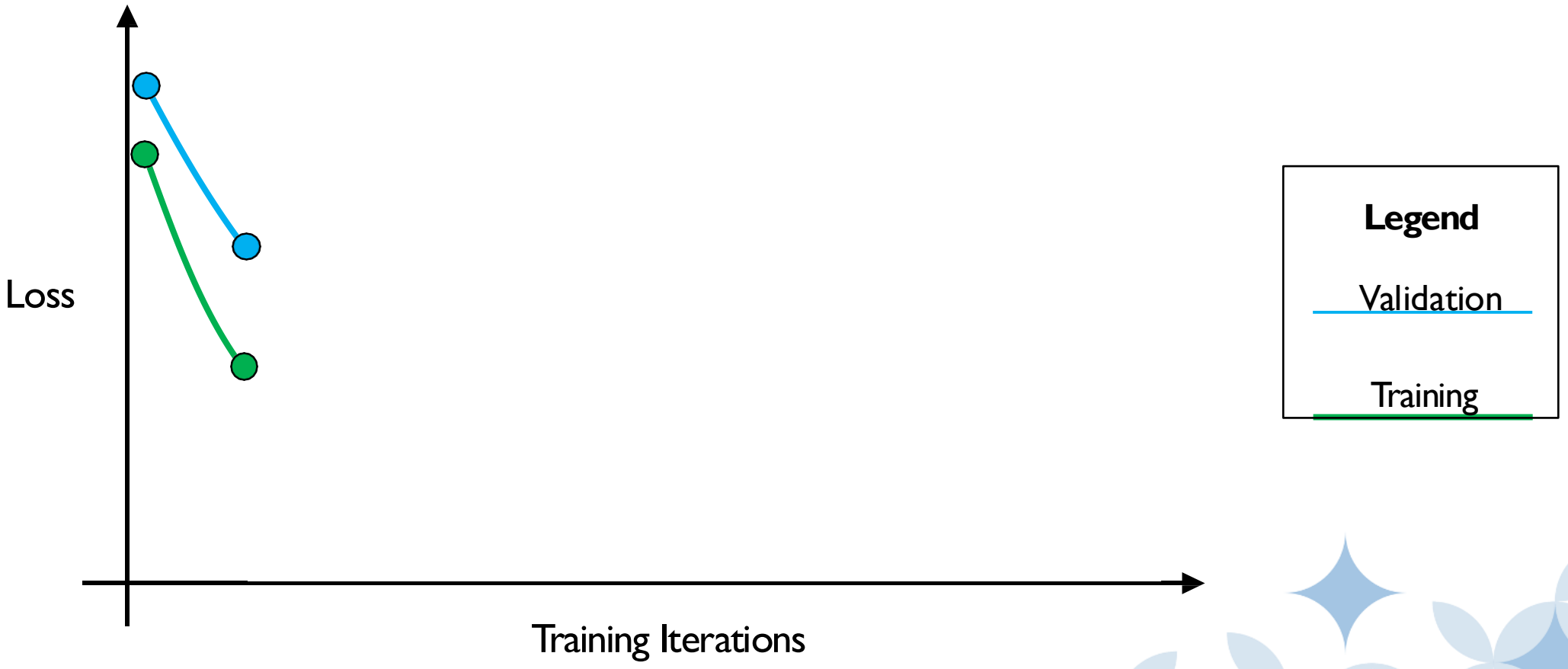
Regularization 3: Early Stopping

Stop training before we have a chance to overfit



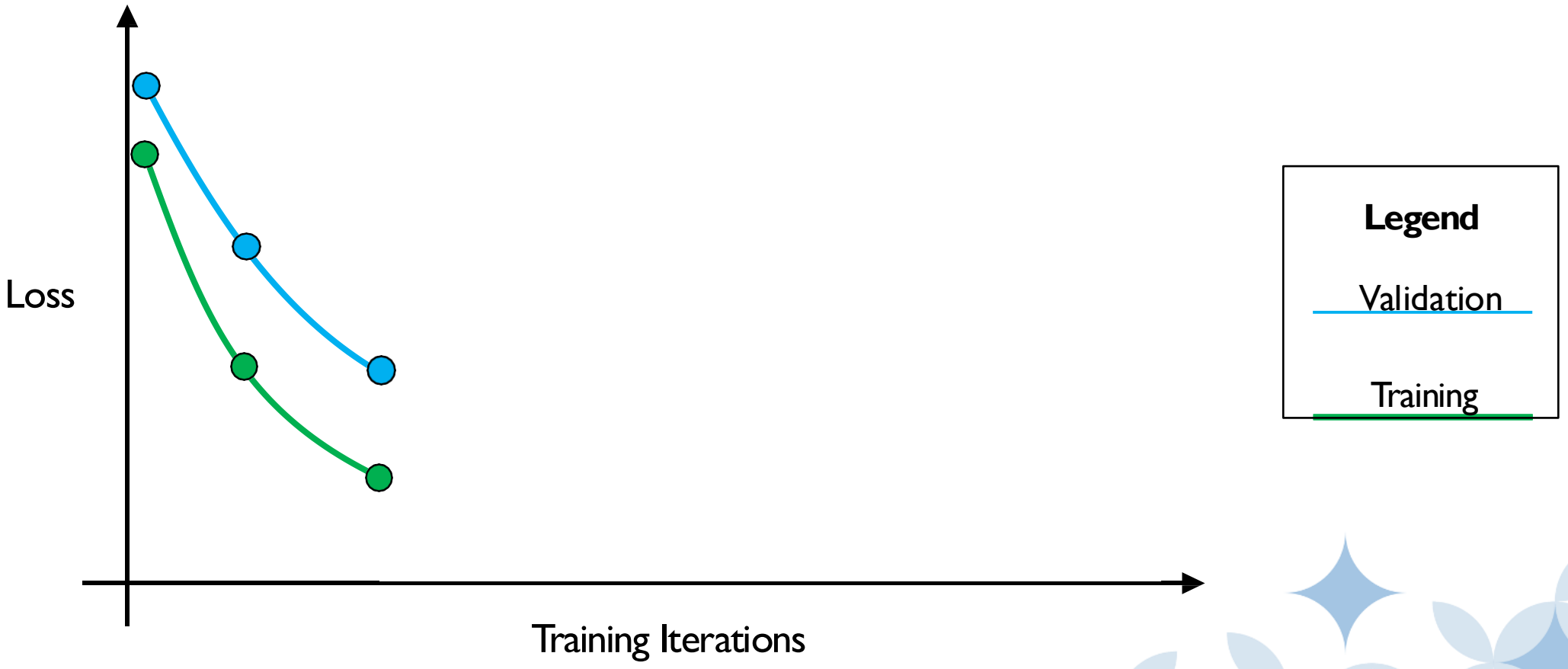
Regularization 3: Early Stopping

Stop training before we have a chance to overfit



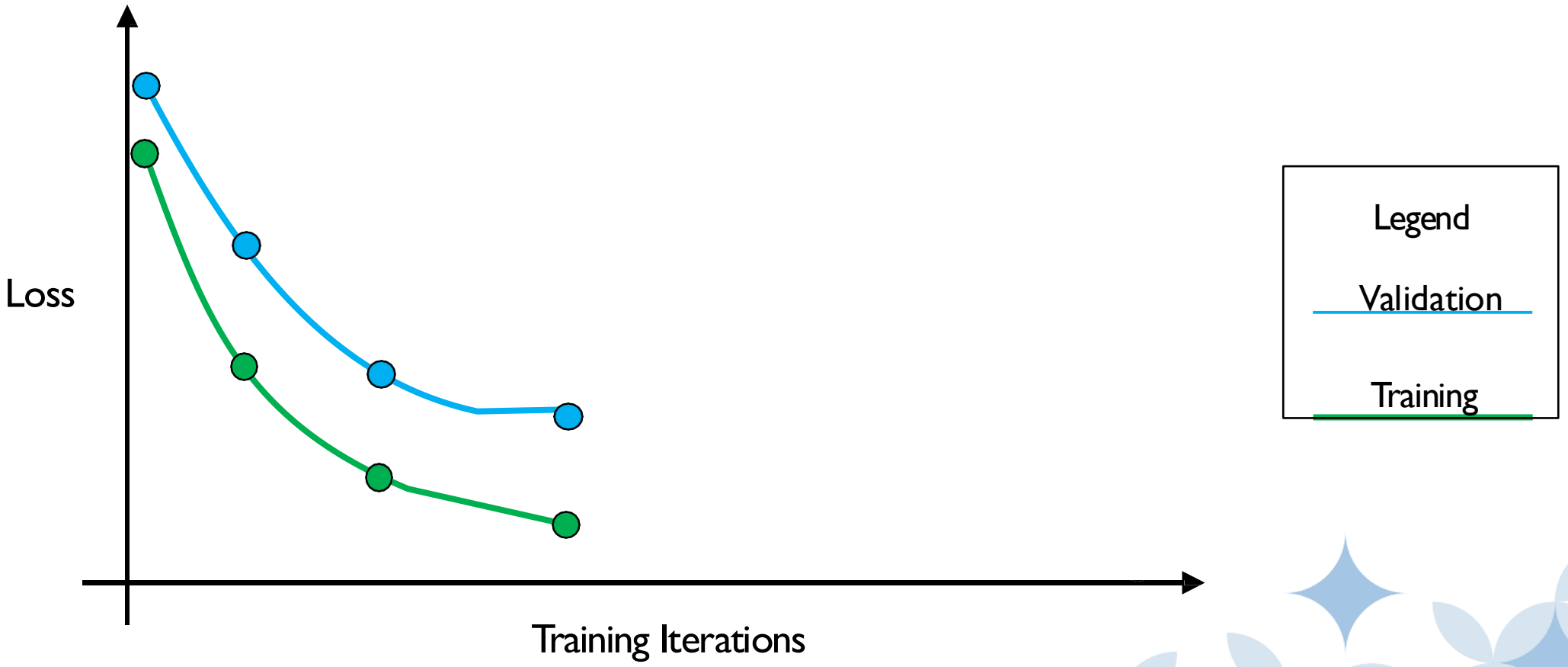
Regularization 3: Early Stopping

Stop training before we have a chance to overfit



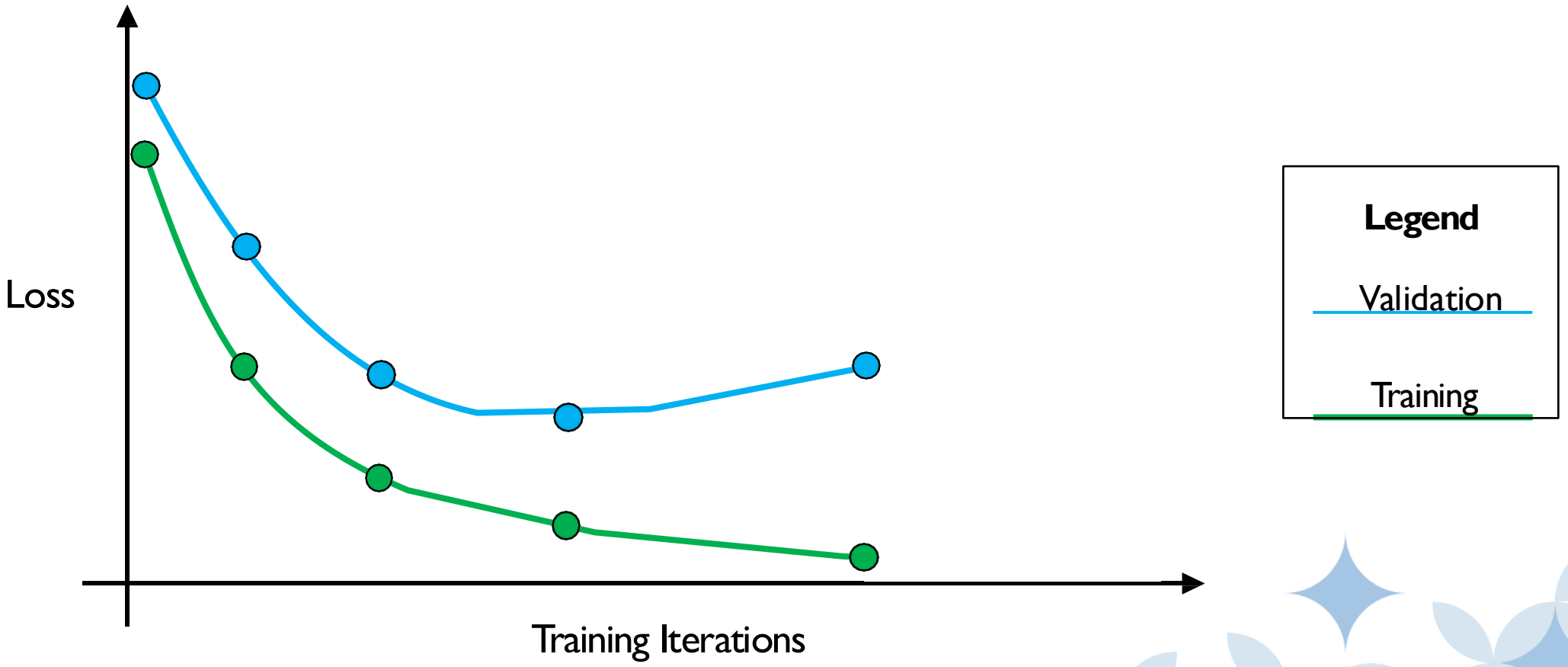
Regularization 3: Early Stopping

Stop training before we have a chance to overfit



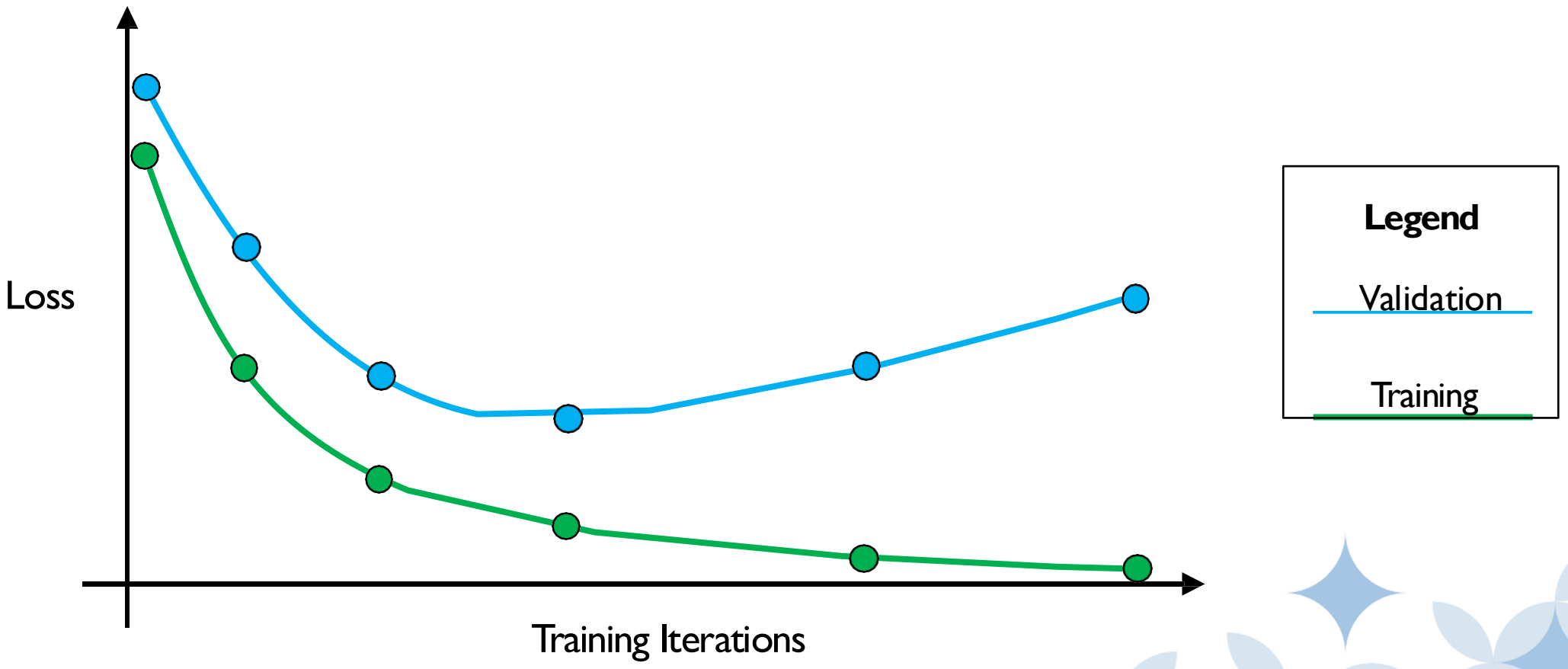
Regularization 3: Early Stopping

Stop training before we have a chance to overfit



Regularization 3: Early Stopping

Stop training before we have a chance to overfit

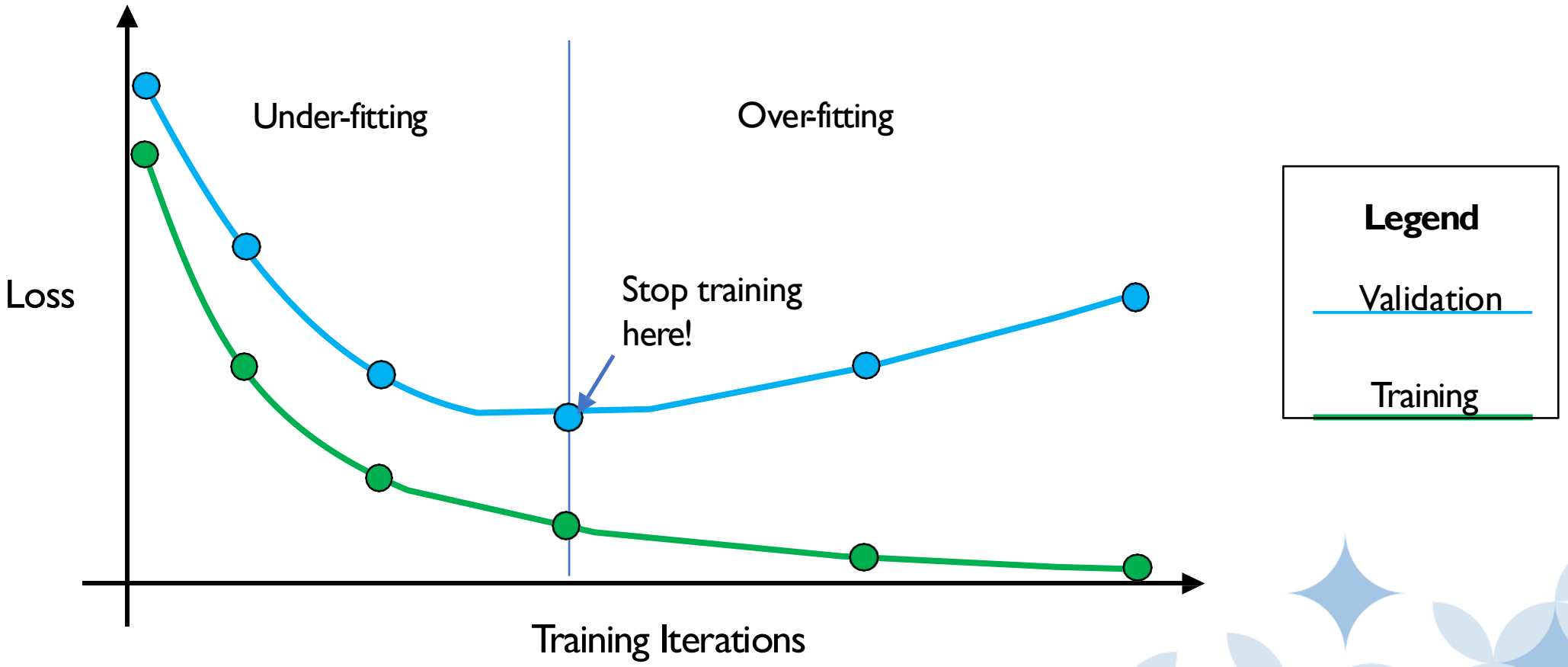


Regularization 3: Early Stopping

Stop training before we have a chance to overfit

Regularization 3: Early Stopping

Stop training before we have a chance to overfit



Questions?